杭州電子科技大学

硕 士 学 位 论 文

题 目：针对混合固态硬盘的读性能优化技术研究

研 究 生 赵育淼

专 业 计算机科学与技术

指导教师 贾刚勇 副教授

完成日期 2024 年 5 月

中国知网 https://www.cnki.net

杭州电子科技大学硕士学位论文

针对混合固态硬盘的读性能优化技术研究

研 究 生： 赵育淼

指导教师： 贾刚勇 副教授

2024 年 5 月

中国知网 https://www.cnki.net

Dissertation Submitted to Hangzhou Dianzi University
for the Degree of Master

Research on Read Performance Optimization Techniques for Hybrid Solid-State Drives

Candidate: Zhao Yumiao

Supervisor: AProf. Jia Gangyong

May，2024

中国知网 https://www.cnki.net

摘要

在大数据时代背景下，随着机器学习、大数据分析、NoSQL 数据库等读密集型应用的不断发展和普及，用户对高容量和高性能数据存储系统的需求日益增长，促使了高低密度闪存相结合的混合固态硬盘（Solid State Drive，SSD）这一新型存储架构的诞生。高密度闪存因其单个存储单元内能存储更多信息单位而减少了单位容量的存储成本，但同时也面临着访问延迟增加和使用寿命缩短等挑战。相比之下，低密度闪存位于高密度闪存的上层，提供更优的访问性能及读写缓存服务。尽管这种分层架构有效掩盖了高密度闪存在读写性能上的不足，但由于闪存介质的特有异地更新机制和低密度闪存空间有限，一旦低密度闪存空间被占满，数据更新就会被迫转移到读写速度较慢的高密度闪存中，从而降低了混合 SSD 在读取这些数据时的性能。针对混合 SSD 读性能下降的问题，现有研究通过数据预取技术和读缓存管理技术进行优化。然而，数据预取技术的低准确率及其与读缓存管理技术之间的不匹配，导致了对混合 SSD 读性能提升效果的局限。

为应对数据预取技术准确性低下及其与读缓存管理技术间协作不佳的挑战，本文提出了一种针对大型服务器环境中混合 SSD 读性能优化的全新方案，该方案包含以下两部分内容：

（1）为了最大化利用 RAM 在混合 SSD 中作为读写缓存的快速介质存储区域所提供的高效读写性能，本研究提出了一种基于模式局部性引导的数据预取（PLGDP）技术，旨在通过更精确及减少负面影响的方式，实现对页面数据的提前预取至 RAM，以应对未来的读访问请求，并快速响应这些请求。PLGDP 技术由两个组件构成：基于地址分段的数据预测算法（组件一），该组件利用时空局部性，周期性更新地址段集合以指导数据预测，并向请求队列发出预取请求；基于预取请求感知的 I/O 请求调度策略（组件二），通过限制最大可阻塞主机端 I/O 访问请求的预取请求数量来最小化对系统性能的负面影响。实验结果表明，与无预取机制（Baseline）、Linux 内置的基于马尔科夫状态转移矩阵的预测机制（Lynx）和顺序预取机制（SP）相比，PLGDP 在平均 RAM 区域命中率上分别提升了 335.92%、56.01%、68.81%，并在混合 SSD 的整体读性能上分别提高了 35.69%、19.14%、29.58%。

（2）考虑到 RAM 空间的有限性及预取数据未来访问时间的不确定性，本研究提出了一种创新的混合 SSD 读缓存管理技术，旨在与数据预取技术相适应，称为基于 RAM 和 SLC 的混合读缓存管理（HRCMRS）技术。该技术包含两个

中国知网 https://www.cnki.net

组件：基于访问感知的 RAM 读缓存管理策略（组件一），基于访问过程的 SLC 读缓存管理策略（组件二）。为了实现对 RAM 中页面访问请求的感知，以及跟踪页面从被缓存到 RAM 直到从 RAM 中剔除的整个过程，HRCMRS 技术对原有的访问请求信息格式及 RAM 内页面管理信息格式进行了调整，从而赋予了系统对访问过程的感知能力。组件一根据读取操作的具体请求及所牵涉页面的详细信息进行决策，以确定是保留页面在 RAM 中快速访问，还是将其转移至 SLC 读缓存中，以优化存储空间利用率和提高访问效率。组件二在 SLC 区域划分了一小块物理空间作为读缓存区，基于页面在 RAM 中的访问过程及页面属性进行决策，以判断是否将页面缓存在 SLC 读缓存中。这种方法旨在应对未来的读访问请求，从而最大化预取数据及读缓存数据的利用效率。实验结果表明，与无预取机制且采用基于 LRU 的 RAM 管理策略的混合 SSD（Baseline）、采用 PLGDP 技术且采用基于 LRU 的 RAM 管理策略的混合 SSD（PLGDP）、无预取机制且采用 HRCMRS 技术的混合 SSD(HRCMRS)相比，结合 PLGDP 与 HRCMRS 技术的混合 SSD（PLGDP+HRCMRS）分别提高 RAM 与 SLC 平均读数据比例之和 28.42%、55.48%、38.07%，并在提升混合 SSD 的平均整体读性能方面分别达到了 41.93%、7.90%、35.62%。此外，由于 SLC 读缓存区接收部分预取数据，预取数据得到了进一步的利用，相较于只采用预取技术的 PLGDP，PLGDP+ HRCMRS 平均提升 19.72%的预取数据利用率。

关键词：混合 SSD，时空局部性，访问感知，数据预取，读性能优化

中国知网 https://www.cnki.net

ABSTRACT

In the context of the big data era, with the continuous development and popularization of read-intensive applications such as machine learning, big data analytics, and NoSQL databases, the growing user demand for high-capacity and high-performance data storage systems has led to the birth of a new type of storage architecture, the hybrid solid state drive (SSD), which combines high- and low-density flash memory. High-density flash memory reduces storage costs per unit of capacity by storing more units of information in a single storage unit, but also faces challenges such as increased access latency and shorter lifespan. In contrast, low-density flash sits on top of high-density flash, providing superior access performance and read/write caching services. Although this layered architecture effectively masks the shortcomings of high-density flash in terms of read and write performance, due to the unique off-site update mechanism of flash media and the limited space of low-density flash, once the space of low-density flash is occupied, data updates are forced to be transferred to the slower read and write speeds of high-density flash, which degrades the performance of hybrid SSDs in accessing such data. To address the problem of degraded read performance of hybrid SSDs, existing research has optimized it through data prefetching techniques and read cache management techniques. However, the low accuracy of the data prefetching technique and its mismatch with the read cache management technique lead to the limitation of the effect on hybrid SSD read performance enhancement.

To address the challenges of low accuracy of data prefetching techniques and their poor collaboration with read cache management techniques, this paper proposes a novel scheme for hybrid SSD read performance optimization in large server environments, which consists of the following two parts:

(1) In order to maximize the efficient read/write performance provided by RAM as a fast media storage area for read/write caching in hybrid SSDs, this study proposes a Pattern Locality Guided Data Prefetching (PLGDP) technique, which aims to achieve early prefetching of page data to RAM for future read access requests and fast response to these requests in a more accurate and reduced negative impact manner. The PLGDP technique consists of two components: an address segment-based data prediction algorithm (Component I), which exploits spatio-temporal locality to periodically update

中国知网 https://www.cnki.net

the set of address segments to guide data prediction and issue prefetching requests to the request queue, and a prefetch request-aware I/O request scheduling policy (Component II), which minimizes the impact on the system performance by restricting the number of prefetching requests to the maximum number of host-side I/O access requests that can be blocked. number to minimize the negative impact on system performance. The experimental results show that PLGDP improves the average RAM region hit rate by 335.92%, 56.01%, 68.81%, and the overall read performance of hybrid SSDs by 35.69%, 19%, 19%, and 19%, respectively, compared to the no prefetch mechanism (Baseline), the Linux built-in prediction mechanism based on Markovian state transfer matrices (Lynx), and sequential prefetching (SP), respectively. improved by 35.69%, 19.14%, and 29.58%.

(2) Considering the limited RAM space and the uncertainty of the future access time of the prefetched data, this study proposes an innovative hybrid SSD read cache management technique designed to be compatible with the data prefetching technique, called Hybrid Read Cache Management Based on RAM and SLC (HRCMRS) technique. The technique consists of two components: an access-aware RAM read cache management policy (Component I) and an access-process-based SLC read cache management policy (Component II). In order to realize the sensing of page access requests in RAM and to track the whole process of pages from being cached into RAM until they are rejected from RAM, the HRCMRS technology adapts the original access request information format and the page management information format in RAM, thus endowing the system with the ability to sense the access process. Component I makes a decision based on the specific request of the read operation and the detailed information of the page involved to determine whether to keep the page in RAM for quick access or move it to the SLC read cache to optimize storage space utilization and improve access efficiency. Component 2 divides a small physical space in the SLC area as a read cache area, and makes decisions based on the access process of the page in RAM and the page attributes to determine whether to cache the page in the SLC read cache. This approach aims to cope with future read access requests, thus maximizing the utilization efficiency of prefetched data and read cache data. The experimental results show that compared with the hybrid SSD without prefetching mechanism and using LRU-based RAM management policy (Baseline), the hybrid SSD with PLGDP technology and using LRU-based RAM management policy (PLGDP), and the hybrid SSD without prefetching mechanism and using HRCMRS technology (HRCMRS), the

中国知网 https://www.cnki.net

hybrid SSD that combines PLGDP with the HRCMRS technology hybrid SSD (PLGDP+HRCMRS) improves the sum of the average read data ratio of RAM and SLC by 28.42%, 55.48%, and 38.07%, respectively, and achieves 41.93%, 7.90%, and 35.62%, respectively, in improving the average overall read performance of the hybrid SSD. In addition, as the SLC read cache receives some prefetched data, the prefetched data is further utilized, and PLGDP+ HRCMRS improves the prefetched data utilization by an average of 19.72% compared to PLGDP with prefetching-only technology.

Keywords: Hybrid SSD, Spatio-temporal Locality, Access Awareness, Data Prefetching, Read Performance Optimization

中国知网 https://www.cnki.net

目 录

第一章 绪论.....1
1.1 研究背景与研究意义.....1
1.2 国内外研究现状.....3
1.2.1 混合 SSD 体系架构设计研究现状.....3
1.2.2 混合 SSD 性能优化研究现状.....4
1.2.3 研究现状总结.....7
1.3 研究问题与主要内容.....8
1.4 本文组织架构.....10
第二章 相关技术概述.....12
2.1 传统 SSD 组织结构.....12
2.2 基于高密度闪存的混合 SSD.....13
2.2.1 高密度闪存.....13
2.2.2 SLC-TLC 混合 SSD 基本架构.....15
2.3 异构闪存的混合方式.....15
2.4 读性能优化.....17
2.4.1 数据预取.....17
2.4.2 I/O 请求调度.....21
2.4.3 缓存管理.....24
2.5 本章小结.....26
第三章 基于模式局部性引导的数据预取技术研究.....28
3.1 数据预取技术问题.....28
3.2 读访问特征分析.....30
3.2.1 时间局部性分析.....31
3.2.2 空间局部性分析.....35
3.3 基于模式局部性引导的数据预取技术.....39
3.3.1 基于地址分段的数据预测算法.....40
3.3.2 基于预取请求感知的 I/O 请求调度策略.....43
3.4 预取技术性能分析.....46
3.4.1 实验平台搭建.....46
3.4.2 实验环境.....48
3.4.3 实验结果与分析.....49
3.5 本章小结.....55
第四章 基于 RAM 和 SLC 的混合读缓存管理技术研究.....56
4.1 读缓存管理技术问题.....56

中国知网 https://www.cnki.net

4.2 混合 SSD 读访问数据位置分析 58
4.2.1 读数据来源分布情况 58
4.2.2 RAM 空间预取数据利用情况 60
4.3 基于 RAM 和 SLC 的混合读缓存管理技术 61
4.3.1 访问请求以及页面属性 63
4.3.2 基于访问感知的 RAM 读缓存管理策略 65
4.3.3 基于访问过程的 SLC 读缓存管理策略 68
4.4 读缓存管理技术分析 70
4.4.1 实验平台搭建 70
4.4.2 实验环境 71
4.4.3 实验结果与分析 72
4.5 本章小结 79
第五章 总结与展望 81
5.1 本文工作总结 81
5.2 后续工作展望 82
参考文献 84

中国知网 https://www.cnki.net

第一章 绪论

1.1 研究背景与研究意义

近年来，大数据时代下的互联网数据量的年增长率呈现出指数级的趋势，这种增长势不可挡。现代大型服务器和超级计算机集群提供了强大的计算能力来服务于例如机器学习、人工智能、大数据分析、高性能计算、NoSQL 数据库、云存储等领域。然而，随着计算需求量的增加，其所需访问的数据量也随之增长。针对于此类场景，大型服务器、超级计算集群不仅需要提供海量数据的存储服务，同时还要保证高带宽、低延迟等读写性能需求[1][2]。计算机数据存储层次按照访问速度、容量和成本等因素可分为四个主要层次：寄存器（Registers）、高速缓存（Cache）、主内存（RAM）以及辅助存储器（SSD/HDD）。图 1.1 为计算机存储层次架构示意图，其中访问延迟最大的辅助存储器是计算机存储系统领域优化工作的焦点之一。

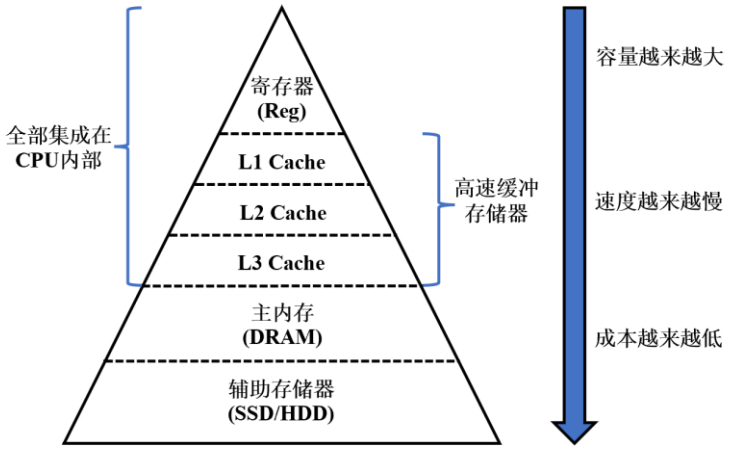


图 1.1 计算机存储层次架构

尽管磁盘驱动器（Hard Disk Drive，HDD）在很长一段历史时期都是数据存储的首选存储设备，但其固有的物理限制难以满足不断增长的性能需求。此外，机械磁盘存储方案在数据密集型场景下不仅会占用巨大的物理空间，同时也会增加大型服务器、超级计算集群的能耗与散热成本。因此，闪存（Nand Flash Memory）技术的兴起给大数据时代下的数据存储需求提供了新的解决选项。其作为一种非易失性存储介质（Non-Volatile Memory，NVM），具有无机械部件、快速读写、低功耗、轻薄便携、使用寿命长、无噪声和抗震动等优点。闪存的基本存储单元是双层浮栅晶体管（Floating Gate Transistor），这种晶体管采用了非挥发性存储原理，通过在晶体管中的浮栅上存储电荷来表示数据的状态。随着对闪存低成本、

中国知网 https://www.cnki.net

大容量需求的日益剧增，业界逐渐尝试在闪存单元中存储更多比特的数据，进而诞生了多种类型的闪存。从单级闪存单元（Single-Level Cell，SLC）到多级闪存单元（Multi-Level Cell，MLC），再到目前已经完成风险产品阶段的三级闪存单元（Triple Level Cell，TLC）[3]，最后到如今尝试广泛应用的四级闪存单元（Quad-Level Cell，QLC）[4]，闪存密度不断增加，存储单元的信息存储能力也不断提升。基于高密度闪存的固态硬盘驱动器（Solid State Drive，SSD）通过存储更多的数据，降低了数据中心的存储成本。随着技术的进步，高密度闪存单元的存储能力得以显著提升，为计算设备提供了更高效的存储解决方案，解决了海量数据存储的基本问题。

虽然高密度闪存具备了高容量和低成本的优势，但这些优点也伴随着在读写性能上的劣势。存储单元存储信息量的增加，意味着电位状态数越多，区分难度也就越大。这一设计不仅增加了访问延迟、擦除延迟、编程延迟，同时也缩减了存储单元的使用寿命。为了解决以上高密度闪存存在的局限性问题，生产商通过采用混合式架构，即通过在同一个固态硬盘内部采用两种及以上的存储介质，从而实现成本和性能之间的最佳平衡。目前主流的混合 SSD 是采用两种闪存介质（低密度闪存、高密度闪存）进行混合，这样固态硬盘内部就形成了两个区域：高性能分区和高密度分区。高性能分区一般采用 SLC 闪存，具有高性能的优点，但由于存储密度比较低，分区的整体容量较小。高密度分区通常由 TLC/QLC 闪存组成，具有大容量的优点，但由于存储密度增加，读写性能会相应下降。高性能分区作为高密度分区的写缓存区，这样既可以保证高速的写性能，又能降低高密度分区的数据更新频率，延长存储设备的使用寿命。混合式架构设计不仅掩盖了高密度闪存读写性能较差以及较短使用寿命的缺点，又降低了固态硬盘的存储成本。与此同时，根据高性能分区大小的可变性，混合式 SSD 又可分为静态混合式和动态混合式两种。静态混合式 SSD 中高密度分区大小是固定的，而动态混合式 SSD 中高密度分区大小是可以随着当前设备使用状态的变化而动态调整，以适配用户的需求。根据异构介质的混合方式，混合式 SSD 又可分为通道级（Channel Level）、芯片级（Chip Level）、单元级（Die Level）、平面级（Plane Level），不同混合方式的区别在于异构介质在哪个层级进行混合。

混合式存储设备的一个主要特点就是将高低密度闪存相结合，在兼顾低存储成本的同时，保证高速的即时读写性能，做到低响应延迟的用户无感知效果。这一特性使其得到了目前主流生产商家的青睐，迅速在消费市场获得普及。目前，基于 MLC/QLC 的混合式 SSD 已成为消费级终端的主要存储设备。各大闪存生产商相应推出了自家的混合式 SSD 设备，例如 Samsung 870 EVO（TLC）[5]、Samsung 870 QVO（QLC）[6]、Micron Crucial P5（TLC）[7]、WD Black SN850P

中国知网 https://www.cnki.net

（TLC）[8]、Intel 670p（QLC）[9]、Samsung 990 PRO（TLC）[10]。

传统的混合 SSD 性能优化工作的重点关注于写性能的优化，而读性能的优化关注较少。然而西部数据指出，现有的大型服务器、超级计算集群中，机器学习/强化学习/深度学习、边缘数据/大数据分析、SQL/NoSQL 数据库、云服务、金融监管与合规存储等服务的读数据操作时间占用总体服务时间的 90%以上，其读取操作强烈依赖于随机读取和顺序写入[4]。在以上服务器/超算集群中存储系统的读性能严重影响着服务质量、任务吞吐以及用户的使用体验。混合存储架构虽然掩盖了高密度闪存的缺点，但是仍然没有从本质上彻底解决数据从低性能区域读取所导致的性能问题，这也代表了混合 SSD 的读性能优化仍具有较大的优化空间。由此可见，在服务场景正在向读密集型场景转变的大趋势下，混合 SSD 迫切需要一种具有针对性且效果明显的读性能优化技术。针对混合 SSD 的测试分析与读性能优化技术设计不仅可以帮助开发者理解混合 SSD 的适用场景和使用特征；也能指导应用开发者避免可能存在的性能问题，提升用户使用体验；更能为存储系统、混合 SSD 的读性能优化指明前进方向。

1.2 国内外研究现状

本节主要讨论混合 SSD 国内外相关领域的研究，其主要可以分为两大类，即体系架构设计和性能优化。

1.2.1 混合 SSD 体系架构设计研究现状

混合 SSD 体系架构通常指的是一种设计理念，即在同一 SSD 单元内部采用两种或两种以上不同类型的存储介质。这种设计旨在结合各种存储介质的优点，比如提高性能、增加容量、降低成本或提高耐久性。闪存技术发展的过程中一个显著特点是闪存的密度不断增加，而闪存的延迟属性，包括读延迟、写延迟、擦除延时均有不同程度的上升，耐久性则不断下降。以目前比较流行的介质为例，MLC 与 SLC 相比，读延迟增加了 4 倍左右，写延迟增加了近 15 倍，使用寿命下降了 6 倍左右；TLC 与 SLC 相比，读延迟增加了 7 倍左右，写延迟增加了近 20 倍，使用寿命下降了 25 倍左右；QLC 与 SLC 相比，写延迟下降 30 倍左右，寿命下降了近 100 倍[11]。容量的提升带来的负面影响是性能的恶化，如何在提升容量和削减制造成本的同时，维持闪存存储设备的 I/O 性能，这一课题逐渐成为了学术界和工业界持续研究的课题。

为了探究解决上述问题的方法，研究人员尝试将单级闪存单元与多级闪存单元相结合使用。Chang 等人[12]尝试结合 MLC 闪存和 SLC 闪存，并设计了相应的数据防止策略以及磨损均衡，实验结果表明相较于传统单介质 SSD，整体性能提高了近 20%。不久之后，Chang 等人[13]首次提出了使用混合 SSD 控制器的概念，

中国知网 https://www.cnki.net

在他们的混合 SSD 控制器设计中，使用了一小块固定大小的 SLC 区域作为存储设备中主要存储介质 MLC 的数据缓存使用。Im 等人[14]也于同一时间段提出了类似的配备有固定大小高性能介质的混合 SSD 控制器设计。利用 SLC 作为写缓存前端设备，主机端的请求写入会立即写入到 SLC 中并返回写入操作完成，因为 SLC 属于非易失性存储，所以基本不会存在掉电数据丢失问题，而高密度闪存可以作为后端设备提供大容量的存储空间。

为了进一步深入探究组合性能，Matsui 等人[15]探究了 MLC/TLC NAND 闪存混合、SCM/MLC NAND 闪存混合和 SCM/MLC/TLC NAND 闪存三混合三种混合 SSD 的架构和算法设计概述（SCM 的英文全称是 Storage Class Memory，），并指出混合 SSD 在性能、耐用性和功耗方面相比仅使用 MLC NAND 闪存的 SSD 有所改进。

近些年来，研究人员开始针对 QLC、TLC 和 SLC 的混合使用展开研究。Shibata 等人[16]针对 QLC 与 SLC 混合使用，通过添加简单的预设命令，引入了 SLC 和 TLC 模式的灵活协议，便于系统实施。实验结果表明，这样做增加了系统设计的灵活性，使得系统能够根据不同的操作需求和性能要求，灵活地使用不同类型的存储单元，从而优化性能和成本。Taki 等人[17]分析了基于 QLC 的混合 SSD 最佳混合配置，该工作分析了 SLC+QLC、SCM+QLC、SCM+TLC+QLC、SCM+TLC 这四种组合类型。实验结果表明，在上述相同条件下，考虑成本的情况下，SCM+TLC+QLC NAND 闪存三重混合配置在 SSD 性能上是最佳选择。

与此同时，可转换 QLC SSD 是也一种创新的存储解决方案，它在传统的固态硬盘技术上迈出了重要一步。这种类型的 SSD 通过允许用户根据实际需求动态调整闪存单元的工作模式，从 SLC（单层单元）模式转换到 QLC（四层单元）模式，或反之，增强了存储设备的灵活性和性能。在 SLC 模式下，每个单元存储一位数据，提供最快的读写速度和最高的耐用性，适合高性能的应用场景。而在 QLC 模式下，每个单元存储四位数据，增加了存储密度，降低了成本，适合存储不频繁访问的数据。这种转换能力使得 SSD 可以在高性能和高容量之间进行灵活切换，根据当前的工作负载和性能要求，自动优化存储资源，这不仅有助于提高系统的整体性能，也能延长 SSD 的使用寿命，并可能降低长期的运营成本。在实际应用中，Intel 665P[18]和 Micron Crucial P1[19]保留了多达 20%的容量，用于 SLC 和 QLC 模式之间的这种转换。

1.2.2 混合 SSD 性能优化研究现状

混合 SSD 性能优化指的是针对已有混合 SSD 体系架构以及其应用场景，通过对其内部的功能模块进行优化，达到提高混合 SSD 读写性能和延长其使用寿命的目的。

中国知网 https://www.cnki.net

（1）混合 SSD 读性能优化

混合 SSD 读性能优化指的是通过改进技术和策略，提高混合固态硬盘在处理读取操作时的速度和效率。这可能包括数据分层管理，将频繁访问的数据存储在更快的存储介质上；利用高速缓存技术减少数据访问延迟；以及采用先进的算法和控制策略来优化数据的读取路径和减少读取等待时间。目的是在不同类型的存储介质之间实现最佳的数据存取平衡，从而提升整体系统性能。

Luo 等人[20]针对现有 SLC/QLC 混合 SSD 架构，提出了 HyFlex 策略以优化混合 SSD 性能和可靠性，具体包括基于速度的 I/O 调度（VIS）、垃圾回收感知的容量调整（GCT）、以及读扰动感知的数据迁移（DAM）。VIS 策略通过识别新写入数据的速度和调度请求来减轻 SLC 和 QLC 区域之间的迁移成本。GCT 策略通过从 QLC 区域租用块来避免 SLC 区域的低效垃圾回收。DAM 策略通过将频繁读取的数据迁移到 SLC 区域来减少 QLC 区域的读扰动。实验结果表明该策略可以有效提高混合 SSD 的读性能。Chang 等人[21]指出，在 MLC 闪存中，页面的读取延迟会随着页面类型的变化而变化。一旦读密集型数据位于具有较高访问延迟的页面类型中，负载的读取性能势必会受到明显的影响。为了提升读取性能，研究人员提出 Fastread 方案将读密集型数据迁移到具有较低访问延迟的页面中来提升读取性能。Shi 等人[22]提出了一种针对高密度 NAND 闪存存储系统的读取延迟变化感知性能优化方案，旨在加速数据和元数据访问以最大化读取性能并减少尾部延迟。该方案包括：读取延迟变化感知的数据放置策略、新的缓存管理方法以及生命周期感知的元数据放置策略。实验结果显示，与现有技术相比，该方法提高了整体读性能，同时尾部延迟也得到了减少。

除此之外，数据预取也是一种提升混合 SSD 读性能很有潜力的方法。数据预取作为提升混合 SSD 读性能的策略，通过智能地预测并提前加载即将被访问的数据到缓存中，展现出巨大的潜力。这种方法不仅减少了数据访问延迟，还有效提高了系统的整体效率和响应速度。在混合 SSD 中，结合高速和高容量存储介质的特点，数据预取可以进一步优化数据存取策略，实现性能与存储容量的最佳平衡。通过分析应用程序的访问模式和行为，数据预取能够精准地识别哪些数据将被需要，从而在提高读取速度的同时，降低能耗和延长设备寿命。然而，目前研究中数据预取技术（启发式预取和神经网络预取）大多是针对于单一闪存介质或混合存储系统（例如 SSD 和 HDD），但是预取技术在混合 SSD 中的应用潜力是不容小觑的。

Xu 等人[23]提出了一个高效的、资源优化的学习预取器，用于固态硬盘（SSD）中，以提高预取的命中率，减少误预取的惩罚，并在考虑 SSD 资源限制的情况下提高读取性能。这个预取器基于 Markov 链的学习算法进行优化，通过更好的

中国知网 https://www.cnki.net

访问模式预测，旨在减少资源使用，同时增加命中率和降低误预取的概率。实验结果表明预取器的引入大大减少了 SSD 的访问时间。Chakraborttii 等人[24]提出了一种基于神经网络的方法来改善 SSD 中的数据预取性能。具体来说，该方案利用神经网络学习 I/O 访问模式，并关注如何准确预测即将访问的数据块。实验结果表明，基于神经网络的数据预取具有良好的准确性。Yang 等人[25]利用 CXL（Compute Express Link）技术和廉价的 SSD 克服内存墙的可行性。通过探索 CXL-enabled 闪存设备的设计空间，展示了缓存和预取技术如何帮助减轻闪存性能和寿命的担忧。实验表明，采用缓存策略和预取技术的联合工作可以有效实现 SSD 达到 DRAM 的性能。

（2）混合 SSD 写性能优化

混合 SSD 写性能优化涉及使用策略和技术提升混合固态硬盘(SSD)在处理写入操作时的效率和速度。这通常包括数据分层，以确保频繁写入的数据首先存储在耐用性更高的介质上；采用写入缓存技术减少写入延迟；实施智能的垃圾回收和磨损均衡算法以减少写入放大效应；以及优化固件和控制算法来提高数据写入路径的效率。目标是在保持数据安全和耐用性的同时，最大化写入性能和存储介质的使用效率。

Park 等人[26]提出了一个新的数据分类策略，名为 MiDA（基于迁移次数的数据生命周期分类），用于提高基于闪存的 SSD 的性能和寿命。MiDA 利用数据在闪存块中的移动模式来分类数据，减少内部数据复制，从而提高设备的性能和寿命。该策略在设备固件层实现，不需要对主机系统进行修改，实验结果表明，与现有技术相比，MiDA 能够显著减少写放大因子（WAF）和提高吞吐量。未来的工作计划考虑垃圾回收算法对 MiDA 性能的影响，并测试其在不同垃圾回收算法下的有效性。Zhou 等人[27]提出了一种新颖的 SSD 设计，称为 Remap-SSD，以减少高密度/混合 SSD 重复写入操作，其具有两个显着特征。首先，它提供了重映射原语，允许主机软件和 SSD 固件以几乎零成本执行重复数据的逻辑写入。其次，采用混合存储架构来保持映射一致性。小字节可寻址非易失性 RAM (NVRAM) 用于以日志结构方式保存重新映射元数据，并与闪存协同管理。实验结果表明，Remap-SSD 可以充分发挥地址重新映射的潜力，减少重复写如数据量以及通道争用，进而提高 SSD 写性能。

（3）混合 SSD 使用寿命延长

混合 SSD 使用寿命延长是指通过采取一系列技术措施和策略来延长混合固态硬盘（SSD）的有效使用期限的过程。这包括采用耐用性更高的存储介质，如单层单元（SLC）NAND 闪存；实施智能的数据分层和写入放大减少技术；以及优化垃圾回收和磨损均衡算法，以均匀分配写入操作，减少对特定存储单元的磨

中国知网 https://www.cnki.net

损。这样做有助于减少因频繁写入操作导致的存储介质磨损，从而延长 SSD 的总体使用寿命。

Cho 等人[28]针对于 SLC 与 QLC 相结合的混合 SSD 设计一种基于机器学习的数据分类器，用于分类并放置数据，以减少 SLC 与 QLC 之间不必要的数据交换。实验表明，其方法可以有效延长混合 SSD 的使用寿命，混合 SSD 的整体性能也得到了提升。Yao 等人[29]提出了一个名为 HDFTL 的按需闪存转换层算法，用于构建带有 SLC 和 MLC 闪存芯片的混合固态硬盘(SSD)。这个算法解决了 SLC 和 MLC 区域统一地址映射、数据分配和 SLC 与 MLC 区域之间的磨损均衡问题。HDFTL 采用了基于页面级别的地址映射方案，并分割缓存映射表以准确识别热数据，还通过相对磨损率动态调整热写入数据识别和数据分配的缓存映射表大小，从而在 SLC 和 MLC 区域之间实现更好的磨损均衡。实验表明，具有最小的存储开销，且可以延长使用寿命。Shi 等人[30]针对高密度和低成本的闪存内存（如四层单元 QLC 闪存）进行了优化设计。研究的主要发现是，现有架构中 SLC（单层单元）和 QLC 模式之间的协调设计还不够完善。因此，论文提出了两种优化策略：基于速率的输入/输出调度（VIS）和垃圾回收（GC）感知的容量调整（GCT）。实验结果表明，混合 SSD 的性能和寿命可以通过灵活的数据布置和闪存模式管理策略得到全面优化。

1.2.3 研究现状总结

在现有的混合 SSD 研究中，体系架构设计主要聚焦于探索不同异构闪存介质的组合方式及其在架构各层级中的配置。目前，基于 QLC 闪存介质的混合方式已获得全面研究，未来研究将进一步探讨更高密度的闪存介质组合。在写性能优化方面，现有方法主要基于数据访问特性与闪存介质的匹配来提升性能，这些方法虽有细微差别但核心理念相似。为延长混合 SSD 的使用寿命，研究着力于减少写放大现象，并优先使用可擦除次数较多的闪存介质承担更多写操作，这一研究方向已较为成熟。然而，关于混合 SSD 读性能优化的研究仍显不足。

首先，当前主流的读性能优化策略主要集中于高精度数据预取和高效的读缓存管理两个核心领域。数据预取技术主要采取两种方法：一是基于复杂的启发式算法，通过分析和识别数据的读取模式进行预测，虽然可以实现较高的预测准确率，但需要占用大量存储资源以记录这些模式；二是应用计算成本较高的神经网络模型，利用深度学习技术对访问模式进行学习和预测，尽管预测精度较高，但需求于较大的计算资源。这两种方法都因其在计算和存储资源上的高开销，在性能受限的 SSD 控制器上难以实施。因此，迫切需要研发一种计算和存储开销较低的轻量级数据预取技术。

其次，少有研究深入探讨数据预取技术与读缓存管理策略之间的协同作用。

中国知网 https://www.cnki.net

在目前的研究架构中，简单地将数据预取与读缓存管理相结合可能引发对复杂数据类型的处理难题。具体来说，传统的读缓存管理通常仅依据数据访问频率进行缓存，未能区分数据是直接由主机请求，还是通过预取技术预加载的。这种不区分来源的处理可能导致不适宜的缓存决策，降低缓存命中率。同时，缺乏针对预取数据特性的有效感知，可能会使得预取数据未被充分利用，进一步影响预取效率。长期来看，这种做法不仅削弱了数据预取技术的表现，还可能负面影响混合 SSD 的整体性能。

综上所述，针对混合 SSD 读性能优化仍具有很大的研究空间，关键需集中于开发轻量且高效的数据预取技术，以及研究适应预取技术引入的混合 SSD 读缓存管理策略。

1.3 研究问题与主要内容

在关键技术领域，如数据中心、大数据分析以及云计算等，数据访问速度是直接决定系统整体性能和用户体验的关键因素。随着读密集型应用场景的不断增多，本研究旨在深入探讨和研究针对混合 SSD 的读性能优化技术，特别是在混合 SSD 应用场景中数据预取技术和读缓存管理技术的应用与挑战。当前，针对这两项技术在混合 SSD 中的应用，存在以下问题：

（1）高精度且低开销的预取技术设计问题。对于混合 SSD 而言，其需同时服务多个并发工作负载，这要求预取技术能够准确记录并识别来自不同工作负载的读访问请求流，并据此执行适当的预取操作。然而，不同工作负载间的读访问特性可能存在显著差异，并且这些特性及其访问模式是动态变化的，给预取技术的实施带来了显著挑战。为了提升预取技术的准确性同时降低其引入的额外开销，有必要实时监测并分析各工作负载的访问情况，以识别其特定的访问特点。现有的基于神经网络的预测算法可能在提高预测准确性方面表现出优势，但这通常伴随着复杂的数据预处理和显著的计算开销，这对于计算能力有限的混合 SSD 来说可能不切实际。此外，预取操作的执行效率同样至关重要，因为如果预取操作阻塞了主机端的访问请求，可能会对用户体验产生不利影响。综合考虑，如何基于历史访问请求以精确且开销低廉的方式预测未来的数据访问，并在恰当时机执行预取操作，成为了提高预取技术性能的关键。

（2）引入预取技术的混合 SSD 读缓存管理设计问题。在传统的 RAM 缓存管理策略中，缓存空间被细分为读缓存区和写缓存区，并分别进行管理。引入预取技术后，作为一种读类型数据的预取数据被加入到 RAM 的读缓存区，这要求现有的 RAM 读缓存空间管理技术对于更加多样化的数据类型进行适应性调整。此外，由于预取数据的未来访问时刻不确定，使得在有限的 RAM 读缓存空间内实现所有预取数据的命中极为困难，预取数据很容易在实际被访问前就被从缓存

中国知网 https://www.cnki.net

中剔除。因此，RAM 读缓存管理的设计需深度整合预取技术，明确区分主动读访问数据与被动预取数据，而非仅将后者作为普通读类型数据进行处理。考虑到混合 SSD 中异构闪存介质的性能差异显著，以及从 RAM 区域剔除的页面如果直接丢弃将导致预取数据资源的浪费，因而在混合 SSD 的高性能区域划分专用空间以二次利用被剔除的预取及读缓存数据，显得尤为必要。这种策略不仅优化了数据的利用效率，也为混合 SSD 的缓存管理带来了新的设计思路。

针对上述所讨论的混合 SSD 中数据预取技术和读缓存管理技术设计存在的问题，本研究深入探讨了这两个领域，并提出了一种将数据预取技术与缓存管理技术深度整合的创新解决方案。本文所提出的研究框架，旨在通过系统化的方法优化混合 SSD 的性能，具体研究内容和方法论将在后续章节中详细展开，图 1.2 展示了本文研究框架结构图。

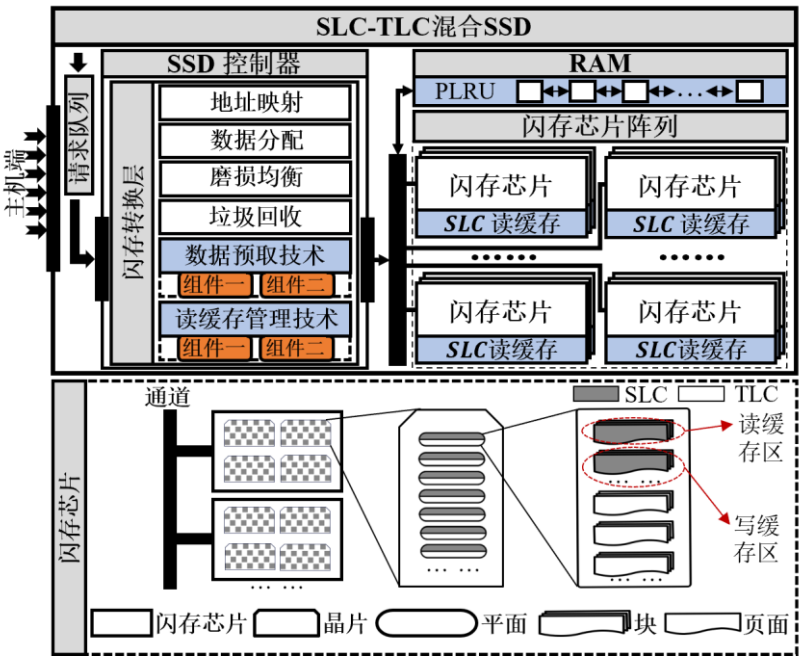


图 1.2 本文研究框架图

（1） 针对数据预取技术设计，提出了基于模式局部性引导的数据预取（PLGDP）技术。PLGDP 技术由两个组件构成：组件一为基于地址分段的数据预测算法，组件二为基于预取请求感知的 I/O 请求调度策略。首先，组件一负责实时监控所有进入混合 SSD 的读访问请求，并定期对这些历史访问记录执行地址分段操作，以形成地址段集合。依据这一地址段集合及当前的读访问请求，该组件能够预测未来可能被访问的数据，并据此生成预取请求，将其加入到请求队列中以待处理。随后，组件二通过预取请求及主机端 I/O 请求的综合感知，执行请求调度工作。它通过设定主机端 I/O 请求可被预取请求抢占的最大数量限制，旨在最小化预取操作对主机端请求可能造成的负面影响。同时，该策略也致力于

中国知网 https://www.cnki.net

提升 RAM 中读访问数据的占比，进而优化混合 SSD 的整体读取性能。

（2）针对读缓存管理技术设计，提出了基于 RAM 和 SLC 的混合读缓存管理（HRCMRS）技术。HRCMRS 技术由两个关键组件构成：组件一为基于访问感知的 RAM 读缓存管理策略，组件二为基于访问过程的 SLC 读缓存管理策略。首先，组件一负责管理 RAM 区域内的读缓存，它通过监测缓存中页面的属性及其被访问的请求类型，执行相应的页面管理操作（如图 1.2 所示的 RAM 内的 PLRU 机制），旨在最大化地保留那些未来可能频繁被访问的页面。其次，组件二在 SLC 缓存区内划分出特定的空间作为 SLC 读缓存区，以便接收从 RAM 读缓存区中淘汰的页面。在此环节，组件二管理 SLC 缓存区内的读缓存，通过分析从 RAM 读缓存区淘汰的页面类型及其在 RAM 中的访问过程，决定是否将这些页面缓存至 SLC 读缓存区以供二次利用，并执行相应的管理策略。通过这两个组件的协同工作，本技术能够显著提升混合 SSD 中 RAM 及 SLC 的读访问比例，从而有效提高混合 SSD 的整体读取性能。

1.4 本文组织架构

本文的组织架构如图 1.3 所示，由以下五部分组成：

第一章：绪论。本章首先阐述了研究背景和研究意义，接着对当前在相关领域的国内外研究现状进行了系统性回顾。继而，明确提出了本文旨在解决的研究问题及主要研究内容，最后详细介绍了论文的组织结构安排。

第二章：相关技术概述。该章详细描述了传统 SSD 的组织结构及其内部组件功能，进而对基于高密度闪存技术的混合 SSD 进行介绍，包括高密度闪存的定义、存储原理以及不同闪存介质之间性能差异的成因。以 SLC-TLC 混合技术为例，论述了混合 SSD 的基本架构，探讨了异构闪存在混合 SSD 中的混合方式及其优缺点，并对 SSD 读性能优化方法（包括数据预取、I/O 请求调度及缓存管理）进行了综合性介绍，讲述了它们的基本流程、面临的难点与挑战，以及现有的研究工作。

第三章：基于模式局部性引导的数据预取技术研究。本章首先对数据预取技术的问题域进行了深入剖析，包括技术流程和设计难点的讨论。然后，通过实验分析公开的工作负载块访问记录，研究读访问请求的时空局部性特征，并基于这些特征提出指导性设计原则。接着，基于这些分析结果，提出了一种新颖的基于模式局部性引导的数据预取技术设计方案，并详细说明了其内部两个组件的实施细节及组件间的数据交互机制。最终，在 SLC-TLC 混合 SSD 仿真平台上对该数据预取技术的性能进行了评估。

第四章：基于 RAM 和 SLC 的混合读缓存管理技术研究。该章介绍并分析了读缓存管理技术面临的问题和设计难点，进而讨论了混合 SSD 中读访问数据的

中国知网 https://www.cnki.net

位置分析结果及其对读缓存管理技术设计的启示。此后，介绍了一种基于 RAM 和 SLC 混合的读缓存管理技术的设计思路，包括对访问请求及页面属性的创新性修改，以及基于访问感知的 RAM 读缓存管理和基于访问过程的 SLC 读缓存管理策略。同样，该技术在 SLC-TLC 混合 SSD 仿真平台上经过性能验证。

第五章：总结与展望。本章对全文的主要工作进行总结，指出研究过程中的不足之处，并对未来可能的研究方向进行了展望。

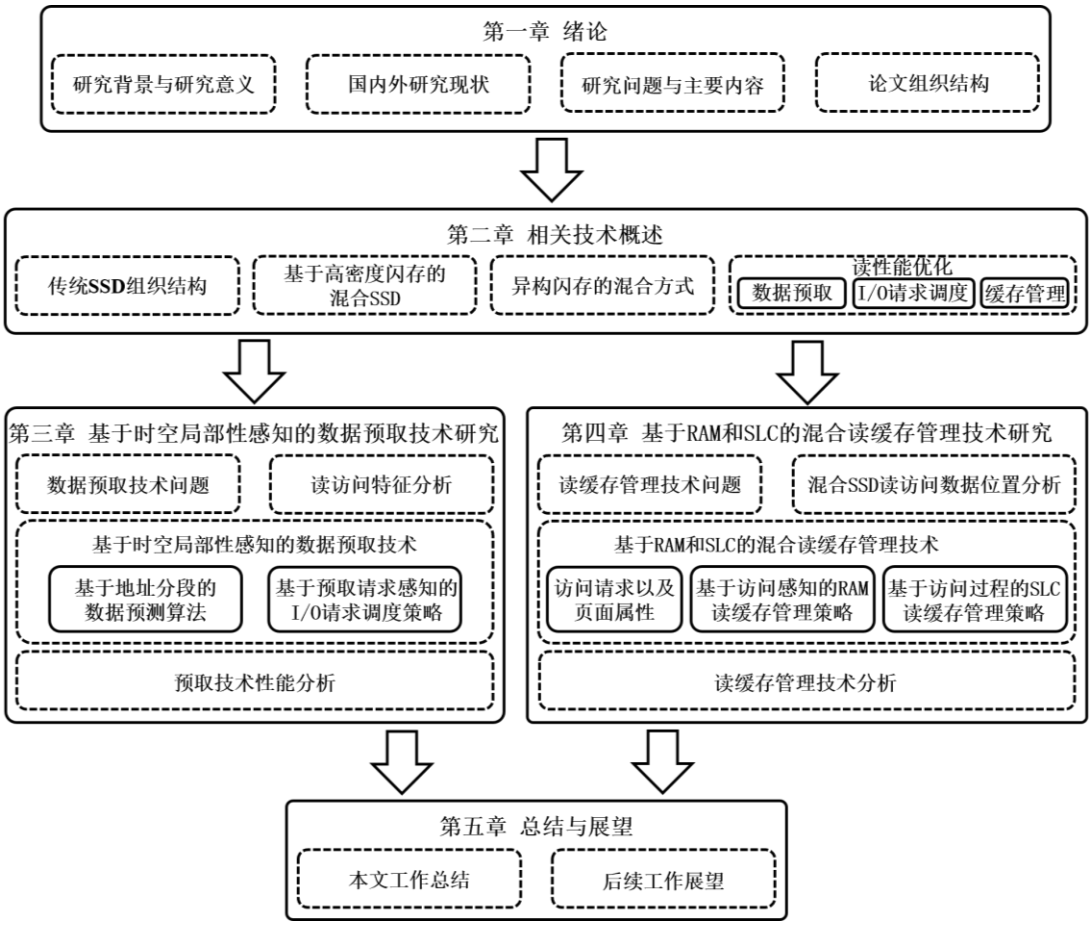


图 1.3 本文组织架构图

中国知网 https://www.cnki.net

第二章 相关技术概述

近年，闪存技术的迅猛发展，促进了基于闪存的存储设备的普及。这种高密度闪存技术既带来了存储领域的新机遇，也引入了诸多挑战。为了设计出基于这类闪存的高效存储系统，研究者必须深入了解其内部结构及使用特点。本章节首先回顾了传统 SSD 的组织结构，继而探讨了基于高密度闪存技术的混合 SSD 的基本结构与工作原理，并概述了现有研究中关于异构闪存的混合方式。最后，详细介绍了从数据预取、I/O 请求调度及缓存管理三个维度对 SSD 读性能优化的相关研究。

2.1 传统 SSD 组织结构

现代传统 SSD，广泛应用于过去乃至当下的存储领域，通常由基于 NAND 闪存技术的单一介质构成，特点包括高速的读写性能、低延迟、低能耗、良好的抗震抗振能力、紧凑的体积、无噪音运行及高可靠性等。

现有研究[31]指出传统 SSD 的结构主要由多个通道（Channel）组成，每个通道下连接有若干 NAND 闪存芯片（Chip）。每个闪存芯片内部分为多个晶片（Die），每个晶片又细分为多个平面（Plane），而每个平面包含多个块（Block）。在更细的层级上，每个块由多个页（Page）构成，页作为 SSD 的基本读写单位，其尺寸根据制造商的默认设置，通常为 4KB 或 8KB。如图 2.1 所示，展现了具有 4 个通道、每个通道 2 个闪存芯片、每个闪存芯片 2 个晶片、每个晶片 2 个平面的典型传统 SSD 的组织结构。这一层次结构（通道、芯片、晶片、平面）提供的并行性一直是以往高性能研究的重点[32][33]。为管理闪存并充分利用此并行性，SSD 控制器采用了闪存转化层（Flash Translation Layer，FTL），包含四个组件：地址映射（Address Mapping，AM）、数据分配（Data Allocation，DA）、磨损均衡（Wear-Leveling，WL）以及垃圾回收（Garbage Collection，GC）。同时，SSD 内部装有 RAM 缓存设备，以提高 I/O 性能。

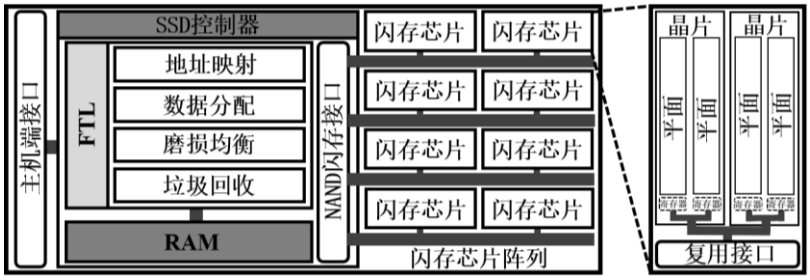


图 2.1 传统 SSD 组织结构

中国知网 https://www.cnki.net

针对以上各个模块组件，本段给出了如下详细的介绍：

主机端接口：主机端接口是指 SSD 与计算机主机或其他设备进行通信的接口，这个接口的作用是允许计算机主机与 SSD 之间传输数据、命令和控制信息。不同类型的 SSD 使用不同的主机端接口，常见的接口包括 SATA、PCIe、M.2 等。其中，基于 PCIe 的 NVMe（Non-Volatile Memory Express）协议是一种专为 SSD 设计的高性能、低延迟的接口协议，相较于其他传统接口，能够更好的发挥 SSD 的性能潜力。

SSD 控制器：SSD 控制器是固态硬盘内部的重要组件，负责管理控制 SSD 的各项内部操作以及与主机之间的数据交互，其内部包含四个组件。地址映射组件（Address Mapping，AM）负责管理逻辑地址和物理地址之间的映射。基于操作粒度，有三种类型的映射方案：页面映射[34]、块映射[35]和混合映射[36][37][38]。在本文中涉及的所有实验均基于一个假设：广泛采用页面映射技术，这是因为它往往能够提供更好的性能。数据分配组件（Data Allocation，DA）是为了确定写入操作的通道、闪存芯片、晶片和平面的分配[39][40]。磨损均衡组件（Wear-Leveling，WL）将写入的数据均匀的分布在闪存上，以延长 SSD 的使用寿命[41][42][43]。由于闪存页面在重新编程（写入）之前需要擦除操作，因此现代 SSD 采用异地更新方案进行数据更新。当可用页面数量降至预定义阈值之下时，垃圾回收组件（Garbage Collection，GC）将被激活以回收无效页面[44]。

RAM 缓存区：现代 SSD 通常在 SSD 控制器中包含一个内置的 RAM，称为 SSD 缓存区。该区域用于临时存储热读写缓存数据、元数据。由于与闪存相比，RAM 的访问延迟显著降低，配备缓存区的 SSD 可以为驻留在缓存区的数据提供更强的性能[45][46][47][48]。

NAND 闪存接口：此接口起到了连接 SSD 控制器与 NAND 闪存芯片阵列之间的桥梁作用。这个接口允许 SSD 控制器通过相应的协议与 NAND 闪存芯片进行通信，实现对数据的读取、写入以及管理操作。

2.2 基于高密度闪存的混合 SSD

现代 SSD 旨在提供更大的存储容量及更高效的性能。为达成此目标，SSD 采用内部混合架构，结合了高速 RAM（DRAM 或 SRAM）和 SLC 作为数据缓存区，同时利用 TLC/QLC 等高密度闪存单元来扩充存储容量。本研究将重点放在基于 SLC-TLC 闪存技术的混合 SSD 架构。据此，本节将详细阐述高密度闪存技术及 SLC-TLC 混合 SSD 的基本架构。

2.2.1 高密度闪存

NAND 闪存的基本存储单元是浮栅极或电荷捕获的晶体管，两个存储单元

中国知网 https://www.cnki.net

的浮栅极构成电容器。而其中的电荷数量变化，都会引起周围浮栅极电荷变化，彼此相互影响。在高密度闪存时代，随着 3D 堆叠技术的不断推进，闪存制程缩小，存储单元之间的相互干扰也越来越大。这造成了高密度闪存在读写操作延迟方面明显高于低密度闪存。3D 堆叠式 TLC 和 SLC 闪存块内部结构如图 2.2 所示。存储单元以行和列的矩阵排列，每一行中的存储单元连接到字线，每列中的单元耦合到位线。页面大小取决于存储器阵列中位线的数量，SLC 闪存块中的页面数等同于字线数，而 TLC 闪存块中的页面数为字线数的三倍，原因在于 TLC 技术中，三种不同的页面共享同一字线的存储单元。这三个页面分别成为最低有效位（Least Significant Bit，LSB）、互补有效位（Complementary Significant Bit，CSB）、最高有效位（Most Significant Bit，MSB）。

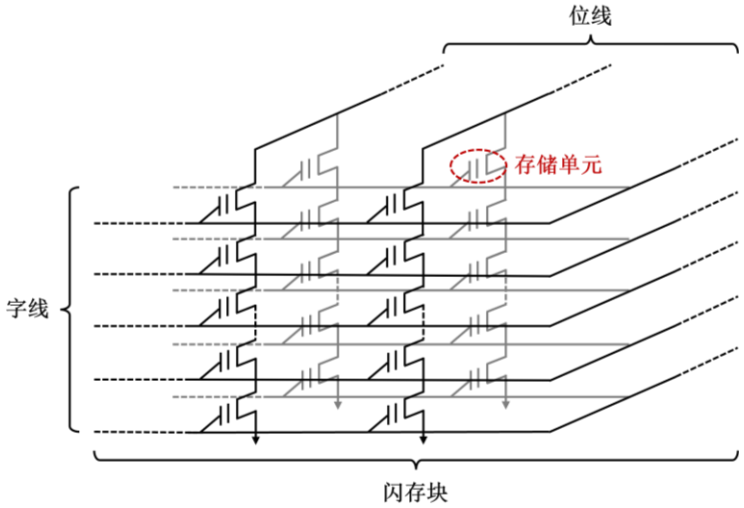
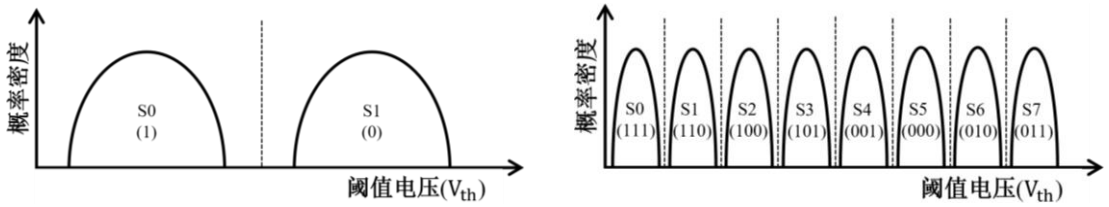


图 2.2 3D 堆叠式闪存块内部结构

如图 2.3 所示，揭示了 SLC 和 TLC 存储单元存储数据量不同的原因。NAND 闪存能够后通过引入不同数量的电子来在单个存储单元中存储 N 位数据，表示不同状态。该过程利用增量式脉冲编程（Incremental Step Pulse Program，ISPP），通过重复充电闪存存储单元，直到达到特定的目标电压状态[49]。对于 N 位数据存储，存在2^N个电压状态。SLC 闪存单元采用 2 种电压状态表示 1 位数据，而 TLC 闪存单元通过划分 8 种电压状态来存储 3 位数据。由于 TLC 在状态间的电压边际较窄，需要更多的 ISPP 周期以实现较小的电压增量，从而增加编程时间。读取数据时，不同类型的闪存单元采用感应电压来区分电压状态。根据定义的电压状态，使用感应电压的次数会有所不同。对于 SLC 闪存单元，只需要一个感应电压即可读出数据，读延迟较低。然而，TLC 闪存单元将 3 位数据（LSB、CSB 和 MSB）编码为 8 个电压状态，需使用多至二或三个感应电压以精确读取 CSB 或 MSB 值。因此，与 SLC 闪存单元相比，从 TLC 闪存单元读取数据所需要的时间更多。

中国知网 https://www.cnki.net



(a) SLC 电压分布　　(b) TLC 电压分布

图 2.3 SLC 和 TLC 电压分布

2.2.2 SLC-TLC 混合 SSD 基本架构

高密度闪存技术虽然降低了存储成本，但伴随而来的是性能指标的全面退化，包括读取、编程（写入）、擦除延迟以及耐久性和可靠性。在这些挑战中，写入性能和耐久性尤为关键，成为 SSD 制造商的焦点。为了弥补 TLC 写入性能的不足， 混合 SSD 架构利用 SLC 闪存介质作为 TLC 的写缓存区，提供稳定快速的写性能[50]。图 2.4 展示了 SLC-TLC 混合 SSD 典型架构，其中 RAM 区域位于 SLC 缓存区域之上，而 TLC 区域则位于下方。SLC 缓存区域作为 TLC 存储区域的缓存，显著提升了整体系统吞吐量。在处理读请求时，混合 SSD 从 SLC 或 TLC 中读取数据，并在传输至主机前暂存于 RAM 中。对于写请求，数据首先存入 RAM，随后根据 SLC 缓存区域的空间情况决定是否直接写入 SLC 缓存区域或是转入 TLC 区域。在数据更新过程中，混合 SSD 进行异地更新操作，新数据在被写入闪存之前，旧数据需要被置为无效。无效数据最终会在 GC 过程中被回收，从而触发擦除操作。此外，为维持 SLC 缓存区域提供持续的高写入性能，其有效数据需定期迁移至 TLC 区域，以释放出更多的 SLC 空闲空间。

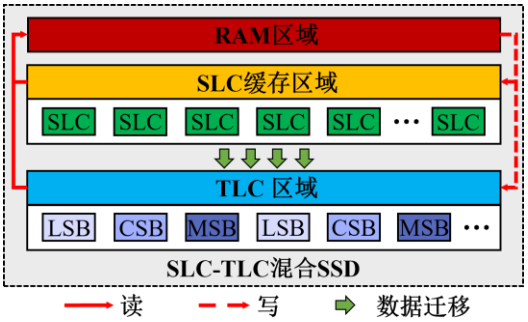


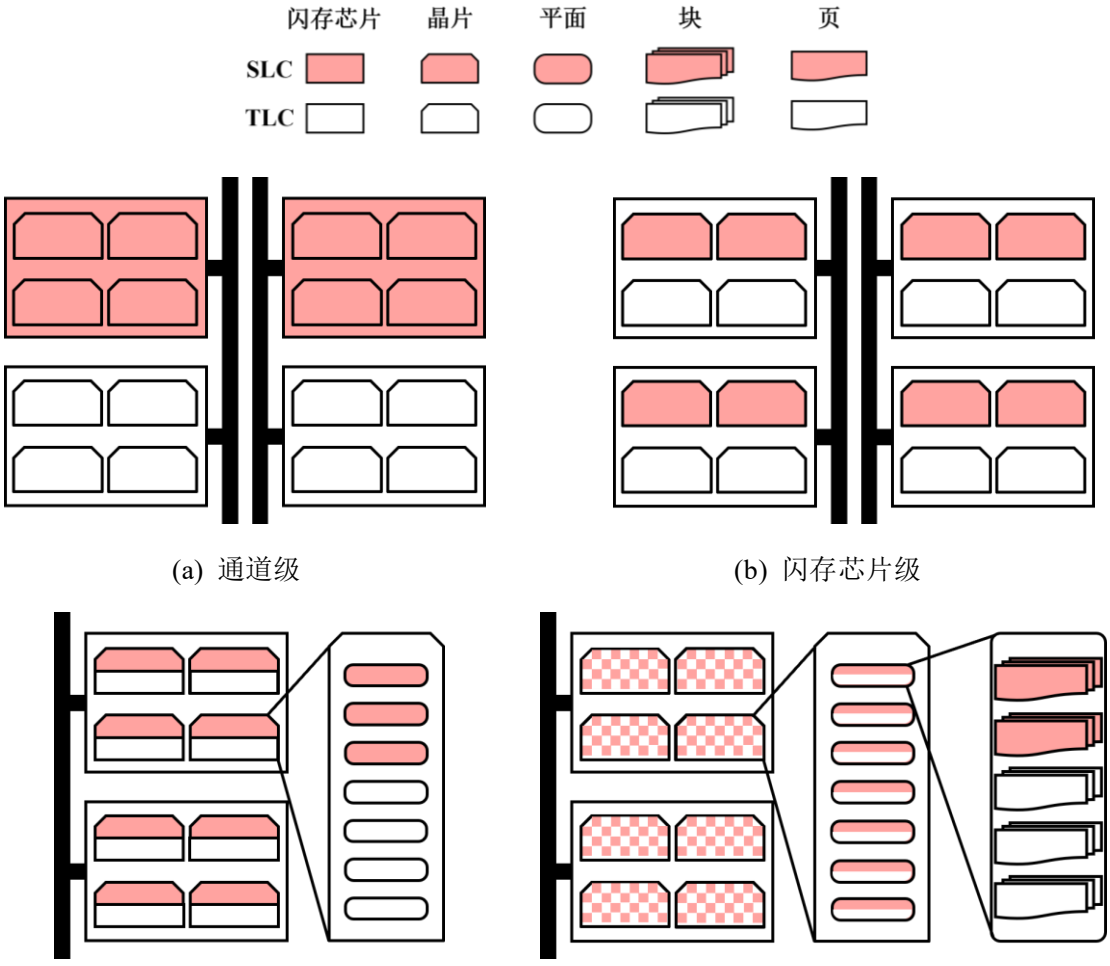
图 2.4 SLC-TLC 混合 SSD 架构

2.3 异构闪存的混合方式

在传统异构闪存混合方式中，高性能分区（即 SLC 缓存区域）通常构建于多个 SLC 芯片上，形成一个集中式缓存区域，而高密度分区（即 TLC 区域）由多个 TLC 芯片组成，充当主要的存储空间，即整个存储空间在物理和逻辑上被划分为两个大区域[51] 。虽然此种双区域设计简化了闪存资源管理，但未能充分利用闪存芯片阵列的内在并行性（通道/闪存芯片/晶片/平面），从而错失了优化

中国知网 https://www.cnki.net

混合 SSD 性能的机会。近期研究中，Wenhui Zhang 等人[32]针对不同级别的 SLC-TLC 混合进行了测试，并证实了采用平面级混合方式对于混合 SSD 性能具有较好的提升效果。本节将详细介绍这四种混合方式的闪存布局。



(a) 通道级　　(b) 闪存芯片级

(c) 晶片级　　(d) 平面级

图 2.5 SLC-TLC 混合 SSD 四种闪存混合布局形式

如图 2.5 所示，为 SLC-TLC 混合 SSD 四种闪存混合布局形式，分别为通道级、闪存芯片级、晶片级和平面级。具体而言，通道级混合方式将 SLC 闪存和 TLC 闪存分别布局在通道内单独的闪存芯片上，闪存芯片级混合方式将 SLC 闪存和 TLC 闪存分别布局在闪存芯片内单独的晶片上，晶片级混合方式将 SLC 闪存和 TLC 闪存分别布局在晶片内单独的平面上，平面级混合方式将 SLC 闪存和 TLC 闪存分别布局在平面内单独的块上。这些混合方式因其结构差异，在并行性和读写成本方面展现出不同特性。特别是平面级布局，由于每个平面都有 SLC 页，SLC 写入可以充分利用通道/芯片/芯片内/平面级的并行性，获得极高的读写性能。相比之下，通道级布局由于大部分芯片缺少 SLC 页，仅能利用通道级的并行性。此外，平面级布局中的数据迁移可通过平面内复制完成，仅需占用单一平面资源；而通道级布局的数据迁移则需跨芯片传输，占用两个芯片和一个通道

中国知网 https://www.cnki.net

资源，这可能对端到端的用户读写性能产生负面影响。

综合考量并行性、读写性能以及数据迁移成本，平面级混合布局显著优于其他方案，成为 SLC-TLC 混合 SSD 的首选混合布局。相对而言，采用 SLC 芯片（硬分区或软分区[51]）实现的通道级混合布局，在并行性和读写成本方面表现最差。据此，本研究的实验均基于平面级混合布局的 SLC-TLC 混合 SSD 进行。

2.4 读性能优化

在 SSD 控制器内部，存在一个高速 RAM 区域用于快速缓存写入请求，从而迅速完成写入操作而无需直接访问较慢的闪存介质。相比之下，读请求通常会遭遇更长的端到端延迟，因为需要从闪存介质中检索数据。特别是在混合 SSD 配置中，由于数据存储位置的变动，读性能对此更为敏感。随着存储系统领域向读密集型场景的转型，读性能优化成为研究的焦点。针对混合 SSD 的读性能优化，研究主要聚焦于三个方面：数据预取、I/O 请求调度及缓存管理。

2.4.1 数据预取

数据预取技术，作为提升 SSD 读性能的关键策略，利用 SSD 控制器中的 RAM 缓存基于历史访问信息和当前访问请求预测未来读取操作，生成预取指令并将其加入请求队列。目的在于提前缓存可能需求的数据于 RAM 中，以便利用 RAM 的高速访问特性加速读请求的处理，如图 2.6 所示概述了数据预取技术的基本工作流程。

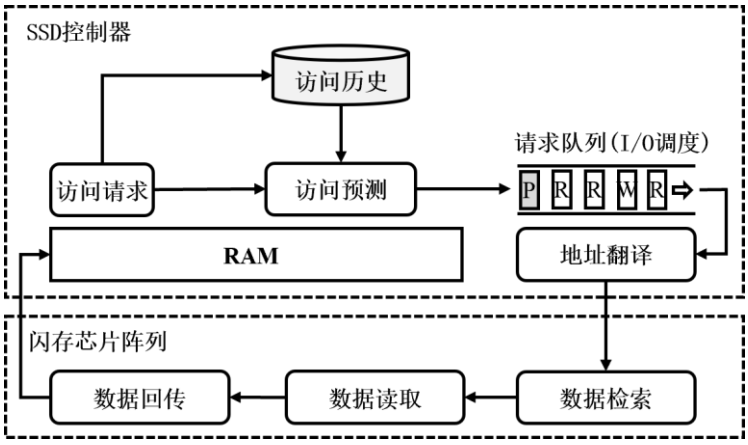


图 2.6 数据预取技术的基本流程

访问请求：系统端要访问数据时，会向 SSD 提交访问请求。其包含的信息如访问类型（读/写）、起始逻辑地址、访问数据大小以及时间戳等。

访问历史：预测算法依赖于经验数据进行预测分析。因此，SSD 控制器通常会在 RAM 中划分一小部分空间以记录历史访问信息。这些历史记录的长度通常由一个固定的超参数决定，但某些预测算法允许根据预测准确性动态调整历史记录的长度。

中国知网 https://www.cnki.net

访问预测：该过程是数据预取的核心，直接影响数据预取的准确性。它依据历史访问信息及当前访问请求，通过预测算法，确定未来可能被访问的数据逻辑地址。

请求队列：继访问预测后，预测模块将生成的预取请求加入 SSD 请求队列。此环节也是决定数据预取技术效果的关键环节，因为预读取请求的加入可能会阻塞原有用户 I/O 请求的处理，增加响应时间。因此，高效的 I/O 调度策略至关重要，以最小化预取请求对系统响应性的影响。后续小节将详细探讨具体的 I/O 调度技术。

地址翻译：根据访问请求的逻辑地址通过地址翻译过程，映射到 SSD 内部的物理地址。

数据检索：利用物理地址信息，操作指令通过通道、闪存芯片、晶片、平面逐级发送至指定的并行处理单元。

数据读取：执行相应操作，从指定闪存介质中提取数据。

数据回传：已读取数据的并行处理单元通过多级并行通道，逐级将数据传输至 SSD 控制器内的 RAM。

预测算法在数据预取过程中起决定性作用，通过分析历史访问信息中的模式并结合当前的访问请求进行预测。预测算法的精度直接决定了预取技术在提升整个存储系统性能方面的有效性。尽管数据预取为SSD读性能优化带来多项益处，但其实施过程中仍面临若干挑战与难题，主要包括：

（1）访问模式变化：预取算法通常基于历史数据访问模式进行预测，但如果应用程序的访问模式发生变化，或者数据的热点位置不断变化，那么预测的准确性就会降低。

（2）空间开销：在 RAM 中预取大量数据可能会占用大量空间，而 RAM 的空间是有限的。因此，需要在提高性能和合理使用 RAM 之间进行平衡。

（3）写入操作的影响：部分数据预取可能会导致额外的写入操作，尤其是在数据被更新或者替换时。这可能增加 SSD 的写入负载，影响性能。

（4）预测算法精度：预测算法的准确性对于数据预取至关重要。如果算法不能准确地预测未来的数据访问模式，那么预取的效果可能会减弱，甚至适得其反。

（5）实时性需求：对于某些应用，如实时数据分析或交互式应用，数据的实时性非常重要。在这种情况下，预取算法需要更加灵活和高效，以适应快速变化的数据访问模式。

（6）成本和复杂性：实现高效的数据预取算法和系统可能涉及复杂的设计和算法开发，这可能增加系统的开发和维护成本。

中国知网 https://www.cnki.net

（7）缺乏全局信息：预取算法通常只能使用已知的历史数据访问信息，而缺乏全局信息可能导致对数据访问行为的不完整理解。

（8）混合工作负载的挑战：在处理混合工作负载（包括读、写、随机、顺序访问等）时，预取算法需要更灵活和智能，以适应不同的访问模式。

面对访问预测的挑战，研究领域主要聚焦于传统算法与智能算法两大类。传统算法涵盖多种启发式策略，如地址关联度预测、访问模式预测、隐马尔可夫模型预测等，各具优势与局限，适用于特定的应用环境。智能算法，包括深度学习与强化学习等，虽需大量数据和计算资源进行训练与迭代，但能够实现更高的预测精度。

下面根据访问预测基于的技术展开介绍：

（1）基于地址关联度的访问预测

地址关联度描述了不同逻辑地址间的关联性或相关性。具体来说，若访问某一逻辑地址时，存在较高概率后续访问另一特定逻辑地址，则这两地址间展现出较高的关联度。此关联性指标可用于预测未来访问模式，进而优化存储系统性能。

Li 等人[52]针对于 SSD 的 FTL 提出了一种支持自适应缓存管理的基于模式的预取方案。方案首先挖掘历史 I/O 请求之间关联的频繁访问模式。然后，将当前时间窗口内的请求与识别出的模式进行比较，以将数据预取到 SSD 的 RAM 缓存中。此方案的设计核心是将逻辑顺序上前后相关性较强的 I/O 请求涉及的数据归类到一起，实验表明其预取方案可降低平均读取延迟。万方祺等人[53]针对模拟仿真工作负载 I/O 访问序列中地址空间分布仿真部分提出了基于地址相关度的地址访问模拟。其首先计算了各个工作负载的 I/O 地址访问空间自相关度。然后，将相关度相似的地址空间归类为一个地址空间类。在后续 I/O 访问中，如果出现地址空间类的一个地址，那么地址空间类中其他的地址大概率也会出现。实验结果表明，各个地址类中地址出现次数相似且与真实负载接近。

（2）基于访问模式的访问预测

访问模式反映了请求序列中的前后地址偏移规律性和访问大小的规则。例如，顺序访问模式表现为连续请求的地址偏移量固定，访问大小一致；固定地址访问模式则表现为对特定几个地址的反复访问，其中访问顺序和大小保持不变。识别并总结这些模式，可使得一旦连续请求匹配某一模式特征，便可据此进行访问预测，并指导数据预取，从而优化存储系统性能。

Xu 等人[54]提出了一种基于热访问模式的 SSD 内部的数据预取方案。他们首先统计访问历史中频繁出现的访问模式，并将其作为访问模式集保存起来。在后续的访问请求中，当命中某个访问模式一定数量后，将访问模式集中其余地址空间数据预取到 SSD 的 RAM 中。实验表明，数据预取方案可以提升 SSD 部分读

中国知网 https://www.cnki.net

性能。Wang 等人[55]针对于基于闪存的存储系统提出了智能文件感知预取方案。该方案通过文件感知方法，挖掘文件访问模式，其采用的是马尔可夫状态转移矩阵对其访问模式（跳转关系）进行归类总结以预测未来的访问请求。实验结果表明，通过感知文件信息，再进行访问模式的总结具有更加有效的相关性捕获效果。Laga 等人[56]针对 Linux 预读预取系统，提出了基于马尔科夫链的文件预取方案。该方案在学习阶段检测 I/O 工作负载模式并计算文件页面之间的转换概率。预测阶段使用生成的马尔可夫状态机预取预测的文件页面。实验表明，相较于 Linux 传统文件预取方案，减少了将近一半的访问查询时间。

（3）基于隐马尔可夫的访问预测

隐马尔可夫模型是对马尔科夫模型的改进版本，传统的马尔可夫模型是一个纯粹的概率模型，其中每个状态的转移仅依赖于其前一状态，呈现无记忆性。这限制了马尔可夫模型捕捉观测序列中潜在结构与复杂动态模式的能力。隐马尔可夫模型通过引入隐藏状态和观测值的概念，改善了这一局限，能够更精确地描述序列数据中的隐含结构和模式。其对序列数据处理、不完全观测数据分析和复杂系统建模的优越性，使其在众多应用场景中成为优选模型。

Jung 等人[57]提出了基于隐马尔可夫模型的内存访问预测方案。该方案通过将内存地址空间划分为多个地址段以减少空间开销，并根据历史访问信息构建隐马尔可夫状态转移矩阵，最后通过当前访问的地址空间段归属以及跳转关系概率矩阵来进行预测。实验结果证明，隐马尔可夫模型具有相较于马尔可夫模型更加优异的预测准确性。

（4）基于深度学习和强化学习的访问预测

深度学习[58]和强化学习[59]是现代机器学习领域中的两个重要分支，它们在访问预测中有着广泛的应用。深度学习技术能够预测访问请求的到达时间、逻辑地址及其大小，并据此指导数据预取至 SSD 中的 RAM，进而减少访问延迟。另一方面，强化学习通过学习 SSD 内部资源状况，支持预取数据操作和资源分配，以优化操作决策。

Ganfure 等人[60]提出了一种受深度神经网络启发的新型上下文感知的内存预测方案。该方案采用长短期记忆网络（Long Short-Term Memory，LSTM）来进行上下文感知数据预取。实验结果表明，基于长短期记忆网络可以感知到长期和短期局部性信息，对于预取精度具有较好提升效果。Akgun 等人[61]针对于操作系统内存组件提出了一种基于深度学习框架的预读取参数调控方案。该方案通过利用神经网络动态调整操作系统内部访问预测长度参数，以达到最有优访问预测的效果。Li 等人[62]针对于多流 SSD 提出了基于强化学习辅助的资源分配方案。该方案通过强化学习框架可以更准确地预测 I/O 流的需求以及访问信息，以实现更精

中国知网 https://www.cnki.net

准的数据预测。Yang 等人[63]提出了一个基于注意力机制的模型 G&L，旨在改善固态硬盘（SSD）中的预取性能。G&L 模型包括生成式预训练（GPT）预取器和逻辑块地址（LBA）+I/O 大小表两个部分。GPT 预取器使用解码器+唯一框架和自注意机制预测下一个 LBA，而 LBA+I/O 大小表则映射 LBAs 到它们最常出现的 I/O 大小，以预测 I/O 大小。实验结果表明，G&L 模型在大多数情况下的准确率、覆盖率和 F1 分数均超过了基线模型，展示了其在提升 SSD 预取性能方面的有效性。

2.4.2 I/O 请求调度

I/O 请求调度是一种常用的读写性能优化的技术，通过在 SSD 中构建多个请求队列，用于管理和处理 I/O 请求，旨在避免因访问相同硬件而引发的读/读冲突（Read/Read Conflict，RRC）和读/写冲突（Read/Write Conflict，RWC），从而提升存储系统的性能和效率。I/O 请求调度流程通常涵盖六个主要步骤，包括请求接收、请求队列、调度算法、调度执行、数据传输以及更新状态，如图 2.7 所示。

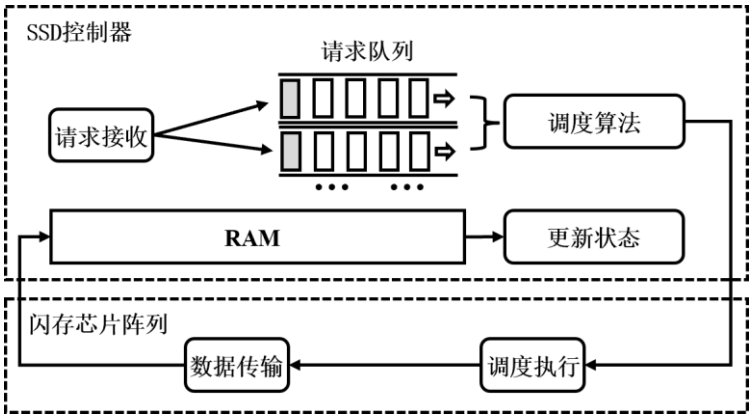


图 2.7 I/O 请求调度流程

请求接收：存储控制器接收来自主机的 I/O 请求，包括读取和写入操作，以及相应的逻辑块地址等信息。

请求队列：接收到的请求被放置在请求队列中。请求队列可以分为读取队列和写入队列，也可以按照特定算法及优先级策略进行细致管理。

调度算法：存储控制器使用调度算法来决定请求处理的优先顺序。主流调度算法包括先来先服务（FCFS）、最短寻道时间优先（SSTF）、最高优先级优先（HPF）、电梯算法（SCAN 或 LOOK）等，选取最适合当前性能指标和应用场景的算法以优化请求处理。

调度执行：依据所选调度算法的指导，存储控制器执行 I/O 请求操作。在 SSD 中，执行过程可能包括块擦除、数据写入、数据读取等具体操作。

数据传输：存储控制器将数据从存储介质传输到主机或者从主机写入到存储介质，完成 I/O 操作。

中国知网 https://www.cnki.net

更新状态：存储控制器更新相关的状态信息，包括已经完成的请求、正在处理的请求等。

调度算法在 I/O 请求调度中扮演至关重要的角色，主要负责规划 I/O 请求的执行顺序，以减少请求间资源竞争。设计及优化调度算法时需评估多个因素，确保算法能够适应不同应用场景，优化系统性能。以下是设计 I/O 调度算法需面临的关键难题与挑战：

（1）延迟和响应时间：考虑降低 I/O 请求的响应时间，特别是对于需要快速响应的应用场景，如实时系统或交互性应用。

（2）吞吐量：提高整体系统的吞吐量是一个重要目标，特别是在大规模数据传输的情况下，如批处理作业。

（3）公平性：保证所有 I/O 请求都能够得到公平的服务，防止某些请求被长时间地阻塞，从而避免饥饿问题。

（4）优先级：考虑 I/O 请求的优先级，确保对于某些重要任务或紧急任务，系统能够及时响应。

（5）均衡性：尽量实现负载均衡，避免存储设备或通道的过度利用，以提高系统整体性能。

（6）请求合并：考虑将相邻的读取或写入请求合并成更大的块，以减少寻址和传输的开销，提高效率。

（7）数据局部性：利用数据局部性原理，通过选择合适的调度策略，以提高命中率，减少访问延迟。

（8）适应性：考虑算法的自适应性，使其能够根据系统的运行状况和负载情况调整调度策略，以适应动态变化的环境。

（9）预测性：对于一些调度算法，尤其是在 SSD 等存储介质中，考虑使用一些预测技术，提前预测出哪些请求可能会被未来访问，以加速数据的获取。

（10）可扩展性：确保算法在不同规模和负载下都能够良好地扩展，以适应系统的变化。

针对 I/O 请求调度的难点和挑战，尤其在提升读性能的背景下，现有研究主要聚焦于缓解读读请求干扰和读写请求干扰。对于前者，研究采用基于吞吐量最大化和响应时间最小化的贪心调度策略，考虑各读请求的执行时间及成本；对于后者，通过读写数据分区或引入多 I/O 请求队列等方法来降低请求间的冲突率。

下面根据 I/O 请求调度采用的技术展开介绍：

（1）基于贪心思想的 I/O 请求调度

贪心算法是一种求解最优问题的算法，其核心在于求解问题的过程中，每步选择当前条件下的最优解，期望通过一系列局部最优决策达成全局最优。

中国知网 https://www.cnki.net

Cui 等人[64]提出了在 I/O 调度中利用延迟变化来实现访问特性引导的读写延迟最小化的方案。作者首先，通过了解读取延迟、写入延迟和原始误码率（RBER）之间的关系，讨论利用这种关系来减少读取和写入延迟的不同方法。然后，提出了一种基于贪心思想的 I/O 调度方案，利用访问数据的热度和保留期限来确定写入或读取的速度，优先考虑快速写入和快速读取以减少冲突。实验结果表明，采用贪心策略，追求极致的快速响应，可以大大提高存储系统整体的读性能。Li 等人[65]提出了关于写入数据和读刷新操作的调度方案，以减轻读扰动引起的负面影响。方案首先建立了一个模型，根据块的 P/E（编程/擦除）周期和累积读取次数的因素，将 SSD 块分为两个类别，根据估计的读错误率进行分类。然后，基于贪心的思想，将被热读取的数据重定向到具有较小读错误率的块，因为即使数据将被频繁请求，它对读扰动也不敏感。实验表明，贪心的将频繁读取的数据尽可能地存储在低错误率的 SSD 块中可以优先减少重复读取次数，提升读取吞吐。

（2）基于读写分区的 I/O 请求调度

鉴于写请求执行时间往往是读请求执行时间的几倍，读写冲突下的读请求往往受阻塞更加明显。读写分区作为解决此类冲突极为有效的策略之一，其核心理念在于将读写操作分配至不同的物理区域，促使读写活动在独立的并行单元中进行，从而降低冲突发生的概率，据此提高读性能。

Lv 等人[33]提出了基于读写分区调度的 I/O 干扰减少方案。该方案的基本思想是将 SSD 的并行单元划分为不同区域，并根据读写访问冲突特征将数据放置在相应的隔离区域，以避免冲突。实验表明，通过将相互干扰的读写数据调度到不同分区后，可以大大较少读写冲突事件的发生，提高了 SSD 整体的读性能。Deng 等人[66]提出了基于读写设备分区调度的 I/O 干扰减少方案。该方案在固态硬盘（Solid State Drive，SSD）和机械硬盘（Hard Disk Drive，HDD）混合环境中，通过在运行时自适应地将一些 SSD 写请求定向调度到空闲的 HDD，以缓解在 SSD 中因写操作导致的读请求阻塞现象，提升 SSD 读操作吞吐。实验表明，通过 I/O 设备分区调度既提升了系统的整体读性能，同时又延长了 SSD 的使用寿命。

（3）基于多 I/O 请求队列的请求调度

针对于 I/O 冲突问题，采用多 I/O 请求队列的请求调度策略能有效缓解冲突影响。每个 I/O 设备（例如固态硬盘、磁盘、网络接口）配备独立的请求队列，这些队列可以是独立的硬件队列，亦可基于软件的逻辑队列形式存在。各队列存储待执行的输入/输出请求，通过引入多 I/O 请求队列，实现 I/O 请求在多个队列间的分配及队内请求的执行调度，从而有效降低 I/O 冲突。

Zhu 等人[67]针对 NVMe SSD 的 I/O 冲突问题，提出了低开销的冲突感知 I/O

中国知网 https://www.cnki.net

(CFIO)机制，将 I/O 请求分组为无冲突批次，并分批分派请求，以利用 NVMe SSD 的内部并行性。实验表明，多队列保存无冲突批次 I/O 请求进行分批执行可以有效减少 I/O 请求冲突，提高存储系统整体的吞吐量。

2.4.3 缓存管理

混合 SSD 的缓存管理指的是在混合 SSD 中将部分数据暂时存储或缓存备份在高性能分区或 RAM 中并加以管理，以快速应对未来的访问请求[68]。混合 SSD 结合了高密度分区（TLC/QLC）的大容量和相对较低成本与高性能分区（SLC）的高速读写性能。缓存管理通常是为了充分利用 SSD 的性能优势，提升系统的相应速度和整体性能。混合 SSD 缓存管理流程可分为三个步骤，包括记录访问历史、数据特征分析、缓存管理策略，如图 2.8 所示。

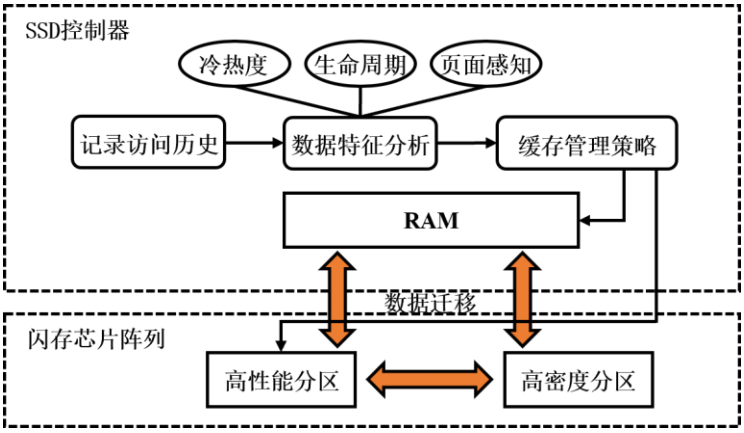


图 2.8 混合 SSD 缓存管理流程

记录访问历史：在缓存管理之前，一项重要的流程就是记录 SSD 控制器所接收并执行的 I/O 请求，通常会记录请求的访问时间戳、起始逻辑地址、访问大小、访问类型等基本信息。此外，访问历史记录也不会是无限大的，一般 SSD 控制器会设置相应的阈值，当记录长度达到阈值则会进行相应的记录清理，并激发数据特征分析流程。

数据特征分析：在数据特征分析阶段，历史访问记录会被深入研究和分析，以了解数据的访问特性、访问模式和重要性。数据特征分析一般包括冷热度分析、生命周期分析、页面感知等。这项分析的目的是为了更好地指导制定缓存管理策略，从而提高系统的整体性能。

缓存管理策略：在混合 SSD 缓存管理流程中，缓存管理策略是指根据系统中数据的特性和需求，制定合理的计划和方法来管理高性能介质中的数据，包括 RAM 区域和高性能分区。策略的目的是最大化 SSD 的性能优势，通过控制各个介质间数据迁移以及介质内的数据组织管理，将未来被访问的数据暂存在高性能介质中，提高系统的响应速度，并在资源有限的情况下有效地利用混合存储结构。需要注意的是，现有缓存管理方式包括读缓存管理、写缓存管理以及统一管理。

中国知网 https://www.cnki.net

数据特征分析及缓存管理策略构成缓存管理流程的核心，它们相辅相成。即依据一种或多种数据特征制订缓存管理策略，以指导数据在存储介质内与存储介质间的有效管理。因此，不同的特征视角可能促成多样的缓存策略。设计过程中，需关注以下难题与挑战：

（1）访问频率和热度：需要重点关注哪些数据被频繁访问，以确定热数据。与此同时，要具体到数据的热度，即数据的访问频率相对于其他数据的高低。

（2）数据大小和分布：掌握数据的大小和分布可以有利于了解系统中各种大小的数据量。数据大小对存储介质是存在一定影响的，尤其是在闪存介质中。

（3）访问模式：一般来说，数据的访问模式大致分为随机访问和顺序访问。需要结合闪存介质在处理各种访问模式的性能表现。

（4）关联关系：数据与数据之间在访问中具有相关性，识别数据之间的关联关系，有助于确保相关的数据集一并迁移，避免在不同闪存介质之间拆分具有关联性的数据。

（5）历史访问趋势：分析数据的历史访问趋势，了解是否存在季节性或周期性的模式。考虑历史趋势对未来工作负载的影响，制定动态的迁移策略。

（6）重要性和业务需求：评估数据的重要性，确定哪些数据对系统性能和业务需求最为关键。将对整体性能有更大影响的重要数据纳入优先考虑。

针对于缓存管理的难点和挑战，现有研究聚焦于数据的冷热程度、生命周期和页面感知等特征指标，基于这些指标制定缓存管理策略，旨在优化混合 SSD 的 I/O 性能。

下面根据缓存管理采用的技术展开介绍：

（1）基于冷热程度的缓存管理

基于冷热程度的缓存管理，通过区分数据的访问频率或热度（热数据指频繁访问的数据，冷数据指很少被访问的数据），将数据分配至不同存储介质，旨在优化系统性能。核心理念是将热数据置于速度更快的存储介质中，提升访问效率，而冷数据则存于成本较低但速度较慢的介质，实现成本优化。

Jun 等人[69]针对于 SSD 中模式化读取冲突以及意外读取冲突问题，提出了分离存储热读取冲突数据以缓解 SSD 内读取冲突，提升 SSD 读取性能。其方法通过识别读取访问中经常发生读冲突的数据，并将这些数据分别存储到不同的并行单元中，以利用 SSD 的并行度避免读冲突。实验表明，此方法可以有效降低 SSD 读操作的平均延迟和尾部延迟。Yoo 等人[70]针对基于冷热数据迁移技术中冷热阈值难确定的挑战，提出了基于强化学习的动态冷热阈值确定技术。该技术通过监视 I/O 工作负载的变化来动态调整参数，以达到最优 SLC 缓存参数的目的。Choe 等人[71]针对于闪存友好文件系统（Flash-Friendly File System，F2FS）

中国知网 https://www.cnki.net

通过将数据分为热/温/冷三个等级来进行管理，以最大限度地发挥闪存的优势和高效的数据管理。Li 等人[72]通过利用冷热数据划分，将热数据存储在 SSD 中高可靠的闪存块中，提升了 SSD 的读性能。Lin 等人[73]针对于传统 SSD，通过划分冷热数据，并指导 SSD 优先缓存热数据来提高 SSD 的 DRAM 命中率。Long 等人[74]针对于 3D TLC SSD 通过划分冷热数据，将热数据迁移或存储到 LSB 和 CSB 低访问延迟页面上，以提升读写速度。Zhu 等人[75]针对 ZNS 固态硬盘中读优化问题，利用将热数据备份缓存在未满的 ZNS 区域中，以利用并行性快速响应读访问请求，达到提高 ZNS 固态硬盘的读性能的目的。

（2）基于生命周期的缓存管理

数据生命周期定义为两次连续更新请求之间的请求数量或时间跨度，反映数据在存储介质中的静态驻留期。在混合 SSD 数据管理中，利用生命周期指导数据迁移主要聚焦于写入性能的优化。按生命周期分类，将生命周期相似的数据存储于同一块中。考虑到写更新前需执行读取操作，通过批量读取同块内数据至内存，可减少后续读取操作需求，从而提升系统的读取效率。

Chen 等人[76]针对现代程序中大量短期数据占用 SSD 中用于保证期限内可靠存储数据强大的数据保护问题，作者通过分析识别数据的生命周期，并只将长生命周期的数据采用高强度的纠错码来保存在高可靠的 SSD 闪存中，而针对于短期数据则采用低强度纠错码以减少写入延迟，并迁移到低可靠性闪存中。实验表明，该方法可以有效减少写入延迟，并在一定程度减少由于写操作导致的读请求阻塞延迟，提高整体读性能。

（3）基于页面感知的缓存管理

页面感知，即对物理页面特性的认识，对于指导含有多种页面类型的混合 SSD 中的数据迁移至关重要。例如，在 SLC-TLC 混合 SSD 中，根据数据所在物理页面类型（SLC/LSB/CSB/MSB）或页面访问延迟，进行数据迁移，能显著提升存储系统的读性能。

Li 等人[77]提出了针对 SLC-QLC SSD 的一种页面延迟感知的数据迁移策略。该策略通过识别出读取延迟较高的频繁读取页面，并将其迁移到 SLC 区域，以提高整体读取性能。实验表明，该策略可以有效提高存储系统的整体读取性能。Chen 等人[78]针对于 QLC SSD 的读性能优化，提出了一种新颖的闪存页面类型感知读取缓存方案。该方案将 QLC 逻辑上或分为四个区域，并根据页面的区域归属，优先缓存高延迟区域的页面。该方案极大的优化了 QLC SSD 的平均读取延迟。

2.5 本章小结

本章深入探讨了与本研究紧密相关的技术领域，涵盖了传统 SSD 的组织结

中国知网 https://www.cnki.net

构、基于高密度闪存技术的混合 SSD、异构闪存的混合方式，以及 SSD 读性能优化四个部分。首先，本文对传统 SSD 的内部物理构造及其模块组件进行了阐释；随后，以 SLC-TLC SSD 为例，详细介绍了高密度闪存混合 SSD 的介质特性及其架构；其次，讨论了四种不同层级的异构闪存介质混合方式；最后，分析了 SSD 领域内读性能优化的理论基础、面临的研究难题与挑战，并概述了包括数据预取、I/O 请求调度和缓存管理在内的现有研究进展。

中国知网 https://www.cnki.net

第三章 基于模式局部性引导的数据预取技术研究

本章的主要研究内容是设计一种高效的数据预取技术。该技术包含两部分：预测算法、I/O 请求调度策略。针对不同工作负载的访问特性，本研究引入一种基于地址分段的预测算法来预测未来访问数据，并生成预取请求发送至请求队列中，旨在提高 SSD 控制器中 RAM 空间的利用率，提升混合 SSD 读性能。同时，为减轻由预取请求引入的主机 I/O 请求阻塞问题，本研究提出一种基于预取请求感知的 I/O 请求调度策略，以最小化预取过程对主机请求的负面影响。

3.1 数据预取技术问题

本节讨论针对于不同工作负载的数据预取问题，如图 3.1 所示：

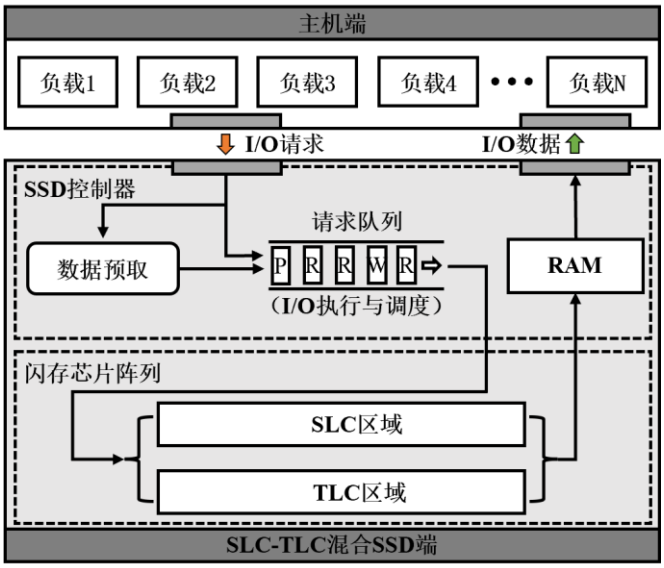


图 3.1 混合 SSD 数据预取技术概览

一个数据预取技术通常实现在 SSD 控制器内部，主机端的各个工作负载在运行过程中会不断发送 I/O 请求（图中的 W 表示写请求，R 表示读请求）到 SSD 控制器中的请求队列，以等待读写请求的执行。数据预取模块会实时获取到达的 I/O 请求，并通过预测算法来预测未来可能会读取的数据，并生成预取请求（图中的 P 表示预取请求）发送至请求队列中。等到预取请求被调度执行时，会将预测的数据提前读取到 SSD 控制器的 RAM 中，以应对未来的读请求。通常来说，在现有服务器中，不同工作负载的访问特性差异性较大，且考虑到 SSD 控制器芯片的计算性能有限，因此如何设计一个高精度且低开销的预测算法是需要解决的核心问题。与此同时，由于请求队列中预取请求的添加，原有的主机端请求会

中国知网 https://www.cnki.net

受到一定的阻塞，因此用于最小化预取过程对主机请求影响的 I/O 请求执行与调度策略也是重中之重。

数据预取技术问题的难点在于：**1）一个 SSD 可能服务于主机端多个工作负载，这些负载的访问特性是存在差异的。**这意味着每种应用程序产生的数据访问模式（如读写比例、访问频率、数据局部性等）可能都不相同。数据预取需要在混杂的 I/O 请求流中更加智能地识别和适应不同的数据访问模式。表 3.1 展示了本文统计有关剑桥微软研究院公开的 15 条服务器 I/O 跟踪记录[79]的访问模式信息。这 15 条工作负载记录中，每条都包含了 100 万 I/O 访问请求信息，包括其请求时间戳、访问起始逻辑地址、访问大小以及操作类型等相关信息。在表 3.1 中，本文统计了各个工作负载的 8 项特征指标，包括读请求比例（读请求数量占总请求数量的百分比）、邻近读比例（与上次读请求数据地址空间相交的读请求数量占读请求数量的百分比）、邻近写比例（与上次写入数据地址空间相交的写请求数量占写请求数量的百分比）、访问空间（I/O 请求作用的地址空间大小）、读数据量（因读请求导致从 RAM 传输到主机端的数据量）、写数据量（因写请求导致从主机端传输到 RAM 的数据量）、读页面数（读请求所涉及的逻辑页面量）、写页面数（写请求所涉及的逻辑页面量）。

表 3.1 剑桥微软研究院公开 I/O 跟踪记录

工作负载	读请求比例	邻近读比例	邻近写比例	访问空间(GB)	读数据量(GB)	写数据量(GB)	读页面数(GB)	写页面数(GB)
HM_0	24.9%	31.1%	20.3%	1.10	2.76	8.03	0.65	0.76
HM_1	96.9%	2.9%	3.0%	0.19	8.54	0.35	0.14	0.10
PRN_0	10.6%	20.9%	36.2	3.80	2.69	11.88	1.57	2.58
PRN_1	69.2%	17.8%	3.8%	7.86	12.08	4.05	7.22	1.66
PROJ_1	90.7%	28.2%	12.3%	35.56	37.55	1.98	34.41	1.55
PROJ_3	90.3%	12.5%	18.0%	4.06	12.95	2.78	3.05	1.86
PROJ_4	96.4%	9.1%	5.4%	3.74	9.54	0.51	3.61	0.21
PRXY_0	2.9%	16.5%	75.9%	0.24	0.27	5.81	0.08	0.17
RSRCH	9.0%	11.9%	13.5%	0.33	1.35	11.02	0.07	0.27
SRC1_2	15.4%	11.3%	19.6%	1.62	2.48	35.99	1.11	0.64
SRC2_0	13.5%	12.7%	14.4%	0.71	1.63	9.09	0.40	0.48
SRC2_2	28.0%	18.2%	32.2%	20.27	23.57	39.68	20.22	20.06
STG_0	23.1%	16.7%	11.9%	6.30	7.39	9.31	6.16	0.28
USR_0	37.1%	13.3%	14.1%	2.27	16.8	8.12	2.06	0.44
WDEV_0	20.1%	12.2%	8.9%	0.52	3.18	9.23	0.20	0.34

表 3.1 揭示了一个关键的观察结果：不同的工作负载具有独特且多样化的特征数据。这种特征数据的差异性给设计通用的预取算法带来了显著的挑战。每个工作负载，如数据库查询、文件系统操作或网络数据传输，都有其特定的数据访

中国知网 https://www.cnki.net

问模式，这些模式决定了数据的存取方式。因此，一个有效的预取算法需要能够适应这些多样化的访问模式，以优化数据读取性能和降低延迟。

另一个增加设计难度的因素：**2）同一工作负载的读访问模式的动态性。**在不同的时间段，即使是相同的工作负载也可能表现出不同的数据读访问模式。例如，图 3.2 所展示的 PRN_1 工作负载的读访问分布图清楚地描绘了这种动态变化。在不同的时间段，可以观察到顺序访问、乱序访问、重复访问等多种模式，这些模式不仅变化无常，而且彼此之间还可能交替出现。这种模式的多变性使得预取算法难以持续有效地识别并优化当前的访问模式。

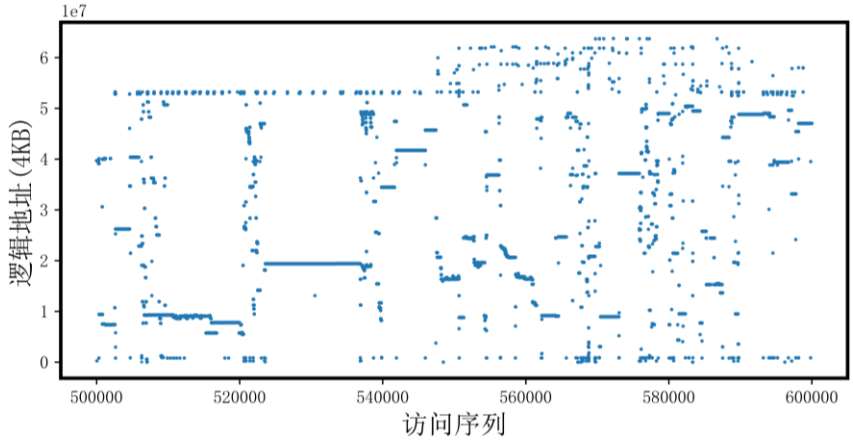


图 3.2 PRN_1 读访问分布截取图

此外，**3）每种访问模式的持续时间通常很短。**这意味着为了保持预取算法的效果，需要频繁地计算和更新算法的状态。这种频繁的计算更新不仅对计算资源的需求量很大，而且也增加了算法的复杂性。设计这样的算法需要考虑到如何在保证预取效果的同时，尽可能减少对计算资源的消耗，以及如何快速准确地适应访问模式的变化。

总之，SSD 数据预取技术的设计面临着来自工作负载特性多样性、动态变化以及短暂性的三重挑战。为了有效地应对这些挑战，预取算法需要能感知到相应的工作负载并能够灵活地适应不同工作负载的特定需求，不仅如此，还需要能够快速响应访问模式的变化，同时尽量减少对计算资源的需求。

3.2 读访问特征分析

读访问特征分析可以提供关键的信息和见解，对于预测算法的设计、实现与优化具有指导意义。针对多工作负载的数据访问请求流，需要以工作负载为粒度进行独立的读访问模式分析。与此同时，针对动态变化且短暂的访问模式，预取技术需要周期性的识别其访问模式并更新模式库。读访问特征分析主要可以分为两大类，即时间局部性分析、空间局部性分析。时间局部性可以指出数据的周期性访问行为并可以确定预取技术的周期长度。空间局部性则可以让预取技术感知

中国知网 https://www.cnki.net

哪些数据的关联性强，并作为预测的依据。本节针对这两类分析工作进行相应介绍，并提出工作负载在这两类分析结果中存在的共性特征，以指导后续的预测算法设计。

3.2.1 时间局部性分析

时间局部性（Temporal Locality）指的是如果一个数据块或页面在当前时间窗口被访问过，那么它很可能在后续的时间窗口中再次被访问。这种特性对于设计高效的数据预取策略至关重要，因为它使得预取算法能够基于最近的访问历史来预测并加载那些可能很快再次需要的数据。

首先，本节探究了各个工作负载的重复访问数据占总数据的比例P，如式（3.1）列出了重复访问数据占总数据的比例P的数学定义：

$$P = \frac{count_{repeat}}{count_{all}} \tag{3.1}$$

其中，$count_{repeat}$表示各个工作负载中重复访问的数据总量；$count_{all}$表示各个工作负载中访问的数据总量。实验结果如图 3.3 所示。从图中可以看出，大部分的工作负载的数据存在重复访问现象，这也证明了工作负载是存在时间局部性的，但是局部性的强弱还是需要其他指标来进行评价。

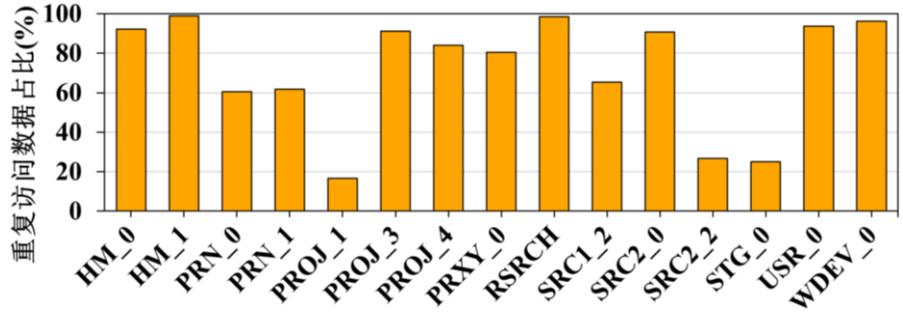


图 3.3 各个工作负载重复访问数据占比

时间局部性的评价指标包含窗口内重复访问比率和窗口间重复访问比率。窗口内重复访问比率表示在某个时间窗口内重复访问的数据项比例，通常用来表示重复性高的数据项在一个时间段内占用的比例；窗口间重复访问比率表示前后两个连续时间窗口的重复数据量占后时间窗口访问数据量的比率。窗口内重复访问比率和窗口间重复访问比率越高，表示时间局部性越强。为了探究 15 条工作负载的时间局部性，本小节采用离线方式对 I/O 跟踪记录进行了三组实验分析，分别是时间窗口大小的确定、时间窗口内重复访问比率分析以及时间窗口间重复访问比率分析。

（1）时间窗口大小的确定

在序列分析与预测中，时间窗口大小决定了观察数据的时间跨度。选择合适的时间窗口大小可以影响对数据的理解和对未来趋势的预测。较小的时间窗口可

中国知网 https://www.cnki.net

以捕获短期变化，而较大的时间窗口可以捕获长期趋势。而在访问请求序列中，在不同时间窗口内，工作负载的行为可能会有显著差异，这就对时间窗口大小的选择提出了挑战。

为了探究尽可能合理的时间窗口大小，地址访问间隔是一个需要探讨的问题。地址访问间隔指的是在一系列的数据访问操作中，对同一数据元素的两次连续访问之间的操作间隔，式（3.2）给出了地址访问间隔ΔC的数学定义：

$$\Delta C = num_{current} - num_{previous} \tag{3.2}$$

其中，$num_{current}$表示当前访问的操作序号；$num_{previous}$表示上一次访问的操作序号。这个概念在计算机科学中尤其重要，特别是在缓存管理、存储优化和算法设计等领域。

考虑到地址的庞大基数以及因分析历史访问信息需要占用SSD有限的RAM空间，本实验仅仅探究了 6 种不同大小的时间窗口，分别是 500、1000、1500、2000、2500 和 3000。通过计算平均地址访问间隔小于上述时间窗口大小的各个重复访问地址所包含的数据量占总重复访问数据的比重ρ，式（3.3）给出了比重ρ的数学定义：

$$\rho = \frac{count_{\overline{\Delta C} \le windows}}{count_{repeat}} \tag{3.3}$$

其中，ΔC表示地址的平均地址访问间隔；$windows$表示窗口大小；$count_{\overline{\Delta C} \le windows}$表示满足平均访问地址小于等于窗口大小的重复访问地址所包含的数据访问量。本实验旨在通过详细比较和分析不同时间窗口大小下所覆盖的重复访问地址数据的比重，从而选定一个既能够有效涵盖尽可能多的重复访问数据，又能保持较低空间开销的最佳时间窗口大小。通过对多个时间窗口所涉及的数据比重进行细致的评估，实验旨在找到一个平衡点，即在此时间窗口下，每个单位的空间开销能够覆盖最大量的重复访问信息。这样的选择不仅优化了数据存储和处理的效率，减少了不必要的资源消耗，还确保了数据分析的准确性和有效性。通过此实验，期望能够确定一个具体的时间窗口大小，它能够作为未来研究或应用中处理和分析重复访问数据时的标准，以实现更高效的数据管理和分析策略，从而提高整体的系统性能和用户体验。

实验结果如表3.2所示，通过表中数据可以发现，随着时间窗口的逐步扩大，15 个不同的工作负载所覆盖的重复访问地址的数据量占总体重复访问地址数据量的比例也相应增加。尤其值得注意的是，在时间窗口大小达到 1000 时，大多数工作负载的占比发生了显著的跃升，表明这一特定的时间窗口对于捕捉重复访问的地址显得尤为重要。然而，在时间窗口继续增大到 1000 之后，虽然所覆盖的重复访问地址数据的比重有所增加，但增长幅度相对较小，表明超过这一点后，增加窗口大小所带来的好处逐渐减少。鉴于使用更大的时间窗口只能换取较小的

中国知网 https://www.cnki.net

比例提升，而这种提升对于整体分析的贡献相对有限，因此，在后续的研究或应用中选择时间窗口时，采用 1000 这一特定大小显得更为合理。这一发现指导本文在平衡数据处理效率和覆盖范围的同时，能够更加精准地识别和分析重复访问的模式，为后续的优化策略提供了重要的依据。

表 3.2 重复访问地址平均地址访问间隔满足窗口大小条件的数据占比

工作负载&窗口大小	≤ 500	≤ 1000	≤ 1500	≤ 2000	≤ 2500	≤ 3000
HM_0	9.09%	18.92%	19.37%	19.90%	20.88%	22.17%
HM_1	55.73%	83.72%	88.01%	91.81%	93.90%	94.67%
PRN_0	8.16%	16.63%	16.79%	17.12%	17.70%	17.90%
PRN_1	23.09%	45.02%	45.67%	46.03%	46.39%	46.67%
PROJ_1	60.67%	96.99%	97.14%	97.21%	97.27%	97.30%
PROJ_3	2.92%	4.15%	4.42%	4.58%	4.81%	4.88%
PROJ_4	11.39%	20.40%	20.40%	20.40%	20.40%	20.42%
PRXY_0	62.08%	81.96%	87.89%	91.07%	91.72%	92.20%
RSRCH	7.42%	12.91%	26.91%	50.74%	73.21%	80.89%
SRC1_2	13.56%	26.21%	40.65%	45.60%	49.40%	51.69%
SRC2_0	12.75%	24.40%	33.24%	39.88%	42.84%	44.87%
SRC2_2	8.95%	18.06%	18.07%	18.11%	18.12%	18.14%
STG_0	21.72%	43.15%	44.69%	45.63%	46.14%	46.85%
USR_0	1.01%	10.89%	82.47%	82.54%	82.56%	82.60%
WDEV_0	12.05%	28.25%	44.08%	80.16%	83.13%	83.56%

（2）时间窗口内重复访问比率分析

在确定好窗口大小的情况下，本文开始探讨与评价各个负载的时间局部性，首先探讨的就是在各个时间窗口内部的重复访问比率。计算特定时间窗口内部的重复访问比率在计算机科学和数据管理领域具有重要意义，尤其是对于理解和优化时间局部性（Temporal Locality）这一概念。时间局部性是指在较短的时间内，某些数据或资源被多次访问的趋势。在不同的计算系统和应用中，这种特性可以被利用来提高系统的性能和效率。

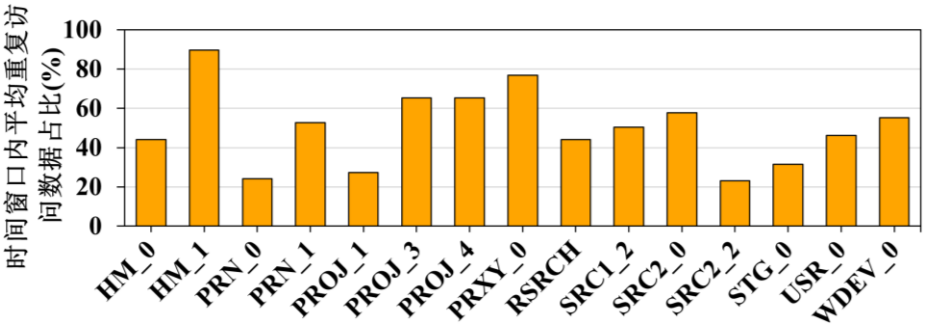


图 3.4 时间窗口内平均重复访问数据占比

中国知网 https://www.cnki.net

如图 3.4 所示，为各个工作负载在大小 1000 的时间窗口内平均重复访问的数据量占时间窗口内数据访问总数据量的百分比P_{window}，其数学定义如式（3.4）所示：

$$P_{window} = \frac{\sum_{i=0}^{num_of_windows} \frac{count^{i}_{repeat_in_window}}{window_count}}{num_of_windows} \tag{3.4}$$

其中，i为工作负载访问记录划分出来的窗口号；$num_of_windows$为工作负载访问记录划分出来窗口总数，值为[**工作负载记录长度/窗口大小（1000）+1**]；$window_count$为窗口内访问的数据量；$count^{i}_{repeat_in_window}$表示工作负载在第$i$个时间窗口内重复访问的数据量。从图中可以观察到，15 条工作负载记录的时间窗口重复访问数据的平均占比均在 20%以上，最好的情况下可以高达 80%（如图中工作负载 HM_1）。由此可见，在 1000 大小的时间窗口内部，各个工作负载展现出较好的时间局部性特征。

（3）时间窗口间重复访问比率分析

时间窗口间重复访问比率凸显了工作负载访问的跨时间窗口的时间局部性。这个概念指的是，如果当前时间窗口内访问的数据中有大量在上一时间窗口中已经出现过的数据，那么表明该工作负载在不同时间段具有较强的数据重用性。这种特征揭示了数据访问模式中的一个重要方面，即数据项在时间上的相互关联性和重访问的倾向。同时，也突显出了较强的跨时间窗口的时间局部性。

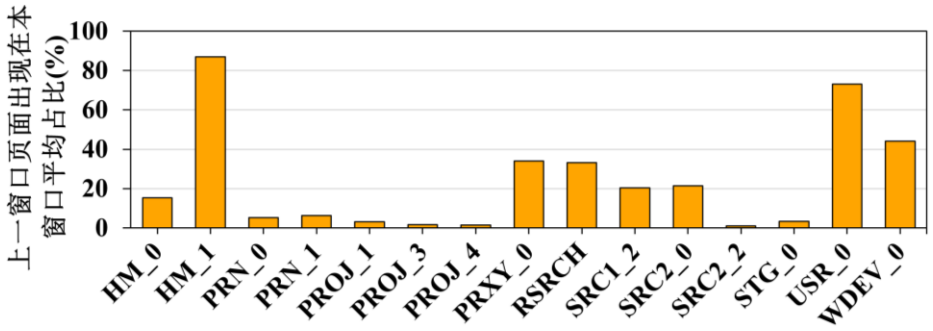


图 3.5 跨时间窗口平均重复访问数据占比

如图 3.5 所示，为各个工作负载在大小 1000 的时间窗口间平均跨时间窗口重复访问的数据占比$P_{window_to_window}$，其数学定义如式（3.5）所示：

$$P_{window_to_window} = \frac{\sum_{i=0}^{num_of_windows} \frac{count^{i}_{access_in_prewindow}}{window_count}}{num_of_windows} \tag{3.5}$$

其中，$count^{i}_{access_in_prewindow}$表示工作负载在第$i$个时间窗口内出现上一个时间窗口访问过的数据量。从图中可以观察到，15 条工作负载记录的时间窗口间重复访问比率有好有坏，例如 HM_1、USR_0 和 WDEV_0 在时间窗口为 1000 的跨时间窗口局部性较好，占比均超过 60%；然而，工作负载 PRN_0、PROJ_1、PROJ_3、

中国知网 https://www.cnki.net

PROJ_4、SRC2_2、STG_0 则表现出较差的跨时间窗口局部性。

总的来说，通过以上三组实验得出如下结论：**1）各个工作负载访问序列都具有一定的时间局部性，但具体的时间局部性指标存在一定的差异。**其中，时间窗口内的重复访问特征比较普遍；与此同时，部分工作负载的跨时间窗口重复访问特征比较突出，也存在一小部分工作负载在这一特征上表现的不是那么的明显。这里分析的主要原因可能是 1000 大小的时间窗口选取可能对于大部分工作负载有较好的效果。但是由于仍存在部分工作负载的平均地址访问间隔大多数分布在 1000 以外，所以单纯采用固定大小为 1000 的时间窗口可能对于此类特征的结果具有影响。**2）工作负载在相邻时间窗口的访问存在相似性。**由于各个工作负载存在相邻时间窗口访问数据部分重合的时间局部性特征，这对于按时间窗口进行基于访问历史访问预测提供指导依据，预测算法可以通过先前的时间窗口来预测下一个时间窗口可能访问的数据。

3.2.2 空间局部性分析

空间局部性（Spatial Locality）是指在特定时间内，数据访问活动倾向于在当前请求逻辑地址空间的邻近区域内集中发生的现象。此概念在计算机科学，特别是在 SSD 性能优化与管理领域中扮演着重要的角色。

从先前的表 3.1 中可以看出，15 条工作负载访问记录的邻近读比例来看，大部分的工作负载的读请求前后的空间局部性并不高，最好的也仅仅达到 31.1%，其他的工作负载的邻近读比例均小于 30%。这一现象表面上看来空间局部性并没有预期中那么好，为此，本小节从空间分布着手，进行了更加细致的分析。

从图 3.2 中可以观察到，例如工作负载 PRN_1 的读访问请求，可以肉眼的观察出顺序访问、重复访问以及乱序访问这三种访问特征。但是在乱序访问中，其实还包含一种空间访问特征是没法肉眼简单观察出来的，即点簇访问。点簇访问指的是单从时间顺序观察的连续访问序列在地址空间中并不存在交集，但从地址空间角度出发，这一系列的访问的地址空间可以最终聚集成一段连续的地址空间。如图 3.6 充分展示了点簇访问这一空间访问特征。其中，灰色块代表一个访问请求，方块的底边对应的纵坐标表示请求的起始地址，方块的顶边对应的纵坐标表示请求的结束地址，方块的高长为访问的大小。从时间维度分析（如图中红色箭头），大部分访问请求涉及到的地址空间前后是不相交的。这也是现有绝大多数预测算法在进行前期模式识别中存在的问题，即单从时间维度顺序分析。然而，从地址空间分析出发，可以看出，这段访问记录通过地址相交结合的方式，可以划分成三个地址段，如图中的 A、B、C。在每个地址段中，如果其中一个地址空间的数据被访问，则代表其他数据在未来也会被访问。这也就指明了地址分段在访问预测中具有很强的指导意义。

中国知网 https://www.cnki.net

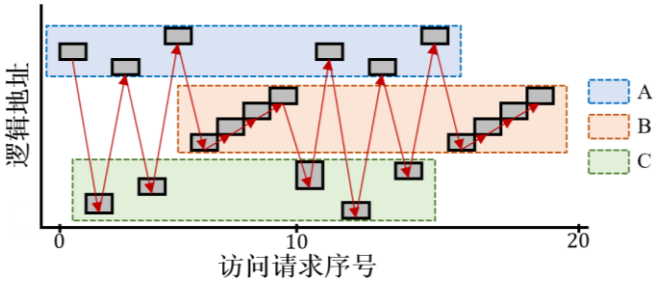


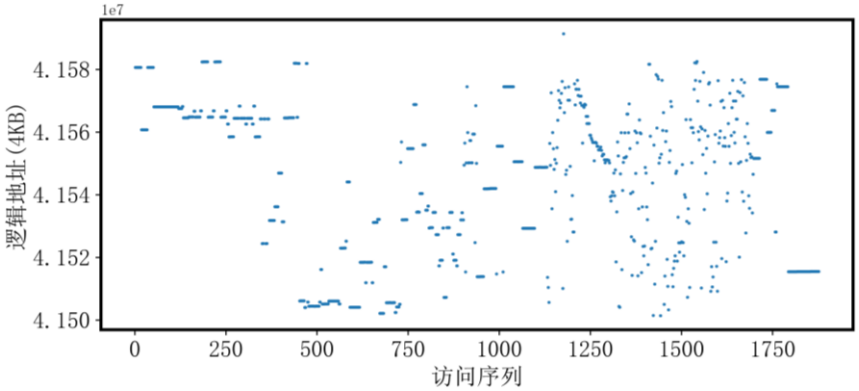
图 3.6 点簇访问特征图

为了加强理解，本文进行了两组实验来证明地址分段的出处以及意义，实验如下：

（1）访问数据空间分布对于地址粒度的敏感度分析

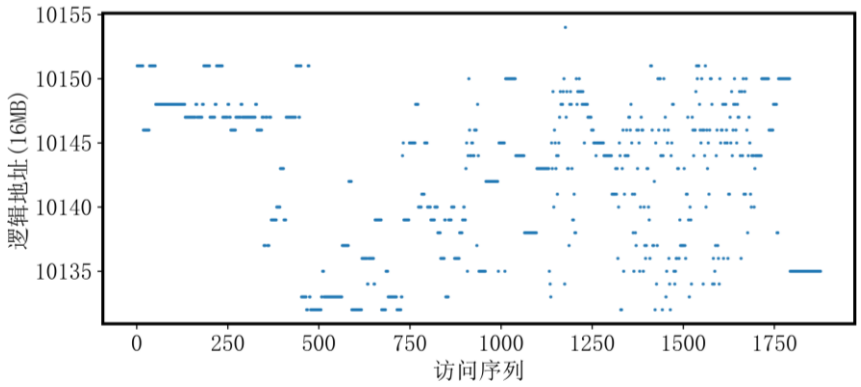
本实验的初衷是为了证明地址分段对于访问空间模式识别具有去干扰的作用。依然以工作负载 PRN_1 为例，如图 3.7 所示，是工作负载一段读访序列在不同地址段粒度（4KB、16MB）划分下的空间访问分布，实验对其访问序号进行重新编号。从图中可以观察到，在 4KB 粒度下的，PRN_1 在前半段部分表现了顺序访问和重复访问行为，中间穿插着部分的乱序访问行为，而在后半段部分表现出了极强的乱序访问特性。然而，当以 16MB 大粒度划分访问时，可以在观察到之前具有极强乱序访问特性的后半段访问被大粒度划分的地址空间所吸收，进而演变成了具有稳定空间局部性的顺序访问和具有稳定时间局部性的重复访问行为。这一现象表明，**1）采用大粒度地址划分可以掩盖掉乱序访问造成的预测干扰。**

然而，单纯的采用统一粒度进行划分仍然存在一定问题。不同数据地址段的大小是多样性的。采用统一粒度进行划分，若粒度过小则地址段无法涵盖所有的相关数据；若地址过大则预取时会发生读放大现象，导致主机端访问请求阻塞，影响 SSD 整体的 I/O 性能。例如，图 3.7 中需要采用相较于原始 4KB 粒度 4096 倍大小的地址粒度进行划分，这对于单次数据预取来说就会有一个 16MB 的读取开销，会严重影响主机端请求的执行效率。不仅如此，有限的 RAM 空间也不允许读取缓存这么大的数据空间。



(a) 4KB 粒度

中国知网 https://www.cnki.net



(b) 16MB 粒度

图 3.7 PRN_1 在不同地址段粒度划分下的空间访问分布

动态自适应的地址分段则可以完美解决此类问题，即通过历史访问信息周期性地构建并更新地址段，以满足地址段划分结果的时效性。此方法可以尽最大可能将空间局部性较强且相关性较强的地址空间结合到一起，形成地址段。各个地址段大小不等，且数据空间较小。然而，带来的额外空间开销就是每个地址段都需要一个元数据来记录该地址段的起始地址和结束地址。算法 3.1 给出了动态自适应的地址分段算法的详细描述。

算法 3.1 动态自适应的地址分段算法

输入: $history$ 历史访问记录（记录格式: $< start_address, end_address >$）

输出: $segment$ 地址段集合 （地址段格式: $< start_address, end_address >$）

```
sort(history, start_address) /* 按照起始地址从小到大顺序对history重新排列 */
for each access in history do /* 循环遍历上一时间窗口的访问记录 */
  if is_first(access) is true then /* 如果是第一个记录，需要初始化地址段 */
    segment_start_address ← access.start_address
    segment_end_address ← access.end_address
  else
    if access.start_address ≤ segment_end_address then /* 地址相交，属于同一段 */
      if access.end_address > segment_end_address then
        segment_end_address ← access.end_address
      end if
    else /* 地址不相交，出现新的地址段，保存现有的地址段，再进行更新 */
      save_segment(segment_start_address, segment_end_address)
      segment_start_address ← access.start_address
      segment_end_address ← access.end_address
```


中国知网 https://www.cnki.net

```
15:         end if
16:         if is_last(access) is true then
17:             save_segment(segment_start_address, segment_end_address)
18:         end if
19:     end if
20: end for
21: output segment
```

算法 3.1 主要分为两个步骤：

步骤 1：首先，针对上一时间窗口的历史读访问记录$history$，按照记录项中起始地址$start_address$从小到大的顺序进行重排。

步骤 2：针对已经排好序的历史读访问记录$history$，开始进行遍历，将前后地址存在相交的地址空间进行合并，形成地址段并保存在地址段集合$segment$中。地址段集合$segment$需要在每一次时间窗口结束后，进行更新操作，以应对工作负载访问模式的变化。

（2）地址段访问种类分析

依据时间窗口进行的地址分段可以有效掩盖因点簇访问对访问预测造成的干扰。为了深入研究读访问请求的空间局部性，实验量化分析了地址段访问模式在各个工作负载中的种类以及比例。

首先，经过根据地址段访问模式的时空局部性特征分析发现，地址段访问种类可以分为三种，即跨时间窗口的重复访问、时间窗口内的扩张访问以及稀疏访问。

跨时间窗口的重复访问：此访问模式指的是在相邻的前后时间窗口中，在前一个时间窗口中本访问过的地址段数据，在下一个时间窗口中大概率也会被访问。这是一种基于空间局部性的时间局部性特征，即在地址段内部各个地址具有较强的空间局部性，在时间窗口间具有较强的时间局部性。

时间窗口内的扩张访问：此访问模式指的是在未结束的时间窗口中被访问的地址段大概率处于成长期，即很有可能在时间窗口结束之前，其地址段相邻的地址数据很有可能会被访问，进而当前地址段边界前后扩张。其在时间窗口内具有较强的空间局部性。

稀疏访问：此访问模式指的是在时间窗口中此类地址段数据一般只会被一次读访问请求所访问。无论是在时间窗口内，还是跨时间窗口的情况，其均具有低时间局部性、低空间局部性的特点。

为了量化分析地址段访问模式在各个工作负载中的比例，本文设置了一组实验。即以 1000 为时间窗口大小，对 15 条工作负载进行划分，并依次统计各个工

中国知网 https://www.cnki.net

作负载时间窗口内三种地址段访问的地址段数的比重。实验结果如图 3.8 所示，其直观地展示了三种段访问模式所涵盖的地址段数量的比重。其中，蓝色部分表示的是跨时间窗口的重复访问，其具有有较强的空间局部性和时间局部性；橙色表示的是时间窗口内的扩张访问，其具有较强的空间局部性；灰色表示的是稀疏访问，其在时间局部性和空间局部性表现较弱。蓝条越长，表明跨时间窗口的重复访问越多，突出工作负载地址段的时间局部性越强。相反，黄色条越长，表明时间窗口内的扩张访问越多，突出工作负载地址段的空间局部性越强，因为地址段内的地址均是连续的。

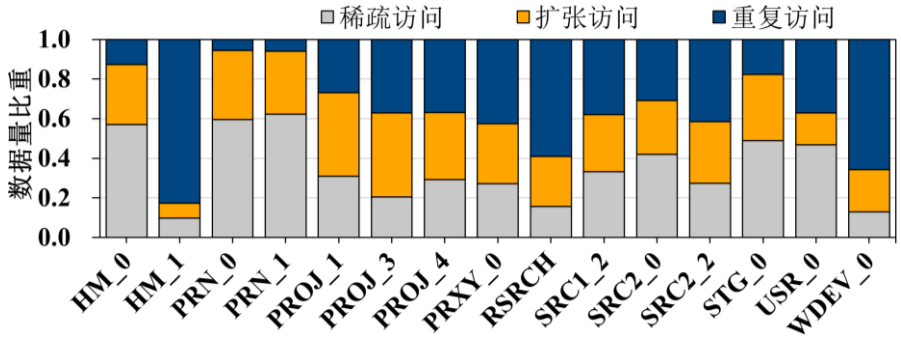


图 3.8 时间窗口内三种地址段访问的地址段数的比重

实验结果表明，在大多数工作负载地址段访问中，扩张访问和重复访问所包含的数据量均占 50%以上。例如，HM_1 的重复访问所占的比重就超过了 80%，而 PROJ_3 的扩张访问超过了 40%。综合两种访问，PROJ_3 在重复访问和扩张访问的数据量之和高达令人深刻的 90%。在此结果中，本文得出了两个地址段访问的结论：**2）跨时间窗口的重复访问具有较强的空间局部性和时间局部性；时间窗口内的扩张访问具有较强的空间局部性。**

3.3 基于模式局部性引导的数据预取技术

基于以上对 15 条工作负载的时空局部性特征分析结果，本文提出了基于模式局部性引导的数据预取（Pattern Locality Guided Data Prefetching，PLGDP）技术，通过预取数据到 SSD 的 RAM 中以快速响应未来的读访问请求来优化读取性能。设计的关键是如何设计较高精确度的数据预测算法以及如何设计引入预取请求的 I/O 请求调度策略。数据预测算法的精确度严重影响了数据预取效果，精确度越高，预取的无用数据越少，预取操作的增益效果越强，系统的读性能也就越强。由于请求队列中预取操作请求的加入，SSD 要额外执行更多的读取操作，导致读读冲突（Read/Read Conflict，RRC）、读写冲突（Read/Write Conflict，RWC）增加，会影响到主机端 I/O 请求的执行。为了最小化预取操作对主机端 I/O 请求的负面影响，请求队列中在预取请求和主机端请求之间进行合理的调度是很有必要的。针对以上两点设计关键，PLGDP 的框架包括两个组件：基于地址分段的

中国知网 https://www.cnki.net

数据预测算法模块（组件一）、基于预取请求感知的 I/O 请求调度策略模块（组件二）。组件一满足对预测算法高精确度的需要，旨在根据访问历史地址分段和当前访问请求对未来读访问进行预测；组件二降低了数据预取操作对主机端请求的影响，旨在在 SSD 请求队列的预取请求和主机端请求之间进行调度。如图 3.9 展示了基于模式局部性引导的数据预取技术的框架图，其中红色箭头表示指令流、蓝色箭头表示数据流、紫色箭头表示组件间的信息交换。组件一会将预测结果转换成预取指令，并传送给请求队列。组件二通过组织调度请求队列中预取请求与主机端 I/O 请求，以降低预取操作对主机端请求的影响。

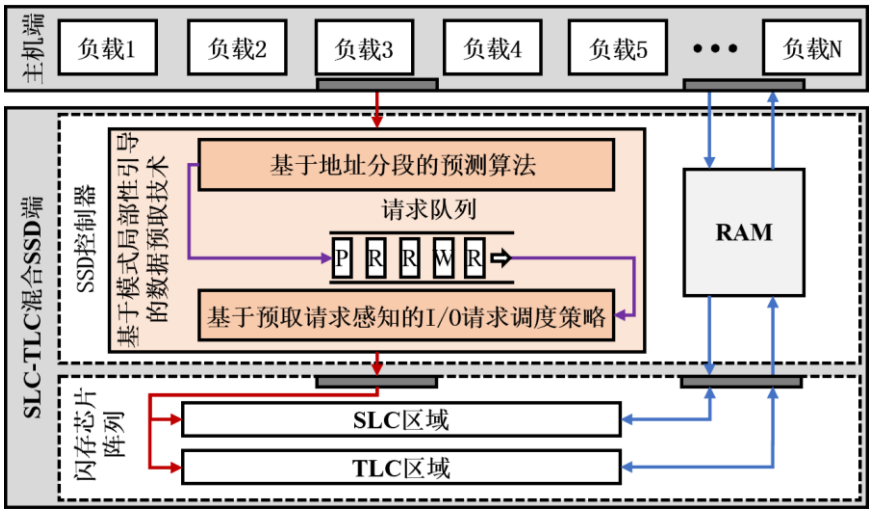


图 3.9 PLGDP 框架图

3.3.1 基于地址分段的数据预测算法

目前大多数数据预测算法都是基于时间线性的分析其工作负载的访问模式或数据的冷热性，即按照访问请求到来顺序分析前后请求所涉及的数据地址的连续，包括地址偏移、大小等。然而，据本文在先前实验所展示的，对地址空间进行分段，更有助于识别其时空局部性特征。因此，地址分段对数据预测的准确性具有较强的提升效果。

本文提出的基于地址分段的数据预测算法是基于历史访问信息的，旨在识别过去读访问请求的局部性特征。预测算法引入了“先时间-后空间”的局部性感知方法，强调了时间感知和空间感知对于预测局部性的重要性。这个预测算法由三个核心部分组成。第一个组成部分涉及周期记录和更新读访问请求历史记录，并将地址空间划分成大小不一的地址段（地址分段的具体执行算法如算法 3.1 所示）；第二个组成部分专注于地址段跨时间窗口重复访问预测，即地址段的时间局部性特征；第三个组成部分专注于地址段时间窗口内扩张访问预测，即地址段的空间局部性。图 3.9 展示了基于地址分段的数据预测过程。

需要注意的是，第一个组件记录上一个时间窗口的地址段集合。这些记录的

中国知网 https://www.cnki.net

地址段信息随后会在第二个组件中被利用，以指导地址段跨时间窗口重复访问预测，利用地址段的时间局部性。如果当前到来的读访问请求与地址段集合中任何地址段都不匹配，则第三个组件将会利用空间局部性来预测后续的读访问请求的数据，即地址段时间窗口内扩张访问预测。

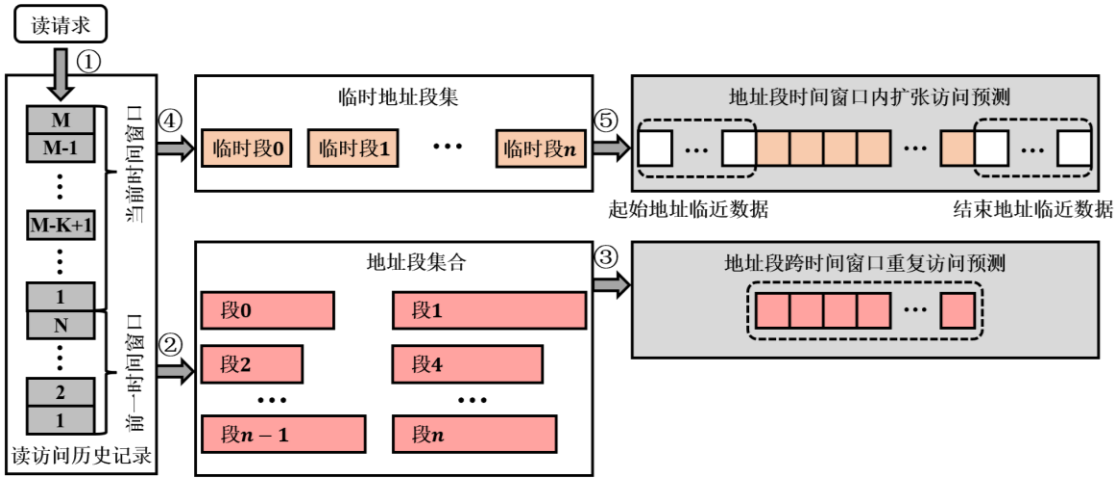


图 3.10 基于地址分段的数据预测过程

（1）地址空间分段：在每个时间窗口中，算法记录从主机端传入 SSD 控制器读访问请求的起始地址和结束地址（这里指的地址是逻辑地址），直到历史记录的请求数量达到特定阈值 N，这里阈值的大小等于时间窗口大小，即 1000（此过程如图 3.9 中①）。一旦达到这个阈值，地址空间分段过程开始（此过程如图 3.9 中②）。首先，按照记录项的起始地址从小到大的顺序重新排列。接下来，对排列好的记录进行遍历，合并任何具有重叠地址的记录。这个合并过程创建了地址段，具体算法实现可参考算法 3.1。最后，合并结束的地址段存储在一个地址段集合中，这个集合作为下一个时间窗口中数据预测的基础。需要注意的是，地址段集合的生命周期只有一个时间窗口，即在每个时间窗口结束后，本周期所使用的地址段集合将会被本周期的历史记录所更新。这么做的好处在于可以实时适应访问模式的动态性，保证地址段集合有效性。

（2）地址段跨时间窗口重复访问预测：当从主机端传入 SSD 控制器读访问请求所涉及的地址空间范围匹配到地址段集合中时，它被认为是一次命中，相对应的地址段被称为热段（此过程如图 3.9 中③）。根据前面地址段访问分析，可以观察到，当地址段内有数据被访问时，那么在当前时间窗口内，该地址段的其他数据也有很高的可能性会被访问。因此，当匹配到地址段时，该地址段内的剩余数据页面将会作为预测算法预测的后续被读取访问的数据。

（3）地址段时间窗口内扩张访问预测：如果读访问请求所涉及的地址空间在地址段集合中匹配不到任何地址段，这意味着该读访问请求属于在前一个时间窗口中不存在的新的地址段空间。为了处理这种情况，预测算法利用空间局部性，

中国知网 https://www.cnki.net

并引入了一种方法，即在当前时间窗口中从 M（$0 \le M \le N$）个历史访问记录中选择最近记录的 K 个读访问请求来构建一个临时地址段集（此过程如图 3.9 中④）。这个临时地址段集合包含了当前时间窗口内临近当前读访问请求的 K 个访问请求（包含当前读访问请求），其中包含当前读访问请求的地址段是一个临时的地址段段，根据前面地址段访问分析，临时地址段很大概率处于边界扩张期，即当前临时段的前后地址空间数据很大概率会被访问。因此，预测算法将临时地址段的前后数据作为后续被读取访问的数据（此过程如图 3.9 中⑤），需要注意的是这里预测的数据量是等于当前读访问请求的大小。

结合前面所述，基于地址分段的数据预测算法的详细描述如算法 3.2 所示。

算法 3.2 基于地址分段的数据预测算法

输入： $access$ 读访问请求（格式: $< start_address, size >$）

输出： $prediction$ 预测结果 （格式: $< start_address, end_address >$）

```
1:  prediction ← [ ] /* 初始化预测结果，保存预测数据的地址空间范围数组 */
2:  fifo_insert(access_hisoty, access) /* 先进先出将请求插入到当前时间窗口记
    录中 */
3:  update_record_count(+1) /* 更新历史记录次数 */
4:  start ← access.start_address
5:  end ← access.start_address + access.size
6:  size ← access.size
7:  segment_set ← get_segment_set( )
8:  segment ← lookup(segment_set, access)
9:  if segment is not null then /* 在地址段集合中找到匹配地址段 */
10:   seg_start ← segment.start_address
11:   seg_end ← segment.end_address
12:   prediction.append(< seg_start, start − 1 >)
13:   prediction.append(< end + 1, seg_end >)
14: else /* 在地址段集合中找不到匹配地址段 */
15:   temporary_segment_set ← address_segmentation(access_history.cur(last_K))
16:   segment ← lookup(temporary_segment_set, access)
17:   seg_start ← segment.start_address
18:   seg_end ← segment.end_address
19:   prediction.append(< seg_start − size, seg_start − 1 >)
20:   prediction.append(< seg_end + 1, seg_end + size >)
21: end if
```


中国知网 https://www.cnki.net

22: $count \leftarrow get_record_count()$
23: **if** $count \geq N$ **then** /* 如果记录次数达到阈值，则进行地址空间分段更新 */
25: $set_record_count(0)$
26: $segment_set \leftarrow address_segmentation(access_history.pre)$
27: $update_segment_set(segment_set)$
28: **end if**
29: *output prediction*

根据预测算法预测出来的结果，SSD 控制器会生成读取请求，并将它们添加到请求队列中，以从 NAND 闪存阵列中检索预测的数据。需要注意的是，过于激进的预取方法可能会阻塞主机请求，从而导致性能下降。为此，本文设置了预测条件，即预测算法发送预取请求到请求队列被设置了条件，类似于 Linux 中的预取机制[80]，它通过仅在前一个预取的数据获得命中时，才会预取额外的后续数据。此外，预取的数据量与连续预取数据的命中次数成正比增加关系。在本文中，如果到来的读访问请求命中地址段集合或临时地址段集中的任意一个，即视为命中，并发出两个读访问请求数据页数量的预取请求。如果这两个预取的数据页中至少有一半数据页获得命中，则预测过程就会继续。这个过程只有在命中的预取数据页数量降至总预取数据页数的一半以下时才结束。

除此之外，在指导数据预取过程设计的主要原则是尽量减少其对主机请求的负面影响。为了实现这一目标，请求队列中请求的调度策略尤为重要。

3.3.2 基于预取请求感知的 I/O 请求调度策略

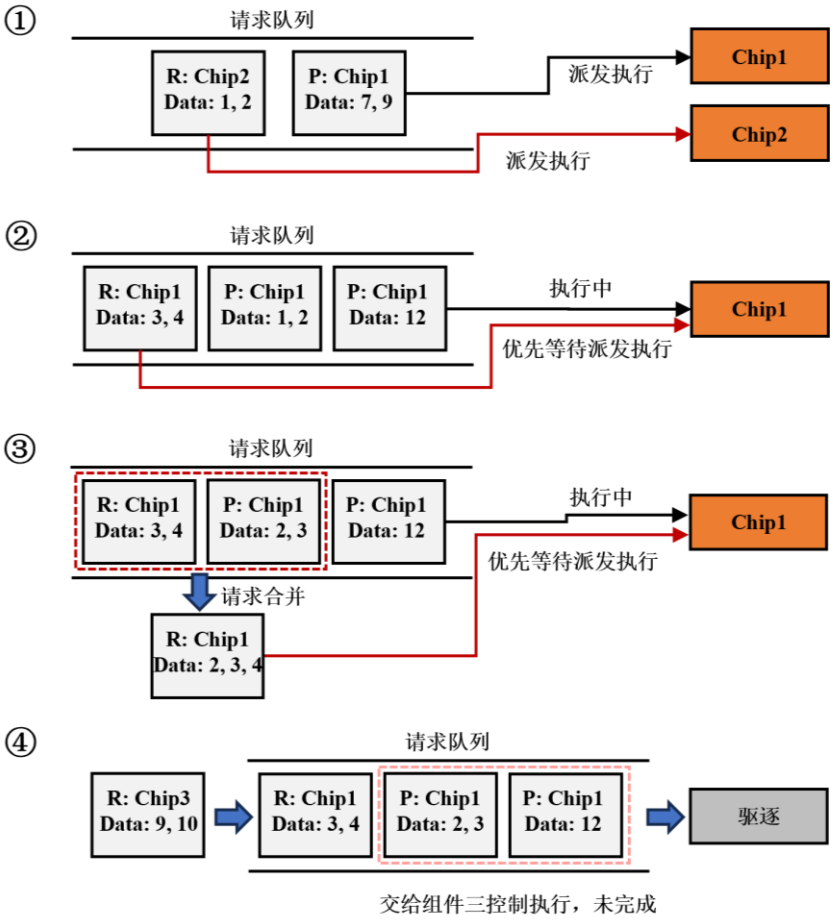
为了有效管理请求队列中因预测算法引入的新增预取请求，并缓解由此产生的读请求冲突与写请求冲突，采取针对性的处理调度策略变得极为关键。预取技术虽然能够显著提高系统的响应速度和数据处理效率，但未经优化的预取策略可能会引入额外的请求冲突，从而降低系统性能。因此，针对于两种冲突，不同情况采用单独且合理的处理调度策略是至关重要的。

读/读冲突（Read/Read Conflict, RRC）：该冲突通常指的是两个或多个读访问请求试图同时访问 SSD 内同一个并行单元（例如通道、闪存芯片、晶片、平面、数据块或页面），从而造成一系列的问题，进而导致性能下降和访问延迟增加。可能导致的问题包括：资源竞争（当多个读取请求试图同时访问同一块数据时，这些请求可能需要排队等候，因为存储设备的某些并行单元可能无法同时处理所有请求。）、性能下降（由于资源竞争，每个读取请求的处理时间可能会增加，导致整体的数据访问速度变慢。）、控制器负载增加（在处理大量并发读取请求时，SSD 控制器的工作负载会增加，可能导致处理效率下降。）。在本文中所指的是主机端读请求与预取请求的冲突。

中国知网 https://www.cnki.net

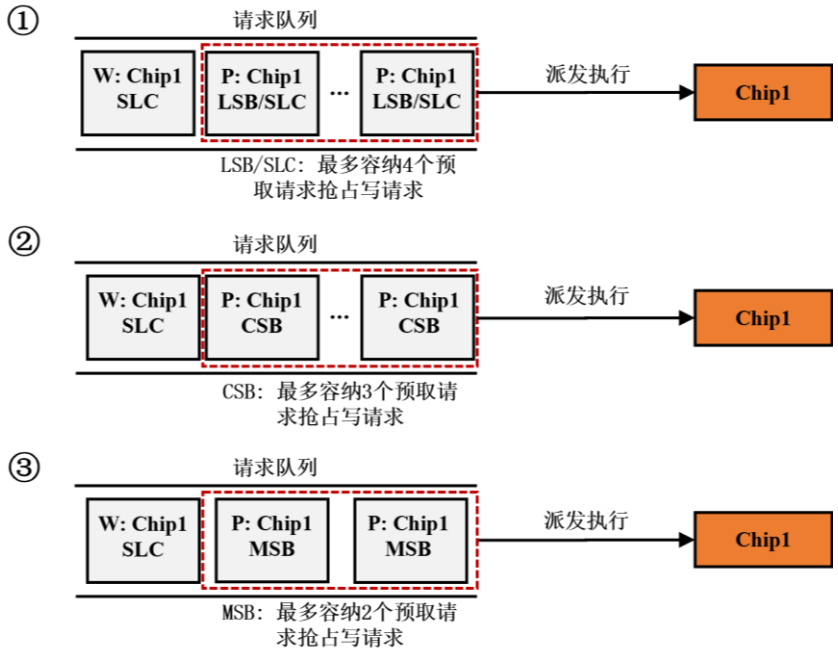
读/写冲突（Read/Write Conflict, RWC）：该冲突通常指的是读写访问请求操作同时针对于同一个数据区域。由于 SSD 使用的是 NAND 闪存作为存储介质，其在同时读写操作执行下会出现一系列的问题，因此如果写访问请求开始在某一区域执行时，此区域的数据是旧数据，读请求需要等到新数据写入到 NAND 闪存之后再进行读取操作的执行。可能导致的问题包括：数据访问延迟（当读取请求和写入请求同时针对同一存储单元时，读取操作通常需要等待写入操作完成才能执行。这种等待会导致数据访问延迟，影响 SSD 的性能。）、数据一致性问题（如果读取操作在写入操作完成之前访问了相同的数据块，可能会读取到不完整或过时的数据。这会导致数据一致性问题，尤其是在数据库和文件系统等对数据一致性要求严格的应用中。）、控制器负载增加（在处理读/写冲突时，SSD 的控制器需要协调并管理这些冲突，这可能会增加控制器的计算负载，从而影响整体性能。）。在本文中所指的是主机端写请求与预取请求的冲突。

针对于上述两种冲突，本文提出了基于预取请求感知的 I/O 指令调度策略，策略执行示意图如图 3.11 所示。



(a) 针对 RRC 冲突 I/O 请求调度策略

中国知网 https://www.cnki.net



(b) 针对 RWC 冲突 I/O 请求调度策略

图 3.11 基于预取请求感知的 I/O 请求调度策略

首先，在 RRC 情况下，由于请求数据的物理地址保持不变，解决此冲突变得相对简单。这个可以通过跟踪访问地址来实现，以便了解读访问位置。当预取请求和主机读访问请求指向不同的并行闪存芯片时，这一点变得至关重要，因为此时它们可以同时执行。因此，一旦所有主机读访问请求都已经派发执行，预取请求仍然可以继续使用其他可用的并行闪存芯片处理预取请求（如图 3.11a 中的①）。然而，在执行的预取请求给新到达的主机读取请求造成阻塞延迟的情况下，主机请求只能容忍单次预取请求的延迟，即预取主机端读访问请求抢占未被派发执行的预取请求（如图 3.11a 中的②），优先执行主机端读访问请求，以避免显著的阻塞延迟。需要注意的是，如果预取请求在长时间内未得到完成，它可能会因为两个主要原因被取消。其一是，如果新的主机读访问请求读取与预取请求相同的数据，预取请求将与它进行合并，并形成优先级高于预取请求的主机端请求（如图 3.11a 中的③）。其二是，如果预取请求预测不准确，那么该预取请求会在请求队列中保存一段时间，当队列已满，再有请求加入到队列中时，它将会被新到来预取请求或主机端请求从请求队列中驱逐出去（如图 3.11a 中的④）。

其次，在 RWC 场景中，非常重要的调度策略是优先处理预取请求。这样做可以避免写请求较长的延迟时间带来的显著延迟。预取请求的高优先级执行确保了预取请求能够高效处理。但是为了与主机写请求保持公平，允许预取请求在某些约束条件下先于主机写请求处理。由于写请求数据通常被定向到 SLC 缓存区，该缓存区受益于较短的延迟，因此 RWC 的严重程度主要取决于预取请求数据的来源地位置。当预取的数据位于具有最长延迟的 MSB 页面时，RWC 变得更加严

中国知网 https://www.cnki.net

重，即预取操作会阻塞写访问请求很长时间。相反，从 LSB 页面或 SLC 缓存区读取预取的数据可以最小化 RWC 的影响。本文通过设置三个等级的抢占方式来解决 RWC，即当从 LSB 页面（或 SLC 缓存区）预取数据时，最多允许 4 个预取请求来阻塞一个主机写请求（如图 3.11b 中的①）；当从 CSB 页面预取数据时，最多允许 3 个预取请求来阻塞一个主机写入请求（如图 3.11b 中的②）；当从 MSB 页面预取数据时，允许最多 2 个预取请求来阻塞一个主机写请求（如图 3.11b 中的③）。通过这种方式，写性能仍然可以在满意的水平上保持一致。

3.4 预取技术性能分析

本节主要对本章提出的基于模式局部性引导的数据预取（PLGDP）技术进行实验分析与效果评估，目的是通过实验验证该技术对提升存储系统性能的实际影响。首先，介绍实验平台的具体搭建，包括实验平台的具体框架、框架内各个组成部分的功能；随后，介绍相关的实验环境，包括硬件参数、模拟器参数、预取技术参数等；最后对 PLGDP 技术在几个关键性指标维度上进行了全面的效果分析和评估，具体包括预测算法的准确性分析、RAM 区域读缓存命中率分析、读放大情况分析以及混合 SSD 读写性能分析。通过这些维度的详细分析，本文旨在展示 PLGDP 技术在提高存储系统性能方面的实际应用价值，从而为未来的存储系统优化提供有力的技术支撑。

3.4.1 实验平台搭建

本文基于 FEMU[81][82] 成功地实现了一个创新的平面级混合的 SLC-TLC 混合 SSD，用来在其上进行复现 15 条不同工作负载的 SSD 访问行为。FEMU，首次在 2018 年 FAST'18 国际会议上被公开介绍，是一种基于追踪事件的 NAND 闪存内存模拟器，其能够在真实计算机环境中通过划分一定的内存空间来模拟虚拟 SSD 的运作机制。通过这种方式，当用户对这个虚拟 SSD 执行读写操作时，FEMU 能够精确地捕捉到这些操作请求，并将它们转发给一个专门的 SSD 模拟管理程序。该程序负责对虚拟 SSD 内部的数据进行相应的修改操作，并且能够详细计算出每一个请求指令的执行时间，这包括了数据通道的传输时间、内存页面的访问时间以及由于硬件资源占用而导致的排队等待时间。这样的实验设置不仅确保了实验的高度仿真性和准确性，而且还允许研究人员在一个受控环境中详细观察和分析 SLC-TLC 混合 SSD 在处理不同工作负载时的行为表现，从而为进一步优化 SSD 的设计和应用提供了宝贵的数据和见解。实验框架的具体示意图详见图 3.12，该图详细展示了实验平台的结构与工作原理，提供了一个对运行机制的直观理解。

中国知网 https://www.cnki.net

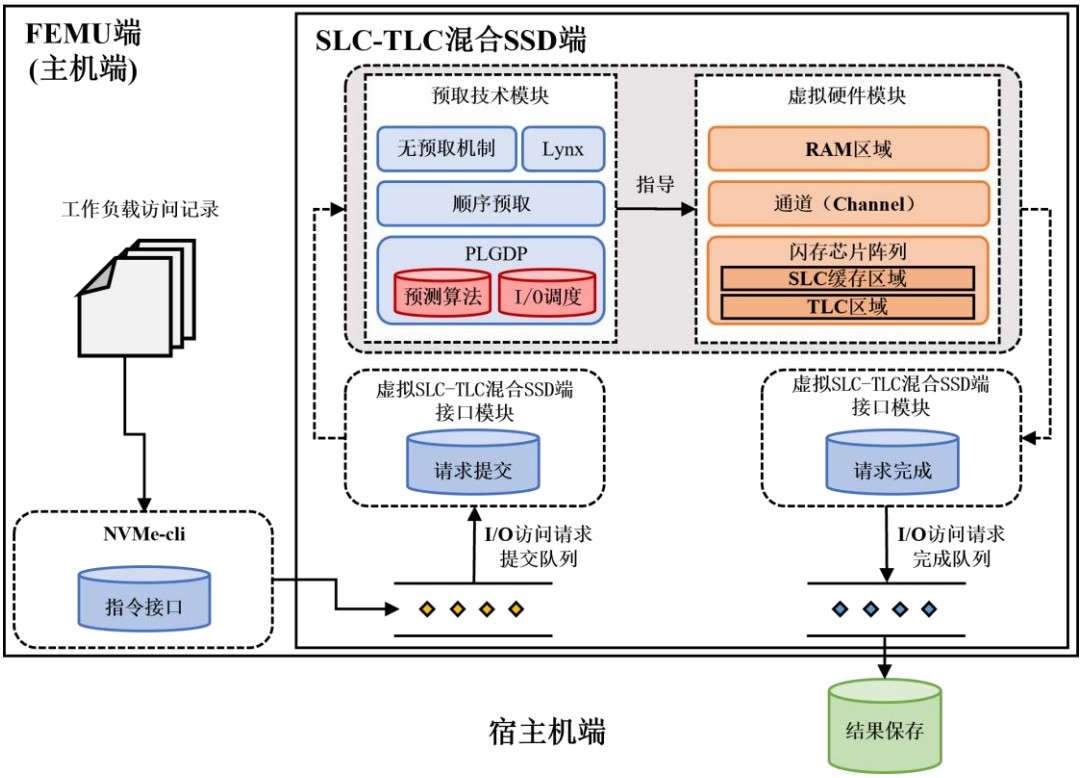


图 3.12 实验平台框架示意图

整个系统可以划分成三个主要角色：宿主机端、FEMU 端以及 SLC-TLC 混合 SSD 端。其中，宿主机端指的是实验采用的真实计算机，其主要作用是跟踪记录各个访问请求的执行延时以及 SLC-TLC 混合 SSD 的硬件状态信息；FEMU 端指的是在真实计算机内部运行的虚拟机，其可以提供和真实操作系统相同的 I/O 功能，用于在其上复现工作负载访问行为；SLC-TLC 混合 SSD 端是一个模拟混合 SSD 内部硬件的线程，用于实时接收 FEMU 端发送过来的 I/O 访问请求，并通过修改内部硬件属性、计算访问请求执行延时来返回给 FEMU 端，并最终透过 FEMU 端将执行情况、混合 SSD 内部信息保存到主机端，以便后续的性能分析。

在该系统中，FEMU 端会通过 nvme-cli 工具[83]发送 I/O 请求到混合 SSD 请求提交队列中。预取技术模块会实时感知到来的 NVMe 协议 I/O 访问请求，通过其内置的预取技术指导虚拟硬件进行相应的预取操作与信息修改。需要注意的是，实际的 I/O 操作以及访问请求完成不在此模块中进行。

虚拟硬件模块是基于平面级混合方式模拟的 SLC-TLC 混合 SSD，其模拟 RAM 区域页面的管理、SLC-TLC 混合闪存芯片的可访问信息以及数据占用情况等基本的 NAND 闪存功能。当虚拟硬件模块接收到 FEMU 端 I/O 访问请求或预取请求时，其会更新相关的硬件信息，并计算出指令时延返回到 I/O 访问请求完成队列中。最终结果将以文本文件形式保存到主机端。

中国知网 https://www.cnki.net

3.4.2 实验环境

本实验采用的 FEMU 运行在一个真实的物理机上，其软硬件配置信息如表 3.3 所示：

表 3.3 物理机软硬件配置

角色端	Linux 发行版本	Linux 内核版本	CPU 型号	核心数	内存
宿主机端	Ubuntu22.04	6.5.0-21-generic	AMD Ryzen(TM) 9 5900HX CPU @ 3.3GHz	8	64GB
FEMU 端	Ubuntu22.04	5.15.0-76-generic	/	/	20GB

宿主机端系统软件直接运行在真实物理机上，作为测试结果保存方。FEMU 端的系统软件则运行在虚拟机中，以模拟连接 SLC-TLC 混合 SSD 真实的物理机器。SLC-TLC 混合 SSD 端包含在 FEMU 端中，作为一个虚拟 SSD 进程，但在 FEMU 端内用户来看，其是一个可以读写的 SSD 硬件设备。

本实验的任务负载采用前面提到的剑桥微软研究院公开 15 条工作负载 I/O 块访问跟踪记录，每个工作负载的 I/O 访问记录由 1000001 条 I/O 访问请求组成，每条访问大小从 8 个扇区（4KB）到 1024 个扇区（512KB）大小不等。由于工作负载的作用物理空间大小不一，为了充分体现本方法的效果并且符合现实场景，针对不同工作负载的 SLC-TLC 混合 SSD 的闪存介质大小设置不同，具体 SLC-TLC 混合 SSD 的设置如表 3.4 所示：

表 3.4 SLC-TLC 混合 SSD 模拟器参数配置

参数名称		数值大小	参数名称		数值大小
闪存类型		SLC/TLC	体系架构	混合方式	平面级
操作延迟	读（页）	$40\mu s$(SLC)		通道数	8
		$56.5\mu s$(LSB)		闪存芯片数/通道	4
		$77.5\mu s$(CSB)		晶片数/闪存芯片	4
		$106\mu s$(MSB)		平面数/晶片	1
	写（页）	$800\mu s$(SLC)		页面大小	8KB
		$820.5\mu s$(LSB)		GC 阈值	40%(常规)
		$2225\mu s$(CSB)			10%（高压）
		$5734\mu s$(MSB)		TLC 区域空间	1.5 访问空间
	擦除（块）	$2ms$(SLC)		SLC 区域空间	1/10TLC 区域空间
		$3ms$(TLC)			
	通道（页）	$30\mu s$		RAM 读区域空间	1/1000读页面数
	RAM（页）	$0.5\mu s$(读)	预取参数	时间窗口	1000
		$2\mu s$(写)		最大预取大小	20 页（160KB）

其中，表 3.4 中的读页面数、访问空间等参数的值可参考先前访问记录的分

中国知网 https://www.cnki.net

析结果表 3.1。GC 阈值分成常规和高压两个档次，当 SLC 空闲空间减少到常规档次时，SLC-TLC 混合 SSD 会在空闲时，对 SLC 缓存区域进行 GC 操作；当 SLC 空闲空间减少到高压档次时， SLC-TLC 混合 SSD 强制进行 GC 操作。

最后，本实验的 RAM 区域采用 LRU 策略进行管理，采用的对照预取技术包括无预取机制（下文简称 Baseline）、针对 SSD 性能模型的学习式 Linux 预取机制（A learning linux prefetching mechanism for ssd performance model，下文简称 Lynx）[56]、顺序预取机制（Sequential Prefetching，下文简称 SP）[84]。需要注意的是，Lynx 是 LINUX 系统自带的基于马尔科夫状态矩阵的数据预取技术，但是其应用于 FEMU 端，为了公平起见，本文在 SLC-TLC 混合 SSD 端内部进行了实现。SP 则是根据当前数据读访问请求，对未来可能的访问的数据进行顺序预测，并产生预取请求加入到请求队列中。这两类预取技术均在真实生产环境中得到应用，因此本文选取这两个预取技术和一个无预取机制作为对比项。

3.4.3 实验结果与分析

3.4.3.1 预测算法准确性分析

本小节旨在比较 PLGDP 技术与其他数据预取技术在预测准确率（Prediction Accuracy，下文简称为 PA）指标上的提升效果。预测准确率指的是预取技术预测的结果在未来访问的比率（需要注意的是，未来无论访问间隔多长，只要被访问就算命中），这能够有效反映预测算法的预测能力。在实验过程中，FEMU 端用户通过按照工作负载记录的时间间隔向 SLC-TLC 混合 SSD 发送 NVMe 的 I/O 访问请求，混合 SSD 会记录所有预取的数据页面，在后面的读访问请求中，只要属于预取的数据则算一次命中。预测算法的准确率越高，则说明此预取技术预测出来数据在未来会被大概率访问。但是，此指标无法直接指明预取技术的好坏，因为何时会被访问是一个不确定的值。因此，虽然低准确性的预取技术一定不会提升读性能，但高准确性的预取技术也并不保证能在混合 SSD 中明显提升读性能。

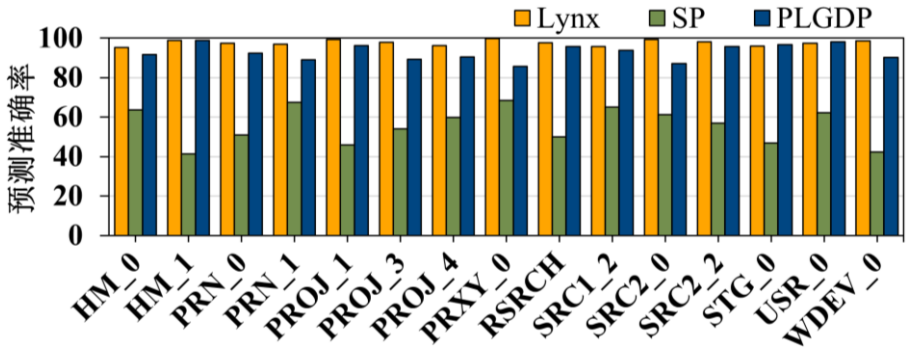


图 3.13 不同预取技术在预测准确率的对比

图 3.13 展示了不同预取技术在预测准确率的对比状况。实验结果表明相比

中国知网 https://www.cnki.net

于顺序预取技术这种主流的预取技术，Lynx 和 PLGDP 在预测准确率上具有明显的优势（平均准确率高于 90%）。由表 3.1 中所指明的负载特征（临近访问比率较低），SP 的表现效果不佳是必然结果（准确率均低于 70%，更有甚者仅仅达到 41.2%，如图 3.13 中的 HM_1），因为顺序预取预测的数据很有可能超出了工作负载地址空间范围，这些范围内的数据根本就不会被访问。然而，Lynx 和 PLGDP 均采用基于历史访问的方式，对当前访问请求进行预测，很难超出以往从来没有出现过的地址空间，所以可以达到很高的预测准确率。但是，相较于 Lynx，PLGDP 的预测准确率还是低一些，主要原因是 PLGDP 引入了地址段时间窗口内扩张访问预测，这一预测有可能也超出了工作地址访问空间范围。

因此总体来看，Lynx 和 PLGDP 具有高预测准确率，作为预取技术加入到 SLC-TLC 混合 SSD 中具有提升读性能显著的可能。然而，SP 不太能够满足对于高准确的需求。

3.4.3.2 RAM 区域读缓存命中率分析

本小节主要分析不同预取技术应用到混合 SSD 中的 RAM 区域缓存命中率（RAM Hit Rate，下文简称为 RHR），具体实验是比较分析了各个对比项在针对各个工作负载下，执行过程中 RAM 空间的读访问请求页面命中数占总读访问请求页面数的比例。

在实验过程中，用户依据工作负载记录向虚拟混合 SSD 发送 I/O 请求，在此期间，跟踪其读访问请求中页面数以及已经保存在 RAM 区域的页面数，最后分别计算命中页面总数以及访问页面总数并相除。图 3.14 展示了不同预取技术下 RAM 区域读缓存命中率对比情况。实验结果表明，相比于无预取机制、基于马尔科夫矩阵的 Lynx 预取技术和顺序预取技术，PLGDP 在提高 RAM 区域读缓存命中率具有明显效果。相较于其他三种方法，PLGDP 技术可以使 RAM 区域读缓存命中率分别平均提升了 335.92%、56.01%、68.81%。虽然，Lynx 在预测算法准确性实验中表现优秀，但落实到实际的混合 SSD 中性能却不如 PLGDP，主要原因是 Lynx 预测出来的数据访问间隔过长，导致从 RAM 区域中被提前剔除，进而应对后续的读访问请求，混合 SSD 无法在 RAM 区域中找到。

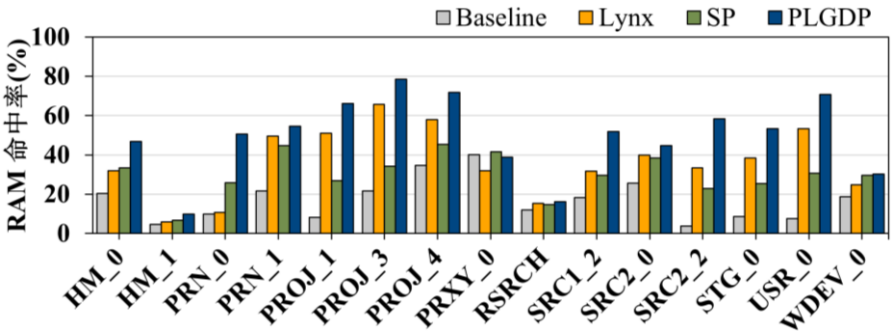


图 3.14 不同预取技术在 RAM 区域读缓存命中率的对比

中国知网 https://www.cnki.net

3.4.3.3 读放大情况分析

本小节对预取技术的读放大情况进行了分析，读放大指的是由预取操作造成的从 SLC 区域和 TLC 区域读取页面数量增加。为了论证本文提出的 PLGDP 技术的准确性，对其读放大情况进行分析是很有必要的，这一指标可以有效评估本技术在各个工作负载执行过程中发生无效读取的数量。读放大比例过大，则说明该算法的短期准确性还是有很大的提升空间。

在实验过程中，混合 SSD 在执行发来的读访问请求过程中，监测 SSD 内部所用从闪存读取数据的页数（引发读取的原因包括数据预取、读访问请求、GC）。图 3.15 展示了不同预取技术从闪存（SLC 和 TLC）区域读取页面数量归一化对比。图中 Baseline 被归一化成 1，代表不引入任何预取操作，从闪存区读取的页面面数。相较于 Baseline，Lynx、SP 和 PLGDP 分别平均读放大了 28.33%、72.87%、17.64%。由于 PLGDP 技术在执行预取请求时，会检测其预取请求中页面的命中率，进而在命中率低下时及时停止执行。因此，从图中可以看出 PLGDP 从闪存介质中读取的页面数相较于 Baseline 提升最少，即读放大效果最不明显。相较于 Lynx 和 SP 两个对比项，PLGDP 的读放大比例分别平均减少了 37.73%、75.79%。这说明，采用 PLGDP 预取技术可以更少地引入无用数据读取，减少数据预取请求对主机端请求的影响。

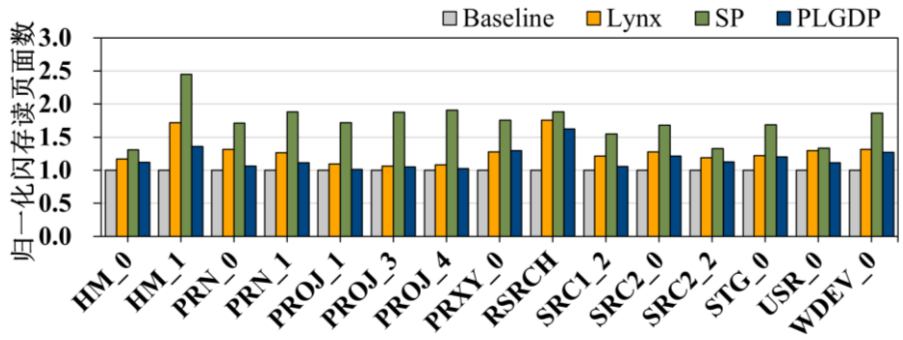


图 3.15 不同预取技术闪存页面读取数量归一化对比

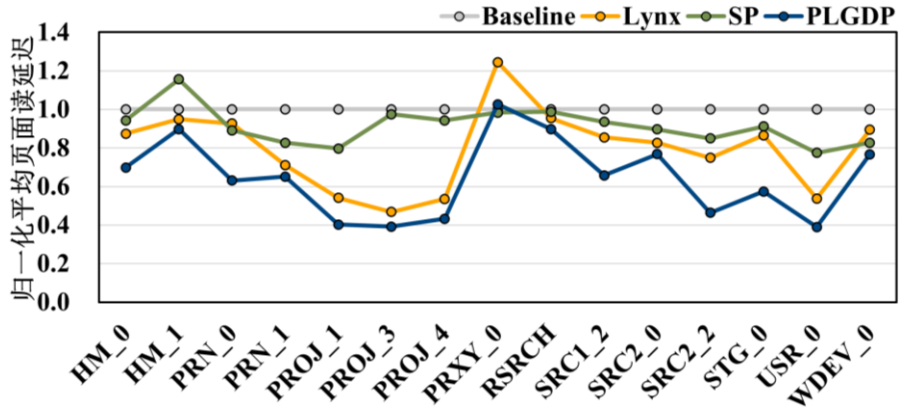
3.4.3.4 混合 SSD 读写性能分析

本小节对引入预取技术的 SLC-TLC 混合 SSD 进行读写访问性能的测试分析，读访问性能指的是 FEMU 端发送给混合 SSD 后读访问请求的执行时延，写访问性能指的是 FEMU 端发送给混合 SSD 后写访问请求的执行时延。这两个性能指标是最能反应出用户的使用体验，因为从用户角度出来，能体会到的只有读写时间。

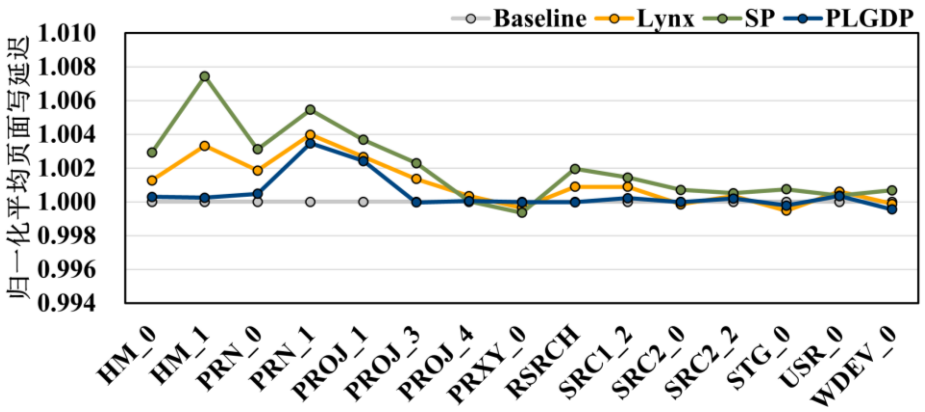
在实验过程中，混合 SSD 记录每一访问请求的起始时间、结束时间以及访问页面数，并计算该请求的页面平均读写延迟。通过以 Baseline 作为基准进行归一化计算，如图 3.16 展示了实验得到的各个负载工作下采用不同预取技术的混合 SSD 页面平均读写访问延迟。相较于 Baseline 页面平均读延迟，Lynx、SP 和

中国知网 https://www.cnki.net

PLGDP 分别平均减少读延迟 20.46%、8.68%、5.69%。与此同时，PLGDP 作为具有高 RAM 区域读缓存命中率预取技术，相较于 Baseline，Lynx 和 SP 对比项，具有明显的页面读取速度优势，平均页面读延迟分别减少 35.69%、19.14%、29.58%，如图 3.16（a）所示。这说明，PLGDP 技术可以有效提高 SSD 的读性能。



(a) 页面平均读延迟



(b) 页面平均写延迟

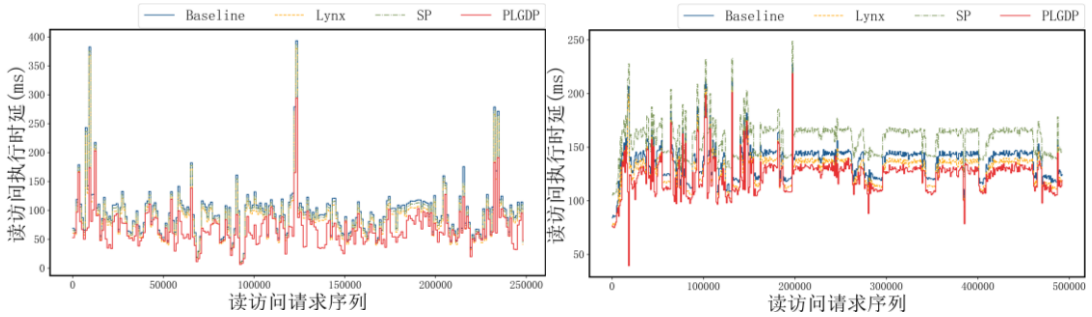
图 3.16 不同预取技术的混合 SSD 页面平均读写访问延迟归一化对比

如实验结果图 3.16（b）所示，由于预取操作的引入，会存在两种情况，一是预取请求阻塞写访问请求；二是预取数据可以快速响应读请求，避免了在为引用预取操作时读访问请求阻塞写请求，如图中工作负载 PRXY_0。相较于 Baseline 页面平均写入延迟，Lynx，SP 以及 PLGDP 分别平均增加了 0.1095%、0.2048%、0.0465%。PLGDP 通过在预取过程中对预取访问操作和主机端 I/O 请求之间进行有效调度，有效减轻了读写冲突（RWC）的压力。相较于 Lynx 和 SP 两个预取技术，PLGDP 实现对写操作更小的影响，平均较少页面写延迟增加量 57.53%、77.29%。

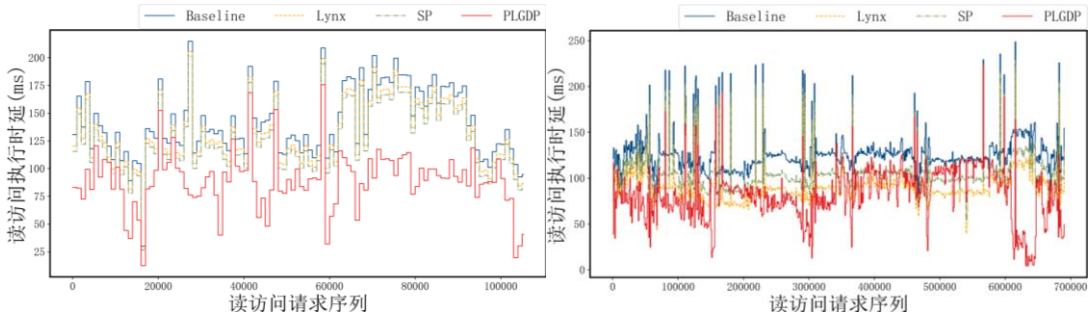
为了清晰展示出 PLGDP 在读访问中所带来的优秀性能提升，图 3.17 展示了 15 条工作负载在执行过程中实时时间段内的读访问请求执行时延。

中国知网 https://www.cnki.net

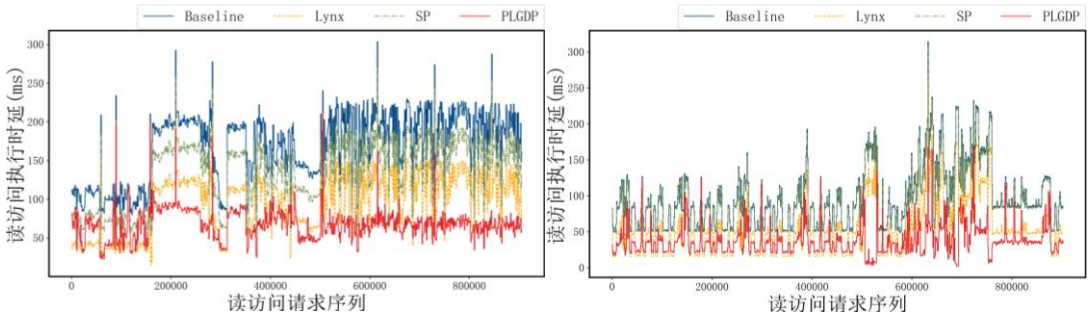
在图 3.17 各个子图中，横坐标代表请求序列号，纵坐标代表以毫米（ms）为单位的请求执行时延，不同颜色线代表采用不同预取技术的 SLC-TLC 混合 SSD 在工作负载下的实时读访问请求执行时延，其中，红色实线代表 PLGDP，绿色点横线代表 SP，橙色虚线代表 Lynx，蓝色实线代表 Baseline。从图中可以观察到，采用 PLGDP 技术的 SLC-TLC 混合 SSD 的读访问请求执行时延总体上低于其他三个对比项。其中，效果最明显体现在 PROJ_1、PROJ_3、PROJ_4、SRC2_2 以及 USR_0 工作负载中，与图 3.16（a）中的平均延迟减少情况基本符合。



(a) HM_0　　(b) HM_1

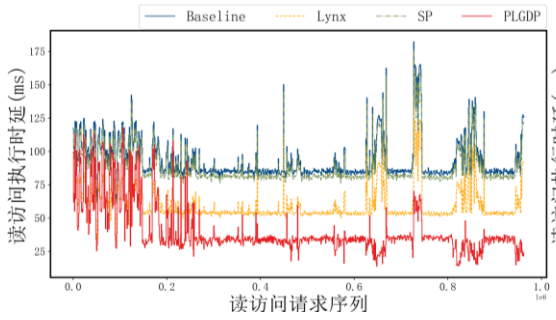


(c) PRN_0　　(d) PRN_1

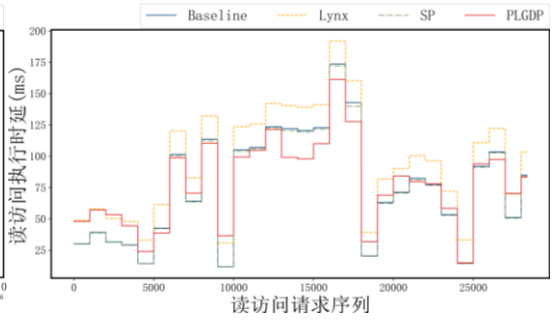


(e) PROJ_1　　(f) PROJ_3

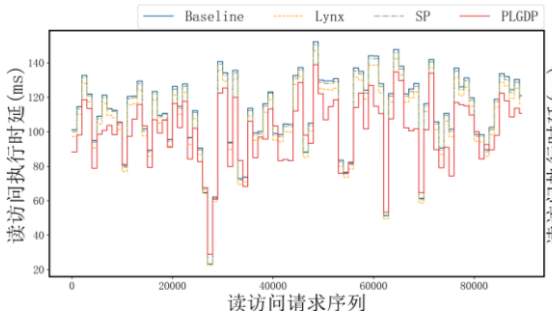
中国知网　https://www.cnki.net



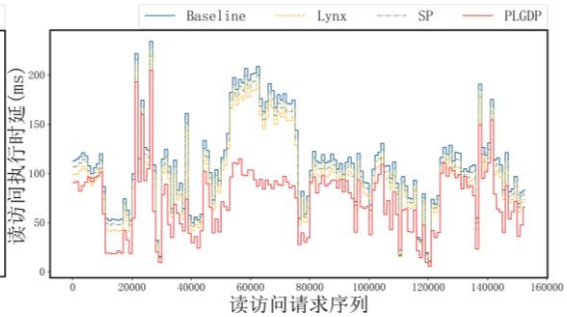
(g) PROJ_4



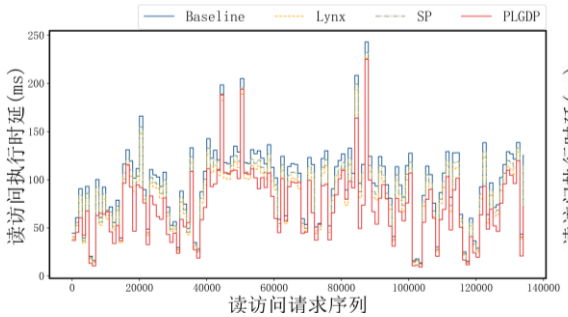
(h) PRXY_0



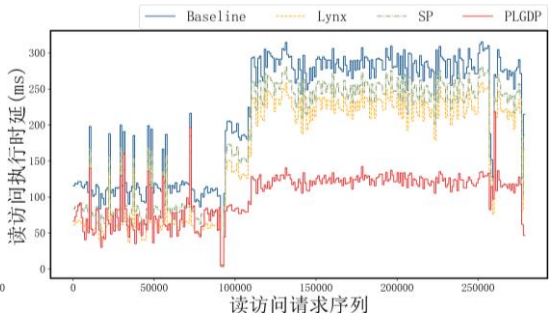
(i) RSRCH



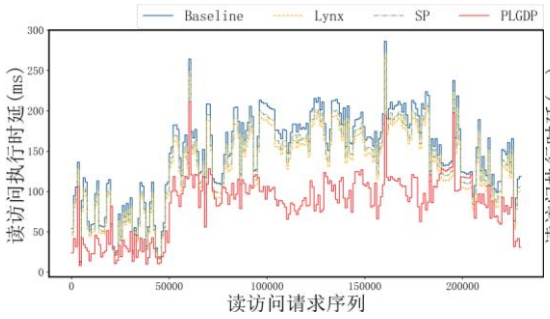
(j) SRC1_2



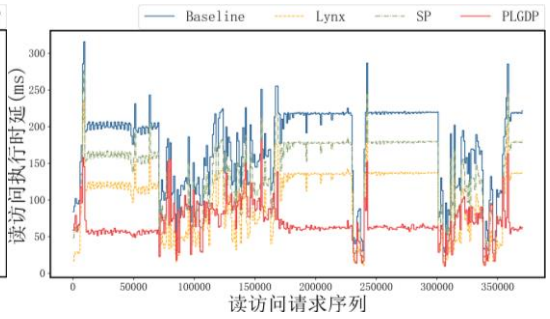
(k) SRC2_0



(l) SRC2_2

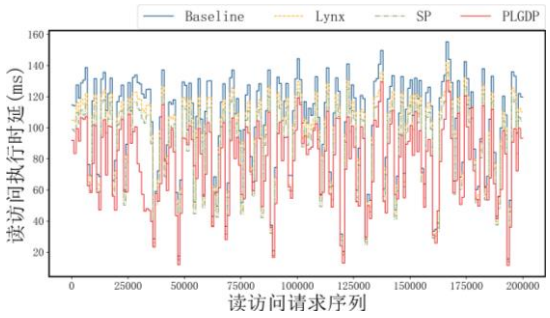


(m) STG_0



(n) USR_0

中国知网 https://www.cnki.net



(o) WDEV_0

图 3.17 不同预取技术的混合 SSD 读访问请求执行期间时延对比

3.5 本章小结

本章主要研究混合 SSD 中的数据预取技术。首先描述了混合 SSD 中的数据预取技术问题，并阐述了其设计过程中存在的难点。继而，本章通过对 15 条开源工作负载块访问记录进行时空局部性特征分析，本研究归纳出关键结论，为数据预取技术的设计提供指导。基于先前的时空局部性特征分析结果，提出了基于模式局部性引导的数据预取（PLGDP）技术，包括基于地址分段的数据预测算法和基于预取请求感知的 I/O 请求调度策略。前者负责未来访问数据预测并生成预取请求，后者负责主机端 I/O 访问请求与预取请求之间的请求调度。为了合理评估 PLGDP 技术的有效性，本章在真实机器上基于 FEMU 搭建平台，来模拟 SLC-TLC 混合 SSD，利用 nvme-cli 工具复现工作负载访问行为，对混合 SSD 进行各项指标的监测与统计。通过与无预取机制（Baseline）、基于马尔科夫状态转移矩阵预取（Lynx）以及顺序预取（SP）的对比表明，PLGDP 的平均混合 SSD 读性能提升了 35.69%、19.14%、29.58%，RAM 读缓存命中率分别提升了 335.92%、56.01%、68.81%。与此同时，在引入数据预取技术后，PLGDP 也是对写性能影响最小的，相较于 Lynx 和 SP，减少了 57.53%、77.29%的页面写延迟增加量。

中国知网 https://www.cnki.net

第四章 基于 RAM 和 SLC 的混合读缓存管理技术研究

本章的主要研究问题是设计一种适配引入数据预取技术的读缓存管理技术。本技术包含两部分：RAM 空间的读缓存管理策略、SLC 空间的读缓存管理策略。针对 RAM 空间同时包含直接读取和预取数据的复合场景，本研究采用了一种基于访问感知的 RAM 读缓存管理策略，通过感知 RAM 内读缓存数据的访问操作来进行相应的数据置换操作，优先保留未来高访问概率的数据，以提升混合 SSD 的读取性能。鉴于有限的 RAM 空间要为写访问操作提供大部分的空间，其读缓存空间受限。同时，考虑到混合 SSD 中 SLC 闪存的页面读性能仍然优于 TLC 闪存中的 CSB 和 MSB 页面，本研究采用了一种基于访问过程的 SLC 读缓存管理策略，通过跟踪 RAM 中高延迟页面的访问动态，优先选择未来可能频繁访问的页面（包括预取页面、读访问页面）缓存在 SLC 中，从而加快数据从闪存芯片阵列至 RAM 的传输速度，快速应对未来读操作需求。

4.1 读缓存管理技术问题

在混合 SSD 架构中，缓存管理策略用于管理各个介质区域内数据的放置问题，通过控制优化数据流，提高性能，延长设备寿命，并确保数据的安全性和一致性。

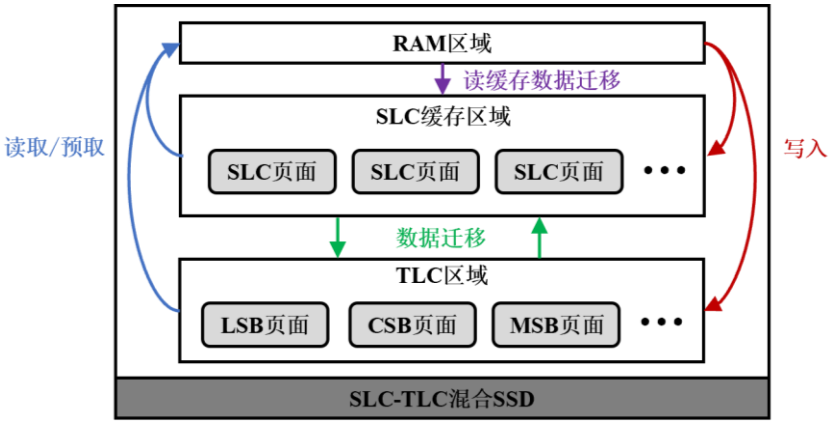


图 4.1 混合 SSD 缓存管理技术概览

图 4.1 为混合 SSD 的缓存管理策略。传统的混合 SSD 缓存管理策略仅仅考虑到写性能，以 SLC-TLC 混合 SSD 为例，通过让 RAM 区域作为一级写缓存区，SLC 缓存区域作为二级写缓存，来提高混合 SSD 的写性能（如图 4.1 中蓝色数据流、红色数据流和从 SLC 到 TLC 的绿色数据流）。然而，针对读性能优化的措施少之又少，现有研究大部分是通过感知数据访问冷热属性将热数据从 TLC 区域迁移到 SLC 缓存区域中，以快速响应读请求（如图 4.1 中从 TLC 到 SLC 的

中国知网 https://www.cnki.net

绿色数据流）。SLC 缓存区域通常充当两个角色，即写缓存区和读缓存区。

本节讨论针对于引入数据预取技术的混合 SSD 读缓存管理问题， 由于引入了数据预取操作，RAM 区域内存在读写缓存数据以及预取数据，RAM 区域的管理变得更加复杂。复杂点在于数据剔除策略，由于部分数据访问模式的改变，导致数据被提前剔除的浪费，尽管这些数据在未来很有可能会被访问。与此同时，作为二级缓存，SLC 缓存区域针对于读优化只是从 TLC 区域中迁移出热数据，但是 RAM 中剔除的数据仍然具有缓存价值，如何利用这些从 RAM 中剔除的数据成为了一个值得去研究的问题（如图 4.1 中紫色数据流）。

总而言之，混合 SSD 的读缓存管理在设计过程中，需要考虑很多因素以指导缓存管理策略，包括：

1）混合 SSD 中 RAM 区域内缓存数据的多样性。在 RAM 区域中，因为读操作引入的缓存数据包括两类，一是读缓存数据，二是预取数据。读缓存数据是由读取访问请求引起的，当混合 SSD 接收到读取访问请求时，其需要先将请求涉及的数据以页（page）为单位先读取到 RAM 区域中。当读访问请求涉及的数据在 RAM 中都齐全时，一并返回给主机端。预取数据是由预取请求操作引起的，当混合 SSD 接收到数据预取请求时，其需要将预取请求涉及的数据以页（page）为单位读取到 RAM 区域中，以应对未来读访问请求，需要注意的是，这些数据不会立刻被访问。由于这两类缓存数据的访问特性，即前者准备就绪就会被立刻访问、后者准备就绪不一定会立刻被访问，因此在设计 RAM 区域读缓存策略是需要考虑这一特性进行设计。

表 4.1 SLC/TLC 延迟概况

介质类型	页面类型	读延迟（Page）	写延迟（Page）	刷新延迟 (Block)	通道传输延迟 (Page)
SLC	SLC	40000ns	800000ns	2000000ns	20000ns
TLC	LSB	56500ns	820500ns	3000000ns	52433ns
	CSB	77500ns	2225000ns		
	MSB	106000ns	5734000ns		

2）混合 SSD 中异构闪存介质页面的性能差异。在混合 SSD 中存在多种页面，这些页面的读写延迟差异较大，例如 SLC-TLC 混合 SSD 中，存在四种页面，即 SLC 页面、LSB 页面、CSB 页面以及 MSB 页面。现有研究给出了 SLC 和 TLC 介质页面的读写延迟测试，如表 4.1 所示。SLC 页面读写性能全方面优于 TLC 中的三种页面，其中 CSB 和 MSB 相较于 SLC 页面差距较大。这种性能差异性会导致在混合 SSD 中，SLC 缓存区域的读缓存策略需要考虑缓存数据的源数据所在页面类型，即当源数据保存在的页面性能相较于缓存区域页面性能很差时，才会需要进行缓存操作。

中国知网 https://www.cnki.net

以 PROJ_4 工作负载为例，在混合 SSD 上运行时，页面读延迟存在着严重的波动现象，如图 4.2 所示。从图中可以看出，页面访问延迟的差异性很大，且存在波动现象，由于存在 RAM 设备，所以排除掉最小延迟，最大延迟与次小延迟之间的差异可以达到惊人的 2.5 倍，这符合 SLC 页面与 MSB 页面间的读延迟差异。这一现象表明，当大量读请求的页面集中在 TLC 区域时，会对混合 SSD 的整体性能产生负面影响。

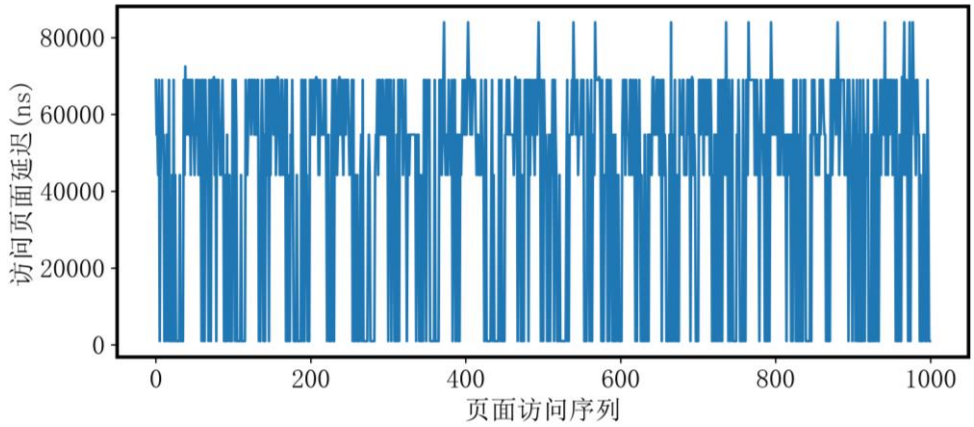


图 4.2 混合 SSD 中页面读访问延迟波动现象（PROJ_4）

总之，混合 SSD 的读缓存管理技术的设计面临着由于预取操作带来的 RAM 区域内缓存数据多样性以及 SLC 与 TLC 页面性能差异性的两重挑战。为了有效应对这些挑战，读缓存管理技术需要针对两种区域（RAM 区域和 SLC 缓存区域）分别设计缓存策略，并需要考虑二者之间的数据流交换。与此同时，SLC 的读缓存策略的设计还需要兼顾写入性能，即尽量减少读缓存策略对混合 SSD 整体写入性能的影响。

4.2 混合 SSD 读访问数据位置分析

为了深入理解缓存策略的重要性，本节基于传统混合 SSD 缓存策略，对使用第三章预取技术前后的读访问数据位置分布进行分析。分析角度包含两方面：读数据来源分布情况和 RAM 空间预取数据利用情况。通过观察分布情况，得出相应的 RAM 区域和 SLC 缓存区域的读缓存策略设计动机，进而指导后续的缓存策略设计。

4.2.1 读数据来源分布情况

读数据来源分布指的是在一个工作负载运行过程中，其读访问请求涉及的数据来源的比例Pf，其数学定义如式（4.1）所示，其中$Area$表示数据来源位置，$count_{Area}$表示来自于$Area$的读数据页面数，$count_{All}$表示读访问涉及的页面总数。数据是源自于 RAM 区域（RAM 区域中已经缓存了读访问请求的数据），还是源自于 SLC 缓存区域（读访问请求的数据在 SLC 缓存区域中保存备份或者原始数

中国知网 https://www.cnki.net

据就在 SLC 缓存区域中），还是源自于 TLC 区域（读访问请求的数据保存在 TLC 区域）。这一指标在评价混合 SSD 读缓存策略好坏具有指导意义，因为读缓存策略的主要目的就是尽可能将未来大概率访问的数据缓存在高性能介质中，以提供读访问请求的快速响应。

$$Pf_{Area} = \frac{count_{Area}}{count_{All}} \tag{4.1}$$

首先，本小节对未采用预取技术的 SLC-TLC 混合 SSD 进行 15 条工作负载的读数据来源分布统计，如图 4.3 所示。从图中可以看出，在没有采用预取技术来预取数据到 RAM 区域的情况下，分布在 RAM 的读数据比例（如图 4.3 中蓝色部分）非常少，在 SRC2_2 工作负载中达到最小比例 3.8%，在 PRXY_0 工作负载中达到最大比例 40.2%。平均下来，数据的 RAM 来源占比仅仅只有 17.068%，这么低的比例具有很大的提升空间。与此同时，SLC 缓存区域的比例（如图 4.3 黄色部分）也不是很高，更有甚者比 RAM 区域的比例还要低，这主要的原因是 SLC 缓存区域主要作为写缓存区，会不断地服务于写访问请求，并将涉及的数据写入到 SLC 缓存区域，进而导致当 SLC 缓存区域空闲空间不足时，SLC 缓存区域的数据需要通过后台的数据迁移操作（如图 4.1 中绿色的箭头）从 SLC 缓存区域迁移到 TLC 区域中。因此，从图 4.3 实验结果来看，超过 60%的读访问数据来自于 TLC 区域，这也是 SLC-TLC 混合 SSD 读性能不佳的主要原因。

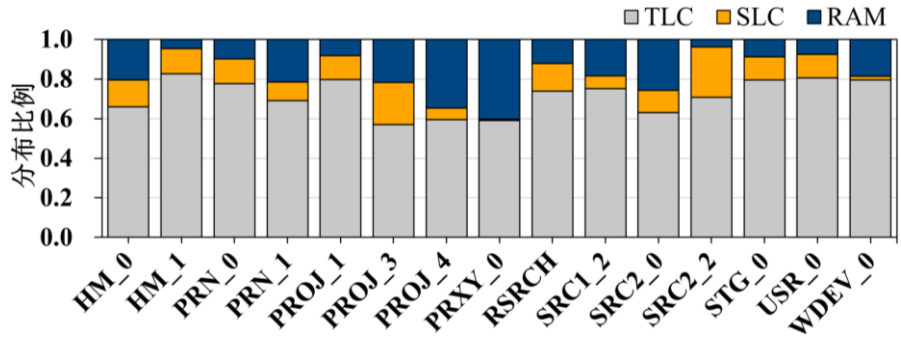


图 4.3 未采用预取技术的 SLC-TLC 混合 SSD 读数据来源分布

其次，本小节对采用第三章 PLGDP 预取技术的 SLC-TLC 混合 SSD 进行了 15 条工作负载的读数据来源分布统计，如图 4.4 所示。从图中可以看出，在采用了第三章采用的预取技术来预取数据到 RAM 区域情况下，分布在 RAM 区域的读数据比例（如图 4.4 中蓝色部分）明显增加，在 PROJ_3 工作负载中达到了最大比例 78.53%，在 HM_1 工作负载中达到最小比例 9.96%。平均下来，数据的 RAM 来源占比可以达到 49.531%，这也是导致在第三章中混合 SSD 读性能提升的主要原因。然而，SLC 缓存区域的比例（如图 4.4 黄色部分）不仅没有增加，反而还减少了。主要原因在于原先可能从 SLC 读取的数据被预先读取出并缓存在了 RAM 中，因此，SLC 缓存区域的比例下降。然而 SLC 缓存区域的空间仅

中国知网 https://www.cnki.net

用于写缓存区比较浪费，其低下的读访问比例暗示了其具有很大的优化空间。而TLC 的读数据来源分布比例平均下来仍然超过 44.5%。

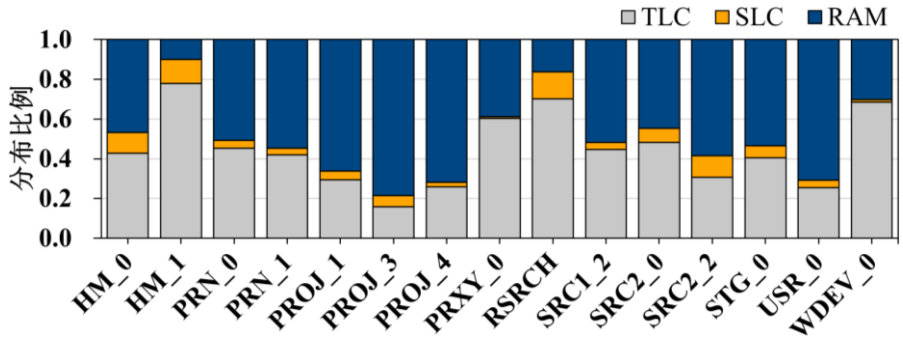


图 4.4 采用 PLGDP 预取技术的 SLC-TLC 混合 SSD 读数据来源分布

总的来说，通过统计引入预取技术前后混合 SSD 读访问数据来源分布情况，得出如下结论：**1）传统的混合 SSD 缓存管理技术没有充分利用 RAM 空间以及 SLC 缓存空间。**首先，RAM 仅仅采用 LRU 这种简单的缓存策略无法很大地提高 RAM 空间的使用率。其次，SLC 缓存空间虽然可以存在一定的读缓存作用，但是其可以提供的读缓存比例太低了，对于读性能提升意义不大。**2）引入第三章 PLGDP 预取技术的混合 SSD 虽然可以提升 RAM 空间利用率，但是 SLC 缓存空间的读缓存功能被幅度削减。**首先，利用预取技术可以在一定程度提高 RAM 空间利用率，然而 SLC 缓存空间在引入预取技术前后其读访问请求数据占比都没有很好的提高，反而有些还下降。这表明，预取技术的引入影响到了 SLC 缓存空间的使用，需要一种更好的 SLC 缓存空间读缓存管理策略来提升 SLC 缓存空间的读访问命中率。

4.2.2 RAM 空间预取数据利用情况

RAM 空间预取数据利用情况指的是，预取到 RAM 区域内的数据在从 RAM 区域被剔除之前被读访问的比例Pa，其数学定义如式（4.2）所示，其中，$count_{access}$表示 RAM 区域中被预取进来的数据页被主机访问的计数，$count_{prefetch}$表示预取到 RAM 区域中数据页面的总数。这一指标主要有两个作用，其一是评价 RAM 区域缓存管理的好坏，其二是评估从 RAM 区域剔除数据的二次利用价值。

$$Pa = \frac{count_{access}}{count_{prefetch}} \tag{4.2}$$

本小节复现了 15 个工作负载的访问行为，在固定 8MB 大小的 RAM 区域进行 RAM 空间 PLGDP 预取数据利用情况的统计，实验结果如图 4.5 所示，此结果于第三章实验 3.4.3.1 中预测算法准确性分析结果不符合。在预测算法准确性分析实验中，平均下来在各个工作负载的预测算法准确性可以达到 90%以上，这说明，预取到 RAM 区域的数据利用率应该高达 90%以上。但是在图 4.5 中所呈

中国知网 https://www.cnki.net

现的平均预取数据利用率仅有 68.37%，即只有在部分工作负载中能够符合预期，然而例如 HM_1、PRXY_0、RSRCH、SRC2_0、WDEV_0 等工作负载的预取数据利用率低下，甚至连预期利用率的一半都没有达到。为此得出的结论是 **3）在部分工作负载中，预取的数据被提前地从 RAM 区域中剔除（由于有限的 RAM 区域空间），导致预取的数据没有被访问就被浪费掉了。**

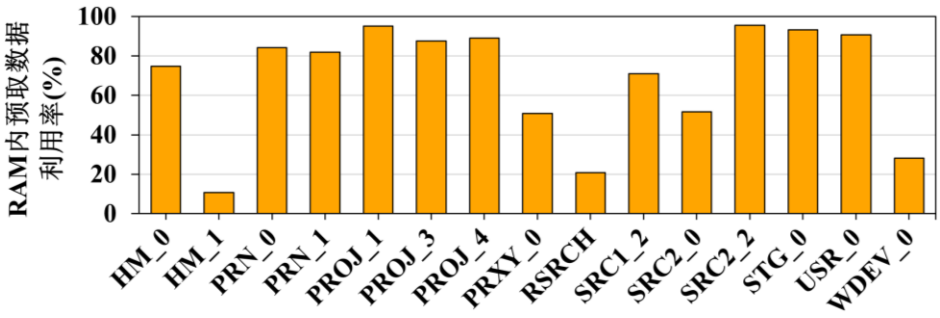


图 4.5 RAM 空间预取数据利用情况

结论指出，现有单纯的采用 LRU 策略来管理 RAM 区域是不明智的，由于缺乏对于数据类型的感知以及近期的访问情况，无法做出明智的数据剔除策略。此外，预取数据的利用率不仅仅取决于预取技术的准确性，RAM 区域的空间大小也是重要的决定因素。RAM 区域空间越大，可以容纳的预取数据就越多，这代表数据可以更久的驻留在 RAM 区域中以应对未来的读访问请求。然而，RAM 设备的单位存储价格昂贵，扩大 RAM 设备容量是不切实际的。由于 SLC 闪存页面的读性能是高于 TLC 闪存中的 CSB 页面和 MSB 页面，SLC 闪存作为读缓存区来缓存从 RAM 区域剔除的高延迟页面是一个解决预取页面利用率低下的有效方法。

4.3 基于 RAM 和 SLC 的混合读缓存管理技术

基于以上对读数据来源分布情况和 RAM 空间预取数据利用情况的分析结果，本文提出了基于 RAM 和 SLC 的混合读缓存管理（Hybrid Read Cache Management based on RAM and SLC，HRCMRS）技术，在 RAM 区域和 SLC 缓存区域分别设计相应的读缓存管理策略。通过页面过滤机制，让 SLC 缓存区域缓存从 RAM 区域剔除的页面数据，提高预取数据的利用率，减少 SLC-TLC 混合 SSD 的读访问延迟，提高整体读性能。设计的关键是如何设计高效的 RAM 区域读缓存管理策略，以最大可能保留未来可能访问的数据；如何设计 SLC 缓存区域读缓存管理策略，以最小的代价去接收缓存从 RAM 区域中剔除来的有价值的数据。RAM 区域读缓存管理策略严重影响了数据预取效果，即 RAM 区域读数据命中率。越优秀的 RAM 区域读缓存管理策略可以尽可能让预取数据在 RAM 区域中得到访问，进而提高混合 SSD 的都访问性能。由于预取数据未来访问时

中国知网 https://www.cnki.net

间的不确定性，在有限的 RAM 空间内很难满足让预取数据全部在 RAM 区域内命中。因此，SLC 缓存区域作为二级读缓存具有弥补读性能损失的重要作用，可以通过在 SLC 缓存区域划分一小部分空间来容纳从 RAM 区域剔除的高延迟页面。SLC 区域的读性能依然高过 TLC 区域的 CSB 页面和 MSB 页面。针对以上两点设计关键，HRCMRS 的框架包括两个组件：基于访问感知的 RAM 读缓存管理策略模块（组件一）、基于访问过程的 SLC 读缓存管理策略模块（组件二）。组件一根据感知数据类型以及访问来满足尽可能保留未来大概率会被访问数据的需求，旨在提高读数据来源于 RAM 区域的比例；组件二根据感知从 RAM 剔除页面在 RAM 区域内的访问过程来满足尽可能缓存未来大概率会被访问数据的需求，旨在提高预取数据的利用率，进而提高读数据来源于 SLC 缓存区域的比例。

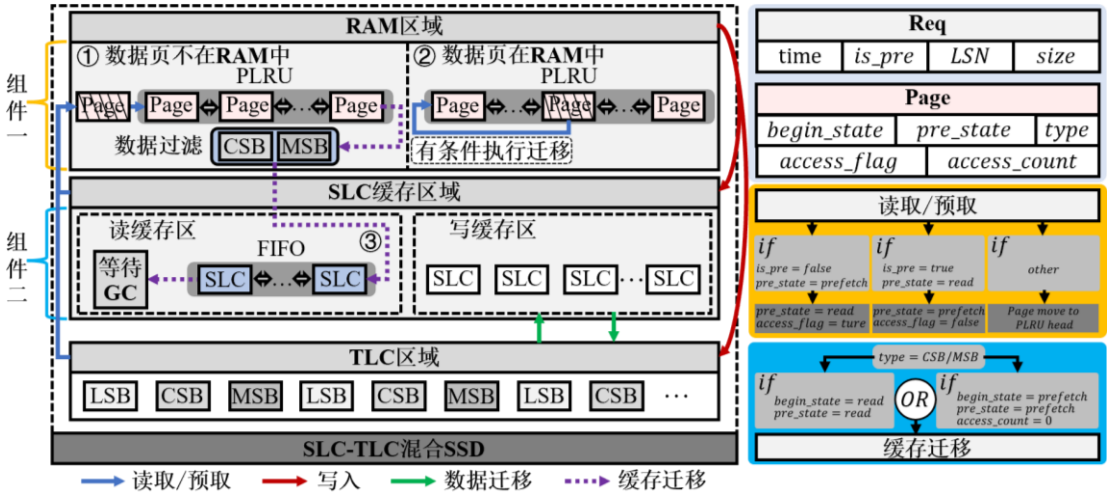


图 4.6 HRCMRS 框架图

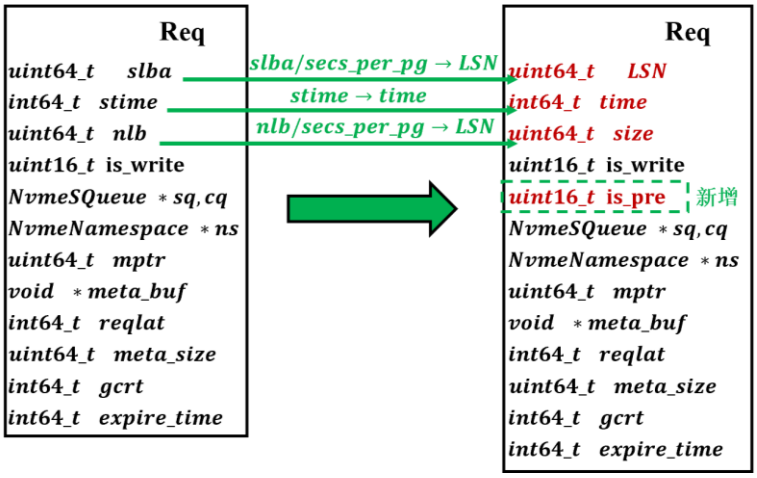
如图 4.6 展示了基于 RAM 和 SLC 的混合读缓存管理技术的框架图，其中蓝色箭头表示读取/预取数据流、红色箭头表示写入数据流、绿色箭头表示数据迁移数据流、紫色虚线箭头表示组件一剔除的数据页面部分缓存迁移到组件二中。组件一中采用基于最近最少使用（Least Recently Used，LRU）修改后的 PLRU 队列进行管理，通过感知队列中页面的访问来进行相应的迁移剔除操作。组件二中，通过将 SLC 缓存区域从物理层划分成两个独立的缓存区（读缓存区和写缓存区），在写缓存区中依然采用传统的写缓存策略不变，而在读缓存中，采用先进先出（First Input First Output，FIFO）方式来管理从 RAM 区域缓存迁移出来的高延迟页面，并通过感知页面在 RAM 区域中的访问过程来决策是否允许缓存。为了实现组件一和组件二对于访问和访问过程的感知，本文修改了两个点，一个是请求格式，如图 4.6 中右边蓝灰色部分中的 Req 部分；二是数据页面在 RAM 区域中的管理属性，如图 4.6 中右边蓝灰色部分中的 Page 部分。图 4.6 中右边黄色部分即为 RAM 区域读缓存管理策略的缩略版本，通过当前 Req 和 Page 的属性进

中国知网 https://www.cnki.net

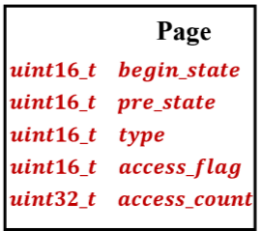
行相应的管理策略执行。图 4.6 中右边蓝色部分即为 SLC 缓存区域读缓存区对于 RAM 区域剔除页面缓存管理策略的缩略版本，通过感知页面在 Page 的各项属性来确定页面的访问过程，进而评估是否有缓存在 SLC 缓存区域的价值，指导二级缓存操作。

4.3.1 访问请求以及页面属性

在介绍读缓存管理策略之前，需要先了解本策略的基础，即访问请求和页面属性的修改和添加。这些修改是读缓存管理策略执行的依据，组件一和组件二需要通过这些属性信息来判断页面是如何进入到 RAM 区域中的（预取还是读取）、在 RAM 内是否被访问、访问的频不频繁等信息，有助于对不同类型的数据进行针对性的管理。



(a) 针对访问请求 Req 信息的修改



(b) 针对 RAM 区域数据页管理信息的设置

图 4.7 访问请求与页面属性的修改及设置

如图 4.7 展示了访问请求与页面属性的修改及设置。其中图 4.7（a）展示了修改前后 Req 访问指令中都包含了哪些变量信息以及修改过程；图 4.7（b）展示了 RAM 区域页面管理信息设置。

首先，访问请求原始变量信息包含 13 个信息，信息的具体含义如下：

slba，一个无符号 64 位的整数，其值是访问请求的起始逻辑块地址（Starting Logical Block Address），表示数据传输的起始位置，其以扇区为单位，即 512B 大小。

中国知网 https://www.cnki.net

$stime$，一个有符号 64 位的整数，其值是以纳秒为单位的时间数值，表示访问请求的开始时间。

nlb，一个无符号 64 位的整数，其值是以扇区（512B）为单位的大小数值，表示访问请求涉及数据扇区的大小。

is_write，一个无符号 16 位的整数，其值是标记位，表示该访问请求是读访问请求（其值为 1）还是写访问请求（其值为 0）。

sq，是一个指针变量，指向的是其 Req 所处于的请求提交队列（Submission Queue, SQ），用于 NVMe 指令的提交。

cq，是一个指针变量，指向的是其 Req 所处于的请求完成队列（Completion Queue, CQ），用于 NVMe 指令的提交。

ns，是一个指针变量，指向命令操作的命名空间的指针。NVMe 支持多个命名空间，允许存储设备被分割成独立的部分。

$mptr$，一个无符号 64 位的整数，其值是元数据的地址位置，指向存储元数据的内存位置。

$meta_buf$，是一个无类型的指针变量（默认 64 位），指向元数据缓存区的地址。

$reqlat$，一个有符号 64 位的整数，其值是该访问请求的执行延迟。

$meta_size$，一个无符号 64 位的整数，其值是数据源数据的大小。

$gcrt$，一个有符号 64 位的整数，其值是垃圾收集（Garbage Collection，GC）的相关时间。

$expire_time$，一个有符号 64 位的整数，其值是访问请求过期时间。

由于，本文修改的目的是让混合 SSD 可以感知到请求的类型，并且与混合 SSD 以页为单位进行读写对齐，所以只对 3 个 Req 内变量进行修改，并通过添加 1 个变量，便于混合 SSD 感知，如图 4.7（a）所示。首先，针对于$slba$变量，为了让其可以与混合 SSD 对齐，本文将$slba$除以每个页中包含的扇区数（$secs_pre_pg$），得到无符号 64 位的LSN（起始逻辑页号）。与此同时，nlb也进行了相同的转换，得到无符号 64 位的$size$。为了方便理解，本文将原有的$stime$更名为$time$。最后，为了让混合 SSD 感知到访问请求是主机端请求还是预取请求，本文在 Req 中加入了无符号 16 位的is_pre，来标记访问请求是否是预取请求。

针对于 RAM 区域页面读缓存管理，本文设置了全新的 Page 管理信息，包括 5 个信息变量，具体信息含义如下：

$begin_state$，一个无符号 16 位的整数，其值是一个标记，指明该页面最开始读入到 RAM 区域是由预取请求引起的还是由读访问请求引起的。如果一个页

中国知网 https://www.cnki.net

面，在起初进入到 RAM 区域是由预取请求引起的，则它在 RAM 内被访问一次后，可能后面就不会被访问了。但如果是由读访问请求引起的，则说明预测算法对此页面有所纰漏，有必要缓存下来。

pre_state，一个无符号 16 位的整数，其值是一个标记，指明该页面最近一次访问是由预取请求引起的还是由读访问请求引起的。如果一个页面的最近一次访问是读访问请求，说明其最近可能立刻被访问的可能性不大，但未来被访问的可能性很大。

$type$，一个无符号 16 位的整数，其值是一个标记，指明该页面的原始数据保存在 SLC 页面、LSB 页面、CSB 页面还是 MSB 页面。如果此页面是 CSB 页面或 MSB 页面，则其在从 RAM 区域剔除时，很有缓存到 SLC 的价值，以快速响应未来的读访问请求。

$access_flag$，一个无符号 16 位的整数，其值是一个标记，指明该页面在 RAM 区域期间是否有被读访问过。如果一个预取页面，在读入到 RAM 区域后，从来没有被访问过，则说明此页面要不就是预取错误，要不就是大概率会被立刻访问。

$access_count$，一个无符号 32 位的整数，其值是一个计数，记录该页面在 RAM 区域期间被读访问的次数。此变量反映了 RAM 区域内各页面的冷热属性。

4.3.2 基于访问感知的 RAM 读缓存管理策略

为了应对由于预取操作引入的预取页面提升 RAM 区域页面类型复杂的问题，感知其中页面类型及访问操作是很有必要的。通过感知页面被访问了多与少、近期是否被访问等信息有助于更有效地管理页面的去留，提高 RAM 区域的读访问数据来源占比，进而提高 SLC-TLC 混合 SSD 的读性能。

本文提出的基于访问感知的 RAM 读缓存管理策略是基于页面访问感知和访问类型的，旨在识别当前页面访问特性。该读缓存管理策略（PLRU）在最近最少使用（LRU）策略的基础上添加了对于数据访问感知的功能，来指导 RAM 区域读缓存管理，以达到提升 RAM 区域读访问数据来源比例。如图 4.8 展示了 RAM 读缓存管理策略执行过程。

首先，当读/预取访问请求 Req 到来时，组件一会解析 Req 中所有数据页面的逻辑页号，构建成LPN数组。通过遍历LPN数组查找对应的LPN是否已经缓存在 RAM 区域中，需要注意的是，为了加快查找速度，可以采用平衡二叉排序树来进行组织构建。

如果 RAM 区域中并没有缓存该页面，该页面需要读入到 RAM 区域中进行缓存。在读入页面之前，若 PLRU 管理队列已满，需要将其尾部（tail）页面剔除发送给组件二，由组件二判断是否缓存其页面到 SLC 缓存区域中（如图 4.8 中③

中国知网 https://www.cnki.net

所示）。之后，组件一将页面插入到队列头部，完成一次操作（如图 4.8 中①所示）。

如果 RAM 区域中已经缓存了该页面，则组件一将根据当前Req内部参数以及对应$Page$在 PLRU 中的内部参数信息来进行决策。这里组件一在设计过程中认为，如果当前Req是读取访问请求且该页面最近访问操作是预取访问，则保持$Page$在队列中的位置不变；如果当前Req是预取访问请求且该页面最近访问操作是读取访问，则仍然保持$Page$在队列中的位置不变；其他情况，均采用 LRU 的原则，进行迁移到队列头部的操作。（如图 4.8 中②所示）

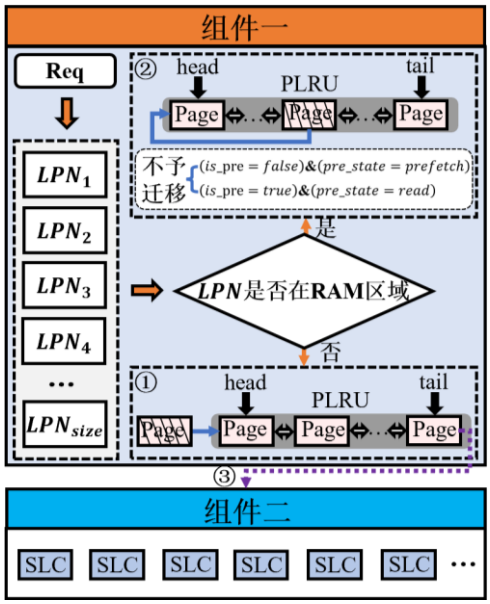


图 4.8 RAM 区域读缓存管理策略执行过程

结合前面所述，基于访问感知的 RAM 读缓存管理策略算法的详细描述如算法 4.1 所示：

算法 4.1 基于访问感知的 RAM 读缓存管理策略算法

输入： Req 访问请求

输出： $Null$ 无输出

```
if Req.is_write is true then /* 如果是写请求则不进行任何操作 */
    output Null
end if
PLRU ← get_page_queue() /* 获取管理 RAM 的 PLRU 页面管理队列 */
Page_LPN_sequence ← get_pages_LPN(Req) /* 获取 Req 请求涉及的所有逻辑页面号 */
for each Page_LPN in Page_LPN_sequence do /* 遍历页面号，进行相应的管理操作 */
    Page ← search(Page_LPN, PLRU)
```


中国知网 https://www.cnki.net

```
8:    if Page is null then /* RAM 区域中无该页面 */
9:        Page ← malloc(sizeof(struct Page)) /* 创建一个空白的Page变量，并初始化 */
10:       Page.begin_state ← Req.is_pre
11:       Page.pre_state ← Req.is_pre
12:       Page.type ← search_type(Page_LPN)
13:       if Req.is_pre is not prefetch then /* 当前是读取访问，则进行相应的Page初始化 */
14:           Req.access_flag ← true
15:           Req.access_count ← 1
16:       else /* 当前是预取访问，则进行相应的Page初始化 */
17:           Req.access_flag ← false
18:           Req.access_count ← 0
19:       end if
20:       if is_full(PLRU) is true then /* 如果 PLRU 已经满了，则剔除尾页面到组件二中 */
21:           tail_Page ← pop_tail(PLRU)
22:           send_Page2SLC(tail_Page)
23:       end if
24:       insert_page2head(Page, PLRU) /* 将 Page 插入到 PLRU 头部 */
25:   else /* RAM 区域中已经缓存了此页面 */
26:       if Req.is_pre is false and Page.pre_state is prefetch then
27:           Page.pre_state ← read
28:           Page.access_flag ← ture
29:           Page.access_count ← Page.access_count + 1
30:       else if Req.is_pre is true and Page.pre_state is read then
31:           Page.pre_state ← prefetch
32:           Page.access_flag ← false
33:       else
34:           move2head(Page, PLRU)
35:           if Req.is_pre is false then
36:               Page.access_count ← Page.access_count + 1
37:           end if
38:       end if
```


中国知网 https://www.cnki.net

39: **end if**

40: **end for**

41: *output Null*

算法 4.1 主要分为两个处理流程：

流程 1：首先，针对于到来的读/预取访问 Req，如果数据页没有缓存在 RAM 区域中，则需要将页面读入到 RAM 区域，并构建相应的$Page$加入到 PLRU 中进行管理。此过程，如果 PLRU 已经满了，则需要将 PLRU 尾部页面剔除到 SLC 读缓存区中，并将$Page$插入到 PLRU 头部。（具体过程为代码 8-24 行）

流程 2：如果数据页已经缓存在了 RAM 区域中，则需要根据当前访问请求 Req 的is_pre以及数据页对应到 PLRU 中$Page$的pre_state来进行决策，并修改相应的$Page$的信息变量。（具体过程为代码 25-39 行）

4.3.3 基于访问过程的 SLC 读缓存管理策略

为了提高 SLC 缓存区域的读缓存利用率以及进一步利用 RAM 区域中的读访问数据和预取数据，SLC 缓存区域的读缓存管理策略至关重要。

考虑到 SLC-TLC 混合 SSD 中 SLC 页面读性能要优于 TLC 闪存介质中的 CSB 页面和 MSB 页面，因此，基于此特性出发并结合感知页面保存在 RAM 区域中的访问过程，本文提出了基于访问过程的 SLC 读缓存管理策略，策略执行流程如图 4.9 所示。

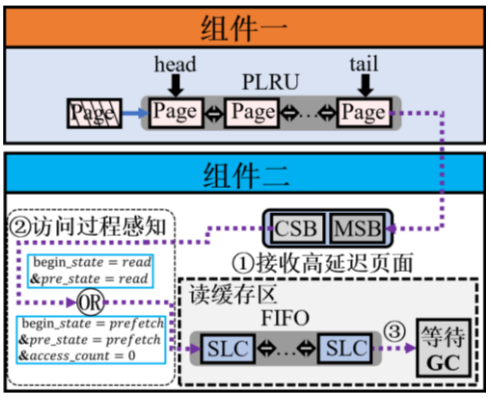


图 4.9 SLC 读缓存管理策略执行过程

SLC 读缓存管理策略主要分成三个组成部分，分别是高延迟页面过滤操作、访问过程感知以及 SLC 读缓存区维护。

（1）高延迟页面过滤操作：通过LPN到物理地址的映射表去查看发送到组件二的$Page$的原始数据是保存在 SLC 页面中还是 TLC 的 LSB、CSB、MSB 页面中。由于 SLC 页面与 LSB 页面读性能相差不多，组件二被设计只接受缓存 CSB 页面和 MSB 页面，其他页面不予以后续操作并直接丢弃。（如图 4.9 中①所示）

中国知网 https://www.cnki.net

（2）访问过程感知：经过高延迟页面过滤操作后的页面都是 CSB 页面或者 MSB 页面，但是考虑到有限的 SLC 读缓存空间，并不是所有的高延迟页面都有缓存到 SLC 缓存区域的必要。组件二通过感知页面在 RAM 区域内的访问过程情况，包括这个页面$Page$刚读入到 RAM 区域时的操作$begin_state$、最近一次访问操作的类型pre_state以及读访问计数$access_count$。组件二认为，如果页面最开始读入到 RAM 区域是由于读访问请求，并且最近一次访问操作也是读访问操作，则它未来很有可能被访问，且是一个比较热的数据，有必要缓存在 SLC 读缓存区中；如果页面最开始读入到 RAM 区域是由于预取请求并且最近一次访问操作也是预取访问操作，则该页面很有可能是被提前剔除预取页面，如果不保留缓存下来，很有可能会浪费其预取操作，因此该页面将会被允许缓存到 SLC 读缓存区中。（如图 4.9 中②所示）

（3）SLC 读缓存区维护：针对于 SLC 读缓存区维护工作，组件二采用最基本的先进先出（First Input Fist Output，FIFO）管理策略。需要注意的是，FIFO 队列长度是远小于读缓存区实际物理大小的，当页面从 FIFO 队列剔除时，该页面处于可垃圾回收状态（Garbage Collection，GC）。当 SLC 读缓存区空间不足时，会执行垃圾回收操作，将无效页面和处于可垃圾回收状态的页面进行垃圾回收，进而释放 SLC 读缓存区空间。

结合前面所述，基于访问过程的 SLC 读缓存管理策略算法的详细描述如算法 4.2 所示。

算法 4.2 基于访问过程的 SLC 读缓存管理策略算法

输入： $Page$　从 RAM 区域剔除的页面

输出： $Null$　无输出

```
if Page.is_type is not CSB/MSB then /* 如果不是高延迟页面则不进行任何操作 */
    output Null
end if
flag ← 0
if Page.begin_state is read and Page.pre_state is read then /* 满足 SLC 缓存条件 */
    flag ← 1
end if
if Page.begin_state is prefetch and Page.pre_state is prefetch
and Page.access_count is 0 then /* 满足 SLC 缓存条件 */
    flag ← 1
```


中国知网 https://www.cnki.net

```
10:  end if
11:  if flag is 1 then /* 标记位开启，执行缓存操作 */
12:    FIFO ← get_slc_page_queue( ) /* 获取管理 SLC 读缓存区的 FIFO 页面管理
       队列 */
13:    if is_full(FIFO) is true then
14:      tail_slc_page ← pop_tail(FIFO)
15:  end if
16:    insert_page2head(Page, FIFO)
17:  else
18:    output Null
19:  end if
```

组件二根据算法 4.2 的执行流程，来进行 RAM 页面的缓存操作。需要注意的是，这里给出的仅仅是组件二如何处理组件一发送过来的页面。当通过主机读取访问或预取请求访问 SLC 读缓存区时，数据页会被检索并重新读回 RAM 区域中。检索到的数据页面不会立刻标记为无效，而是作为有效备份保留在 SLC 读缓存区。然而，当 SLC 读缓存区变满时，在删除回收所有无效页面后，这些数据才会将被丢弃以释放更多空间。其中，SLC 读缓存区中的数据无效仅在其原始数据通过 GC 更新或迁移到 TLC 空间中的 LSB 页面时发生。

4.4 读缓存管理技术分析

本节主要对本章提出的基于 RAM 和 SLC 的混合读缓存管理（HRCMRS）技术进行实验分析与效果评估。首先，介绍实验平台的具体搭建，包括实验平台的具体框架、框架内各个组成部分的功能；随后，介绍了相关的实验环境，包括硬件参数，模拟器参数以及读缓存管理技术参数等；最后对 HRCMRS 技术在几个关键性指标维度上进行分析，具体包括读数据来源分布情况分析、预取数据利用率分析、写放大情况分析以及混合 SSD 读写性能分析。

4.4.1 实验平台搭建

本小节在第三章 Baseline 基础上进行了修改并搭建一个平面级混合的 SLC-TLC 混合 SSD，用来在其上进行 15 条工作负载 I/O 访问行为的复现。实验平台框架如图 4.10 所示，主要修改区域在 SLC-TLC 混合 SSD 端，相较于第三章的实验平台做出了两部分的改动。

首先，在混合 SSD 端内部新添加了一个读缓存管理技术模块，该模块用于管理 RAM 区域和 SLC 缓存区域的数据。通过该模块，可以控制各个区域数据间的迁移和替换。需要注意的是，实际的数据迁移等操作还是由虚拟硬件模块来完

中国知网 https://www.cnki.net

成。其次，在虚拟硬件模块中，由于 HRCMRS 技术的引入，在 SLC 缓存区域中进行了物理意义上的划分，将其划分成了两个物理区域，即读缓存区和写缓存区。前者作为接收 RAM 区域剔除页面的最终归属，通过其对应的 SLC 读缓存管理策略，对到来页面进行判别与筛选，最终将筛选通过的页面缓存在读缓存区中；后者作为写访问操作的缓存区，能够快速响应写访问请求。

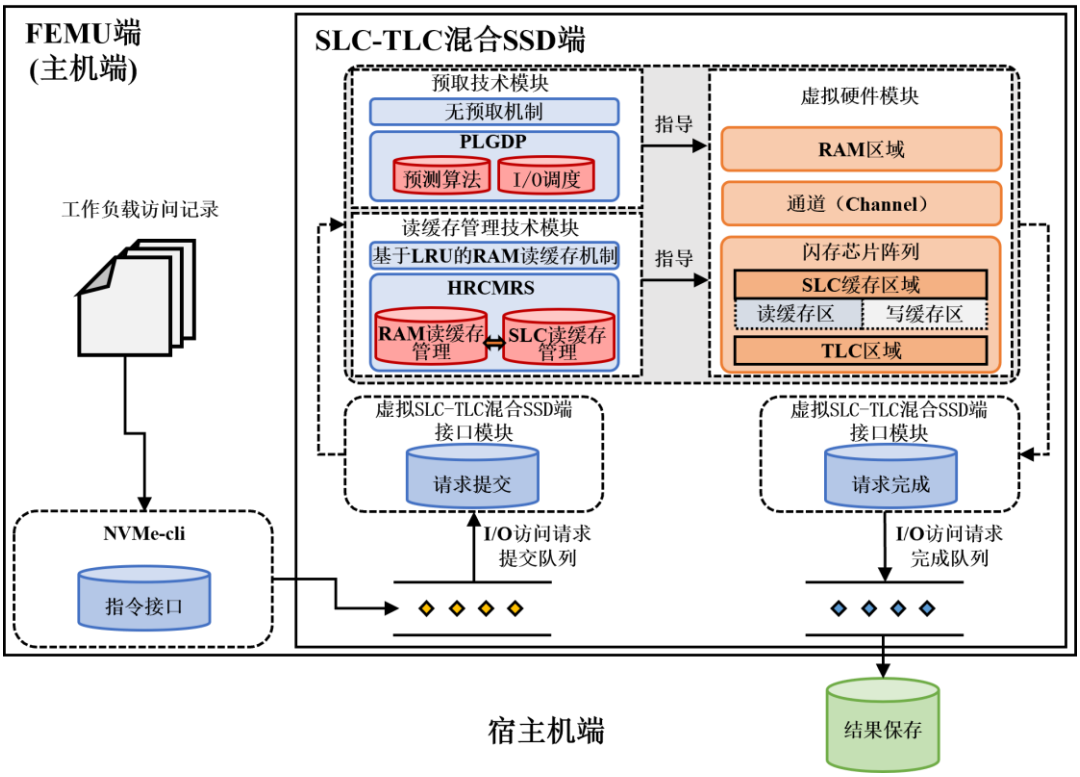


图 4.10 实验平台框架示意图

4.4.2 实验环境

本实验依然基于 FEMU[81][82]在真是物理机上进行，其软硬件配置信息如第三章表 3.3 所示。与此同时，工作负载依然是第三章所使用的剑桥微软研究院公开 15 条工作负载 I/O 块访问跟踪记录[79]，在本小节就不再做过多赘述。由于读缓存管理技术的引入，以及虚拟硬件模块的修改，原有的混合 SSD 模拟器参数配置发生了变化，具体修改如表 4.2 所示。其中，混合 SSD 中各个页面类型的读写延迟时间依然没有变化，采用与第三章表 3.4 和本章表 4.1 一致的延迟时间；混合 SSD 的体系结构仍然是采用平面级混合的方式，且采用 8 通道、每个通道内连接 4 个闪存芯片、每个闪存芯片包含 4 个晶片、每个晶片内包含一个平面。平面内则既包含 SLC 闪存块又包含 TLC 闪存块，具体块数由 SLC 区域空间大小动态决定。页面大小则设置 8KB，若访问数据大小小于 8KB 则依旧申请一个 8KB 页面进行操作。

SLC 读缓存区域大小（新添加的参数）设置为 1/10TLC 区域空间，SLC 读

中国知网 https://www.cnki.net

缓存区则是从 SLC 缓存区域的物理空间划分出 1/100 读页面数的物理空间（各个工作负载的读页面数可以参考表 3.1），这个设置的用意在于尽量不让 SLC 读缓存区的空间大小覆盖掉整体工作负载的读页面空间，避免出现只使用 SLC 缓存区的情况，进而呈现出符合实际应用场景中 SLC 和 TLC 均被使用的场景，且 TLC 作为主存储区。

表 4.2 SLC-TLC 混合 SSD 模拟器参数配置

参数名称		数值大小	参数名称		数值大小
闪存类型		SLC/TLC	体系架构	混合方式	平面级
操作延迟	读（页）	$40\mu s$(SLC)		通道数	8
		$56.5\mu s$(LSB)		闪存芯片数/通道	4
		$77.5\mu s$(CSB)		晶片数/闪存芯片	4
		$106\mu s$(MSB)		平面数/晶片	1
	写（页）	$800\mu s$(SLC)		页面大小	8KB
		$820.5\mu s$(LSB)		GC 阈值	40%(常规)
		$2225\mu s$(CSB)			10%（高压）
		$5734\mu s$(MSB)		TLC 区域空间	1.5 访问空间
	擦除（块）	$2ms$(SLC)		SLC 区域空间	1/10TLC 区域空间
		$3ms$(TLC)		RAM 读区域空间	1/1000读页面数
	通道（页）	$30\mu s$	预取参数	时间窗口	1000
	RAM（页）	$0.5\mu s$(读)		最大预取大小	20 页（160KB）
		$2\mu s$(写)	缓存参数	SLC 读缓存区空间	1/100读页面数

最后，为了印证本文提出的基于 RAM 和 SLC 的混合读缓存管理（HRCMRS）技术更加适用于添加有预取技术的混合 SSD，本实验采用的对照组包括无预取机制且采用基于 LRU 的 RAM 管理策略的混合 SSD（下文简称 Baseline）、采用 PLGDP 技术且采用基于 LRU 的 RAM 管理策略的混合 SSD（下文简称 PLGDP）、无预取机制且采用 HRCMRS 技术的混合 SSD(下文简称 HRCMRS)、采用 PLGDP 技术且采用 HRCMRS 技术的混合 SSD（下文简称 PLGDP+HRCMRS）。这里需要注意的是，HRCMRS 在没有引入预取技术时，预取数据的访问感知就不存在了，所以 HRCMRS 在实验过程中自动退化成 SLC 读缓存区域缓存 RAM 内剔除的高延迟页面（CSB 页面和 MSB 页面）[78]。

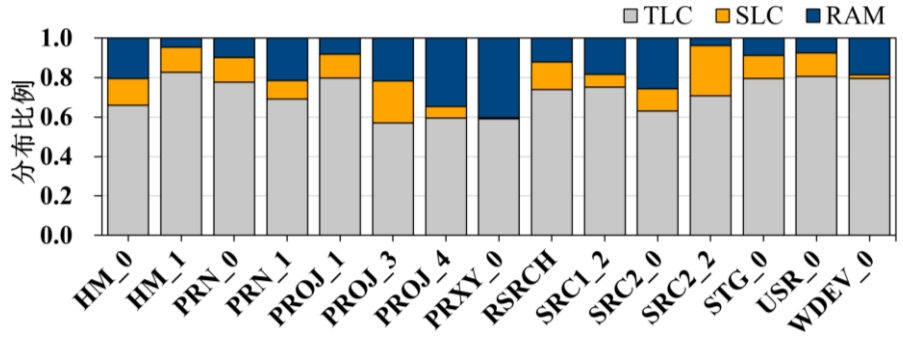
4.4.3 实验结果与分析

4.4.3.1 读数据来源分布情况分析

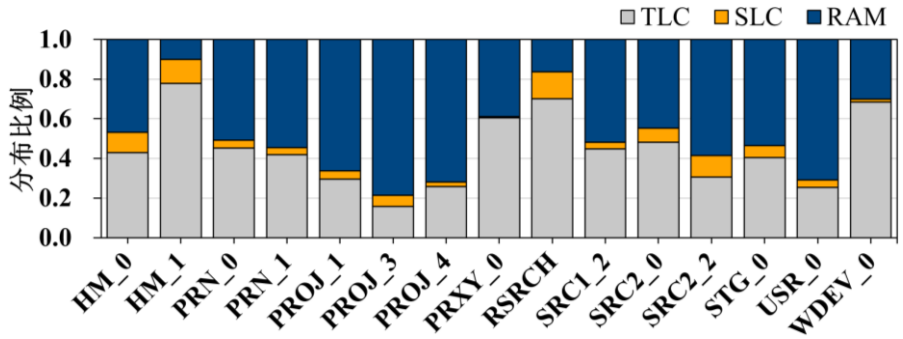
本小节旨在印证 HRCMRS 技术在提升 RAM 读数据比例以及 SLC 读数据比例相较于其他方法具有一定优势。高性能介质，例如 RAM 和 SLC，读数据比例

中国知网 https://www.cnki.net

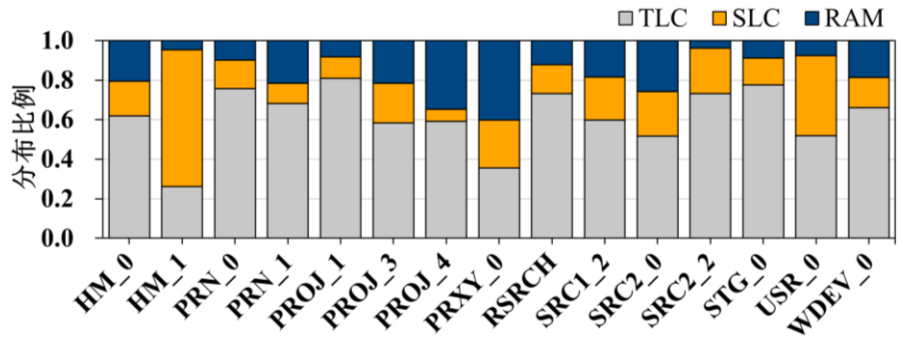
越多，说明读缓存管理效果越好，混合 SSD 读性能越强。在实验过程中，FEMU 端用户通过按照工作负载记录的时间间隔向 SLC-TLC 混合 SSD 发送 NVMe 的 I/O 访问请求，混合 SSD 会记录第一时刻响应该读访问请求的数据来源。



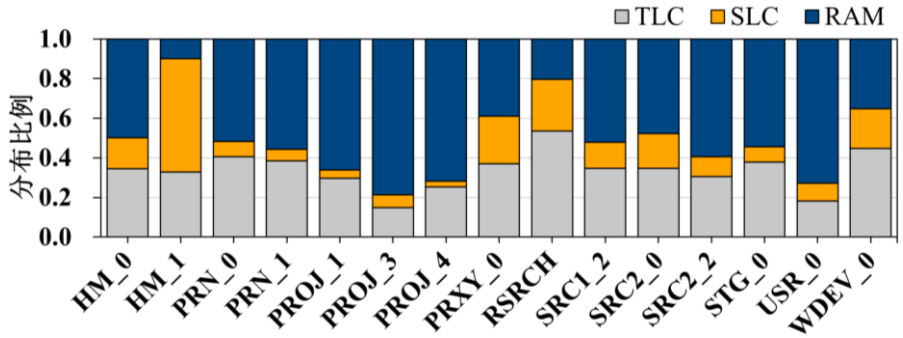
(a) Baseline



(b) PLGDP



(c) HRCMRS



(d) PLGDP+HRCMRS

图 4.11 不同对照组混合 SSD 读数据来源分布情况

图 4.11 展示了不同对照组下混合 SSD 读数据来源分布情况，其中蓝色部分

中国知网 https://www.cnki.net

表示来源于 RAM 的读数据比例，黄色部分表示来源于 SLC 的读数据比例，灰色部分表示来源于 TLC 的读数据比例。Baseline、PLGDP、HRCMRS 以及 PLGDP+HRCMRS 对比项平均 RAM 读数据比例分别为 17.068%、49.531%、17.068%、50.954%；平均 SLC 读数据比例分别是 11.348%、5.947%、21.629%、15.155%；平均 SLC 读数据比例分别是 71.548%、44.522%、61.303%、33.891%。

理论上讲，得益于 PLGDP 技术的时空感知能力，从 RAM 区域读取的数据比例应该会显著增加。然而，由于 RAM 区域有限的空间，未来本应该被访问的数据与其他读取请求或预取数据发生拥挤，导致提前从 RAM 中驱逐。这导致部分预取数据或有价值的读缓存数据被浪费，从图 4.11（b）中依然可以看到 TLC 读数据比例依然很高。与此同时，单一采用 HRCMRS 技术，只能提升 SLC 缓存区域的读数据比例。因为 HRCMRS 技术中的 RAM 区域的缓存管理技术是针对预取数据与读访问数据间的管理，但是当不采用预取技术时，其就变成了普通 LRU 管理策略，因此，可以看出 Baseline 与 HRCMRS 的 RAM 读数据比例相同。而 HRCMRS 技术中的 SLC 缓存管理技术，在不引入预取技术时退化成了只是缓存高延迟页面（CSB，MSB）的 FIFO 策略，从图 4.11（c）中可以看到 SLC 读数据比例对比 Baseline 依然具有提升效果。

当 HRCMRS 与预取技术（例如 PLGDP）相结合时，其优秀的读缓存管理性能才会得到真正的凸显，从图 4.11（d）中可以看到，其二者相结合可以实现 RAM 和 SLC 读数据比例之和可以达到平均 66.11%，而其他对比项的值依次为 28.42%、55.48%、38.70%。

由此可见，HRCMRS 技术更加适用于基于预取技术的混合 SSD 读缓存管理，不仅可以更加合理的管理 RAM 区域的复杂类型数据，还可以借助 SLC 来二次利用预取数据和读缓存数据，进而同时提高 RAM 和 SLC 读数据比例。

4.4.3.2 预取数据利用率分析

本小节主要分析采用 PLGDP 技术的混合 SSD 在结合 HRCMRS 技术前后预取数据利用率的对比情况，在先前图 4.5 中可以看到，在引入 PLGDP 技术后，在部分工作负载中存在预取数据利用率低下现象，即数据预取到 RAM 区域中，但没有被访问就被踢出了。为了印证 HRCMRS 技术可以缓解此类问题，提高预取数据的利用率，在实验过程中，用户依照工作负载记录向混合 SSD 发送 I/O 访问请求，在此期间，混合 SSD 会跟踪 PLGDP 技术预取的数据，这些数据可能会在 RAM 和 SLC 区域内缓存，并会被未来读访问请求所访问，本实验通过记录预取数据中被访问的比例来判断预取数据的利用率。比例越高，说明，HRCMRS 技术可以有效提高基于预取技术的混合 SSD 中预取数据的利用率，达到提高混合 SSD 读性能的目的。

中国知网 https://www.cnki.net

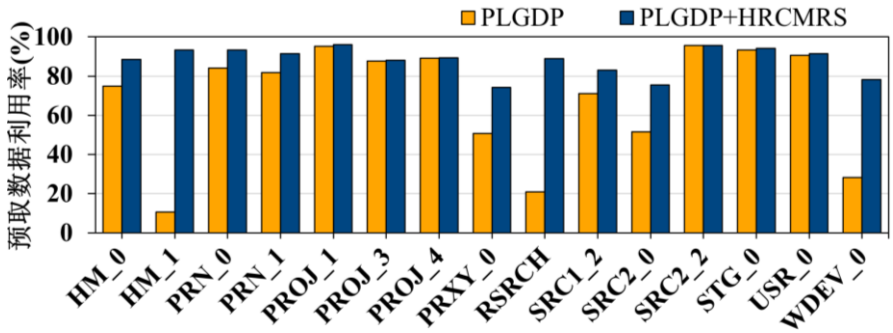


图 4.12 采用 HRCMRS 技术前后 PLGDP 技术的预取数据命中率情况

图 4.12 展示了采用 HRCMRS 技术前后 PLGDP 技术的预取数据命中率情况。实验结果表明，由于 HRCMRS 技术中对于 RAM 读缓存区域不同类型数据进行感知管理以及对于 SLC 读缓存区域的接收并组织 RAM 剔除的数据，可以有效提升不同工作负载中的预取数据利用率，平均提高 19.72%，其中在 HM_1、RSRCH、WDEV 等工作负载中提升效果明显，这在后续的读性能分析中仍可以凸显出其效果。

因此，从总体来看，HRCMRS 技术可以有效引入预取技术混合 SSD 的预取数据利用率，进而可以提高混合 SSD 的读性能。

4.4.3.3 写放大情况分析

本小节对读缓存管理技术的写放大情况进行了分析，写放大指的是由缓存操作造成的 SLC 区域或 TLC 区域闪存写入页面数量增加。为了论证本文提出的 HRCMRS 技术在 SLC 读缓存区管理方面，通过感知页面访问过程，更精准的过滤掉不必要的页面，减少 SLC 缓存区域因不必要缓存引起的写入页面数量增加。

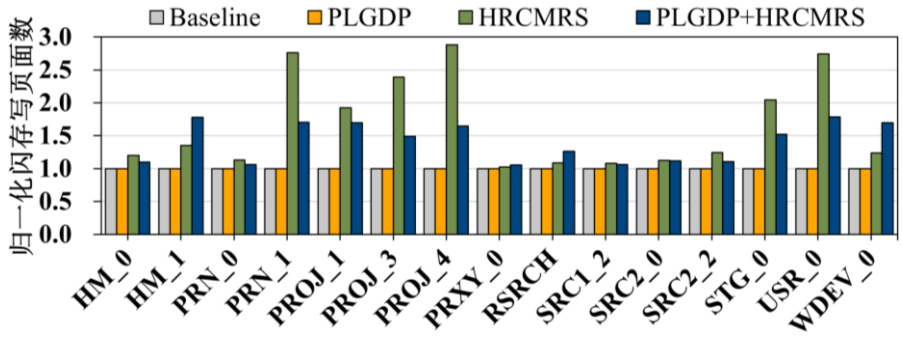


图 4.13 不同对照组混合 SSD 页面写入数量归一化对比

在实验过程中，混合 SSD 在执行发送过来的读访问请求过程中，监测 SSD 内部所有用于闪存写入数据的页数（引发写入的原因包括写访问请求、GC、SLC 读缓存管理、数据迁移）。图 4.13 展示了不同对照组混合 SSD 页面写入数量归一化对比。图中 Baseline 被归一化成 1.0，代表不引入任何多余的缓存策略造成写放大的情况。由于 PLGDP 对比项仅仅是使用了预取技术，而没有使用多余的缓存技术，所以它的页面写入数量与 Baseline 相同。与 Baseline 相比，HRCMRS 和

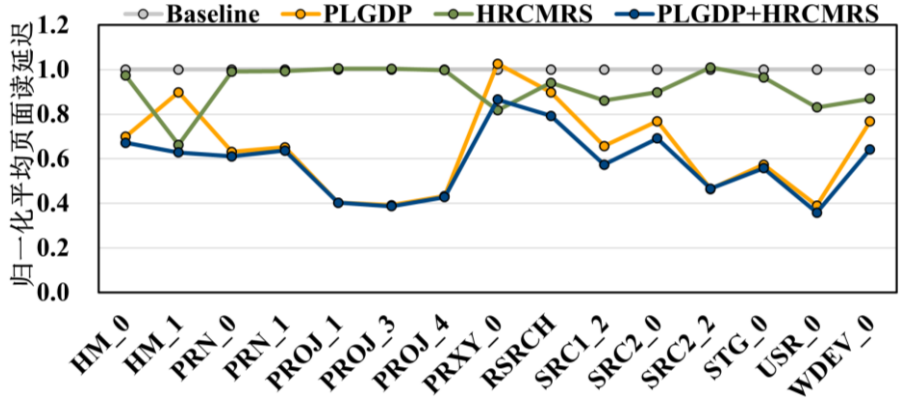
中国知网 https://www.cnki.net

PLGDP+HRCMRS 分别平均写放大了 68.06%、40.69%。这说明，单一使用 HRCMRS 存在写放大过多的突出问题，而采用 PLGDP 技术与 HRCMRS 技术相结合，HRCMRS 技术中 RAM 区域的缓存管理通过感知预取类型与读访问类型数据，尽最大可能缓存未来大概率被访问的数据，避免了页面在 RAM 与 SLC 之间频繁迁移，进而减少了写放大量并延长 SLC 缓存区域使用寿命。

4.4.3.4 混合 SSD 读写性能分析

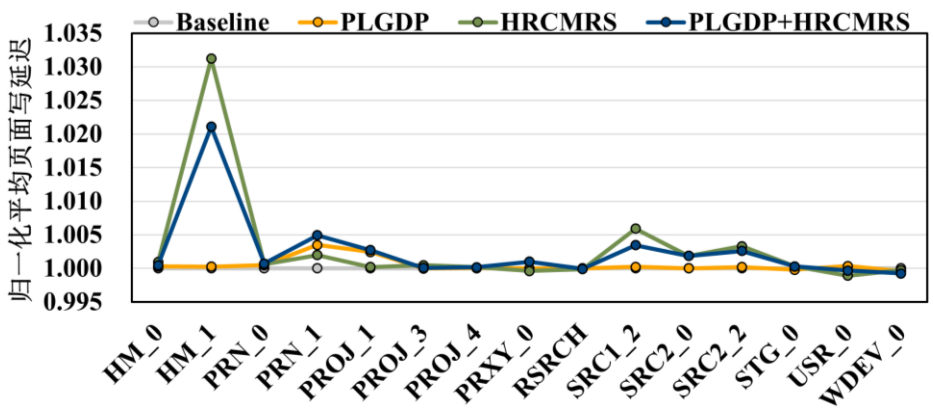
在本小节中，分析了引入读缓存管理技术前后混合 SSD 的读写性能，以深入理解 HRCMRS 对于存储系统性能的实际影响。正如之前讨论的，读性能的提升和写性能的影响是衡量存储优化技术效果的两个核心指标。这两个指标不仅反映了技术改进对于数据访问速度的提升，也关乎写操作的效率，进而影响存储系统的平衡与性能。更重要的是，读写性能的提升直接关联到用户的使用体验，包括数据访问的响应时间、系统运行的流畅度以及在高负载情况下的稳定性等方面。

在实验过程中，混合 SSD 记录每一个访问请求的起始时间、结束时间以及访问涉及的页面数，并通过计算该请求的页面平均读写延迟。实验以 Baseline 作为基准进行归一化计算，如图 3.16 展示了实验得到的各个工作负载在不同对照组混合 SSD 页面平均读写访问延迟。相较于 Baseline 页面平均读延迟，PLGDP、HRCMRS 和 PLGDP+HRCMRS 分别平均减少读延迟 35.69%、7.89%、41.93%。与此同时，PLGDP+HRCMRS 作为具有 RAM+SLC 读数据来源最高的对比项，相较于 Baseline，PLGDP 和 HRCMRS 对比项，具有明显的页面读取速度优势，平均页面读延迟减少了 41.93%、7.90%、35.62%，如图 4.14（a）所示。这说明，HRCMRS 技术在与预取技术相结合可以发挥出一加一大于二的效果，大大提高混合 SSD 读性能，这是由于 HRCMRS 在已有 PLGDP 技术基础上，对 RAM 内复杂类型数据进行了更加细致管理，与此同时，利用 SLC 读缓存区缓存 RAM 内剔除出来的高延迟页面，以应对后续的读访问请求。



(a) 页面平均读延迟

中国知网 https://www.cnki.net



(b) 页面平均写延迟

图 4.14 不同对照组混合 SSD 页面平均读写访问延迟归一化对比

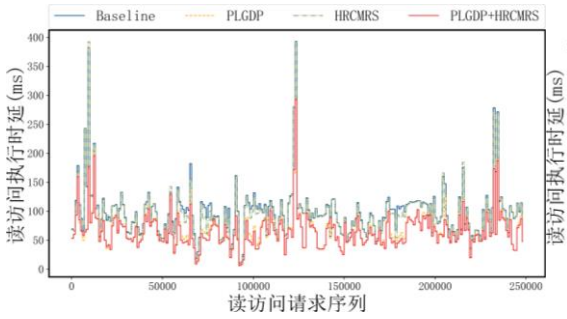
如实验结果图 4.14（b）所示，由于 SLC 读缓存区缓存 RAM 剔除的高延迟页面，PLGDP+HRCMRS 写入 SLC 和 TLC 闪存区的数量会增加。相较于 Baseline 页面平均写入延迟，PLGDP，HRCMRS 以及 PLGDP+HRCMRS 分别平均增加了 0.0465%、0.2995%、0.2538%。相较于 HRCMRS 对比项，PLGDP+HRCMRS 尽可能地降低了写入放大，进而减少了对于写入操作的影响，从图中可以看到，写性能的增加最多也不会超过 2.5%。在大大提高混合 SSD 读性能的情况下，PLGDP+HRCMRS 写性能的减少量是在一个可以容忍的范围内。这表明了在混合 SSD 的设计和优化中，通过巧妙地结合预取技术和读缓存管理策略，可以在确保读性能显著提高的同时，最大限度地减少对写性能的负面影响。

为了清晰展示出 PLGDP+HRCMRS 在读访问中所带来的巨大提升，图 4.15 展示了 15 条工作负载在执行过程中实时时间段内的读访问请求执行时延。在图 4.15 各个子图中，横坐标代表请求序列号，纵坐标代表以毫米（ms）为单位的请求执行时延，不同颜色线代表不同对照组混合 SSD 在工作负载下的实时读访问请求执行时延，其中红色实线代表 PLGDP+HRCMRS，绿色点横线代表 HRCMRS，橙色虚线代表 PLGDP，蓝色实现代表 Baseline。从图中可以观察到，采用 PLGDP+HRCMRS 组合的 SLT-TLC 混合 SSD 的读访问请求执行时延总体上低于其他三个对比项。其中相较于采用单一预取技术的 PLGDP 相比，读访问延迟降低效果最明显体现在 HM_0、PRXY_0、RSRCH、SRC1_2 以及 WDEV_0 工作负载中，这与图 4.14（a）中 PLGDP+HRCMRS 相较于 PLGDP 平均页面延迟减少情况相符合。

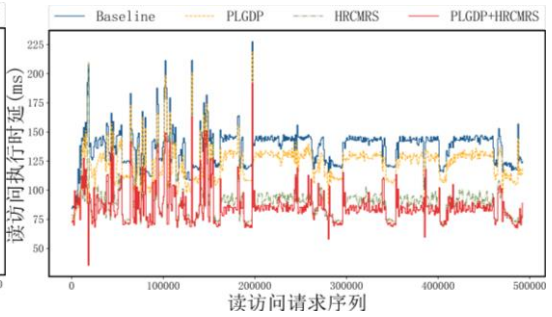
由此可见，这种性能的显著提升不仅体现了 PLGDP+HRCMRS 技术组合在优化读访问性能方面的强大能力，也凸显了其在不同类型和强度的工作负载下，均能保持较高效率和响应速度的优点。这为混合 SSD 的设计和优化提供了有力的技术支持，同时也为提升用户的使用体验和满意度提供了重要保障。通过这种

中国知网 https://www.cnki.net

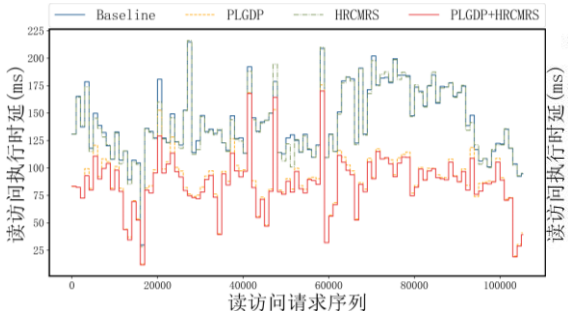
技术的应用，可以有效缩短数据访问时间，提高系统的整体性能，从而满足日益增长的数据处理需求。



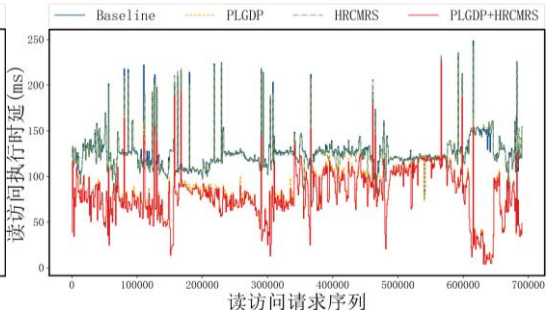
(a) HM_0



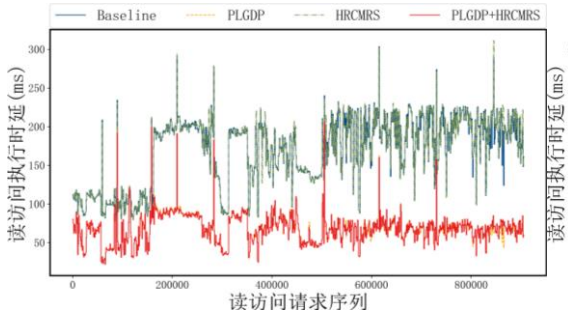
(b) HM_1



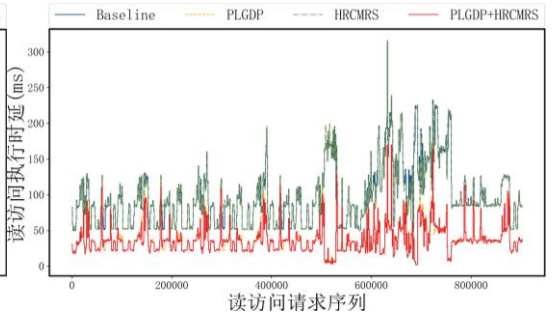
(c) PRN_0



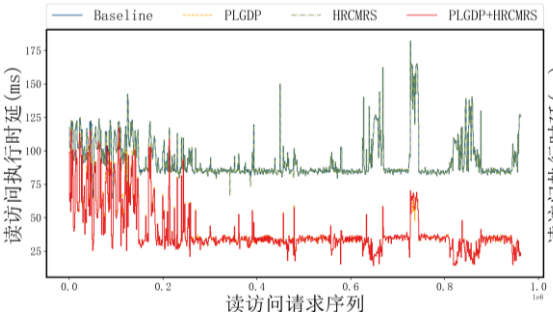
(d) PRN_1



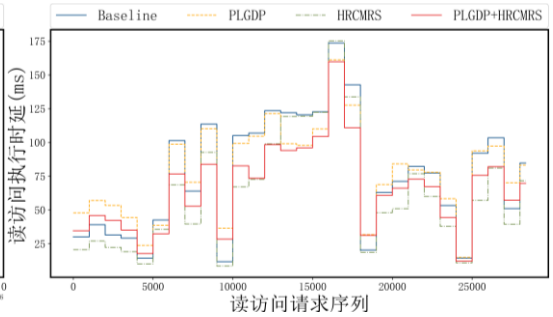
(e) PROJ_1



(f) PROJ_3

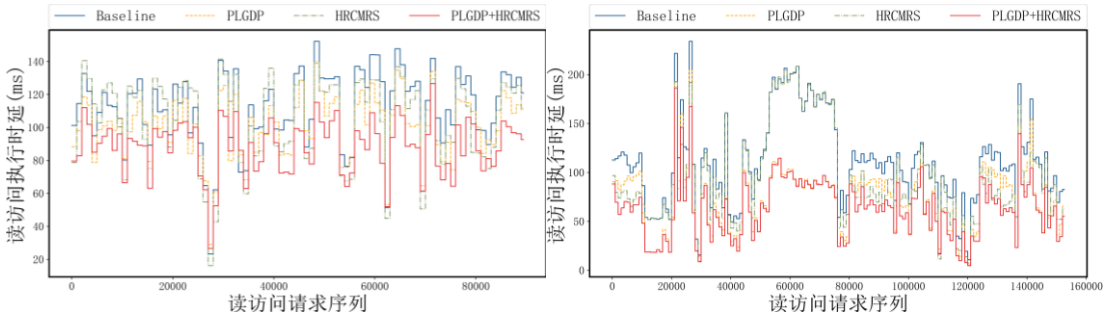


(g) PROJ_4

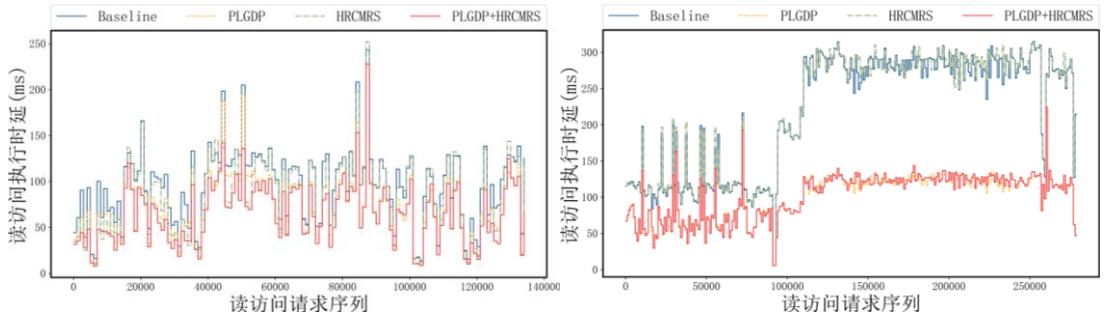


(h) PRXY_0

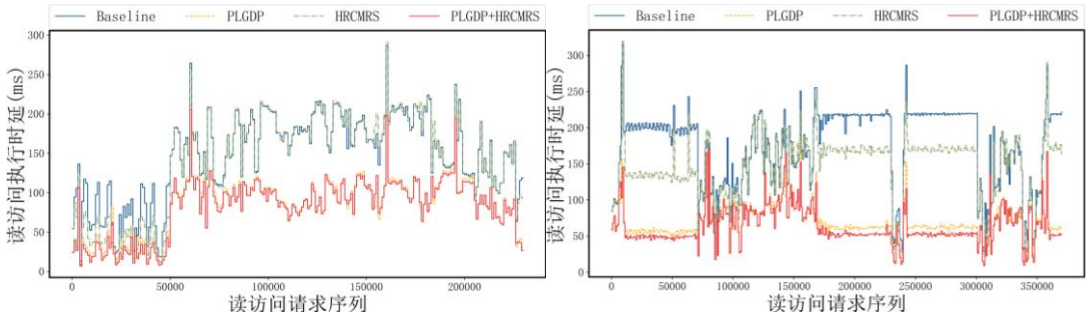
中国知网 https://www.cnki.net



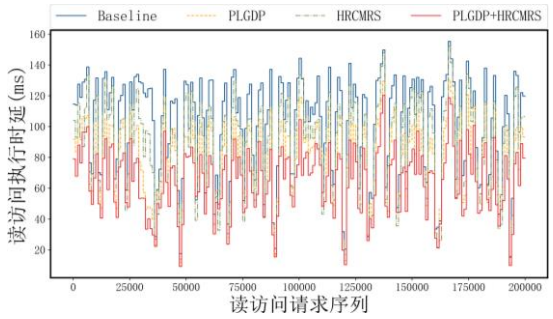
(i) RSRCH (j) SRC1_2



(k) SRC2_0 (l) SRC2_2



(m) STG_0 (n) USR_0



(o) WDEV_0

图 4.15 不同对照组混合 SSD 读访问请求执行期间时延对比

4.5 本章小结

本章深入探讨混合 SSD 中读缓存管理技术的研究，首先概述了在引入预取技术的混合 SSD 中读缓存管理技术的问题，并讨论了设计中的关键要素。接着，

中国知网 https://www.cnki.net

通过分析第三章中 Baseline 以及 PLGDP 技术的读数据来源分布和预取数据利用率，发现了其存在问题及具体成因。基于这些发现，提出了基于 RAM 和 SLC 的混合读缓存管理（HRCMRS）技术，包含基于访问感知的 RAM 读缓存管理策略和基于访问过程的 SLC 读缓存管理策略。前者负责管理 RAM 读缓存中复杂类型数据，以最大可能保留未来大概率会被访问的数据；后者负责筛选并缓存从 RAM 读缓存中剔除的高延迟页面，以充分利用预取数据和读缓存数据。为了合理印证 HRCMRS 技术对于采用预取技术的混合 SSD 具有良好的读性能提升效果，本章对于第三章中基于 FEMU 搭建的混合 SSD 模拟器平台进行了修改，并利用 nvme-cli 工具复现工作负载访问行为，对混合 SSD 进行各项指标的监测与统计。通过与无预取机制且采用基于 LRU 的 RAM 管理策略的混合 SSD（Baseline）、采用 PLGDP 技术且采用基于 LRU 的 RAM 管理策略的混合 SSD（PLGDP）以及无预取机制且采用 HRCMRS 技术的混合 SSD（HRCMRS)的对比表明，PLGDP 技术与 HRCMRS 技术相结合的混合 SSD（PLGDP+HRCMRS）其读性能分别提升了 41.93%、7.90%、35.62%。与此同时，预取数据利用率在 PLGDP 技术与 HRCMRS 技术结合前后提高了 19.72%。最后，PLGDP+HRCMRS 对写性能的影响也是微乎其微，写延迟增大比例均在 2.5%以下。

中国知网 https://www.cnki.net

第五章 总结与展望

5.1 本文工作总结

近年来，闪存技术以显著的进步，实现了存储单元容量的扩增和堆叠层数的增加，从而推动了高密度闪存存储容量的提升和单位容量成本的降低，有效应对了海量数据存储的需求。然而，高密度闪存也伴随着较高的延迟和较短的磨损寿命等挑战。为应对这些挑战，基于高密度闪存的混合 SSD 设备应运而生，通过将低密度闪存作为高密度闪存的写缓存区域，有效提升了固态硬盘的即时性能。

在大数据时代背景下，服务器场景日渐偏向读密集型。然而，传统的混合 SSD 研究主要集中在提升写性能，对读性能的优化较为缺乏。鉴于此，混合 SSD 的数据预取技术和读缓存管理技术面临两大优化需求：1）面对多变且动态的读访问行为，如何设计既准确又对系统影响最小的预取技术以增强混合 SSD 的读性能；2）对于采用了预取技术的混合 SSD，如何更有效地组织不同存储介质的数据，并使缓存管理技术与预取技术深度融合，最大化数据的利用价值，进一步提升读性能。

针对上述问题，本文提出了一套针对混合 SSD 的读性能优化方案，主要内容如下：

（1）为设计一种精确且最小化负面影响的数据预取技术，本文提出了基于模式局部性引导的数据预取（PLGDP）技术。该技术从工作负载读访问特征切入，实时地记录各个工作负载的读访问请求，并周期性地对历史访问记录进行地址空间分段，形成地址段集合。集合中每一个地址段内数据均具有较强的空间局部性，并且连续周期的地址段集合间的地址段具有时间局部性。结合以上特征，提出了基于地址分段的数据预测算法（组件一），利用周期更新的地址段集合指导预取请求的生成与请求队列分发。为减少预取请求对主机 I/O 访问的干扰，引入了基于预取请求感知的 I/O 请求调度策略（组件二），通过限制主机端 I/O 访问请求最大可被预取请求阻塞量来最小化负面影响。最后，为了验证 PLGDP 技术的有效性，搭建了一个基于 FEMU 模拟的 SLC-TLC 混合 SSD 平台，通过公开负载块访问记录来复现各个工作负载的访问行为，并与无预取机制（Baseline）、基于马尔科夫状态转移矩阵的预测机制（Lynx）和顺序预取机制（SP）进行对比。实验结果表明，PLGDP 相比于上述三种对比项，可以分别提高平均 RAM 区域命中率 335.92%、56.01%、68.81%，并分别提高了混合 SSD 平均整体读性能 35.69%、19.14%、29.58%。此外，PLGDP 的平均读放大情况也在相较于其他两个预取技

中国知网 https://www.cnki.net

术分别减少了 37.73%、75.79%。

（2）为了设计更加适配引入预取技术混合 SSD 的读缓存管理技术，本研究提出了基于 RAM 和 SLC 的混合读缓存管理（HRCMRS）技术，包括两个主要组件：基于访问感知的 RAM 读缓存管理策略（组件一），基于访问过程的 SLC 读缓存管理策略（组件二）。HRCMRS 技术通过修改原有的访问请求信息格式以及 RAM 内页面管理信息格式，增强对页面访问行为的感知能力。组件一依据页面访问请求决定其在 RAM 的保留或移至 SLC 读缓存，而组件二则根据页面在 RAM 的访问过程及其属性决定是否缓存至 SLC。为了验证 HRCMRS 技术的有效性，本文基于上一个实验平台进行进一步的改动，并与无预取机制且采用基于 LRU 的 RAM 管理策略的混合 SSD（Baseline）、采用 PLGDP 技术且采用基于 LRU 的 RAM 管理策略的混合 SSD（PLGDP）、无预取机制且采用 HRCMRS 技术的混合 SSD(HRCMRS)、采用 PLGDP 技术且采用 HRCMRS 技术的混合 SSD（PLGDP+HRCMRS）进行对比。实验结果表明，PLGDP+HRCMRS 相结合相较于上述三种对比项，可以分别提高 RAM 与 SLC 平均读数据比例之和 28.42%、55.48%、38.07%，并分别提高了混合 SSD 平均整体读性能 41.93%、7.90%、35.62%。此外，由于 SLC 读缓存区接受了部分预取数据，进一步提升预取数据的利用率，相较于只采用预取技术的 PLGDP，PLGDP+ HRCMRS 可以平均提升 19.72%预取数据利用率。

5.2 后续工作展望

本研究针对混合 SSD 中数据预取及读缓存管理的问题，提出了一种将预取技术与读缓存管理相融合的读性能优化方案，为后续混合 SSD 读性能优化的研究提供新思路和方向。尽管如此，当前的研究仍存在一定的局限性，未来的工作可在以下几个方面进行更深入的探索：

（1）引入段间跳转预测。在本研究框架下，数据预取仅在读访问请求触及地址段集合内某地址段后，对该地址段内剩余数据执行预取，导致预取行为具有一定滞后性，即预取操作需在地址段首次被访问后才激活，因此初次访问地址段时必然遭遇一次 RAM 缓存不命中。针对此问题，未来工作可以通过分析地址段间访问模式，基于历史地址段间访问记录预测未来可能被访问的地址段，以此优化 RAM 缓存命中率，从而提升混合 SSD 的读性能。

（2）更细粒度的访问预测。本研究在读访问请求触及某一地址段时，将整个地址段的剩余数据作为预测目标进行预取，这种方法由于预取数据的未来访问概率未知，可能导致预取资源的浪费。因此，通过深入分析段内数据的访问模式，并将其与段间跳转预测相结合，可以实现更精确的数据预测，从而缓解混合 SSD 的读放大效应，减少预取与主机请求之间的竞争，进一步优化混合 SSD 的读写

中国知网 https://www.cnki.net

性能。

（3）与深度学习相结合的预测算法。虽然本研究采用的是基于启发式的预测方法，但考虑到深度学习，特别是循环神经网络（RNN）、长短期记忆网络（LSTM）、Transformer 等在序列预测方面的潜力，将这些神经网络模型应用于基于地址段特征的读访问预测具有极大的研究前景。利用神经网络强大的关系捕获能力，可以更准确地预测未来的数据访问模式，为数据预取提供指导，从而显著提升混合 SSD 的读性能。

（4）更加复杂且精准的 SLC 读缓存管理策略。本研究中，SLC 读缓存管理策略对于接收何种从 RAM 剔除的页面还有很大的提升空间。在文章实验中，HRCMRS 技术与 PLGDP 相结合，依然存在比较大的读写放大现象。在未来的工作中，通过采用更加复杂的决策方法，准确地缓存更有利用价值的高延迟页面，以减少混合 SSD 都放大现象，进而提高混合 SSD 使用寿命和整体性能。

中国知网 https://www.cnki.net

参考文献

[1] 舒继武, 陈游旻, 汪庆等. 分离式数据中心的存储系统研究进展[J]. 中国科学: 信息科学, 2023, 53(08): 1503-1528.

[2] 魏嘉, 陈默, 王龙翔等. 数据密集型超算现状、挑战以及未来发展趋势[J]. 数据与计算发展前沿, 2023, 5(03): 66-91.

[3] Jeongdong Choe. Upcoming New 3D NAND TLC Devices: Samsung 176L & 238L, SK hynix 176L, KIOXIA/WD 162L[EB/OL]. [2022-07-05]. https://www.techinsights.com/blog/upcoming-new-3d-nand-tlc-devices-samsung-176l-238l-sk-hynix-176l-kioxiawd-162l.

[4] K. Smith. Using QLC SSDs to Improve Cost/Performance Tradeoffs for Warm Data[EB/OL]. [2019-08-06]. https://www.flashmemorysummit.com/Proceedings2019/08-06-Tuesday/20190806_SSDS-102-1_Smith.pdf.

[5] SAMSUNG. 870evoSSD[EB/OL]. [2024-03-13]. https://semiconductor.samsung.com/consumer-storage/internal-ssd/870evo.

[6] SAMSUNG. 870qvoSSD[EB/OL]. [2024-03-13]. https://semiconductor.samsung.com/consumer-storage/internal-ssd/870qvo.

[7] MICRON. Crucial P5 SSD[EB/OL]. [2024-03-13]. https://www.crucial.com/products/ssd/crucial-p5-ssd.

[8] Western Digital. Black SN850P SSD[EB/OL]. [2024-03-13]. https://www.westerndigital.com/en-sg/products/internal-drives/wd-black-sn850p-nvme-ssd-for-ps5?sku=WDBBYV0010BNC-WRSN.

[9] INTEL. Intel 670p[EB/OL]. [2024-03-13]. https://www.solidigmtech.com.cn/products/client/670p.html#quicklinks.

[10] SAMSUNG. 990pro SSD[EB/OL]. [2024-03-13]. https://www.samsung.com/sg/memory-storage/nvme-ssd/990-pro-1tb-nvme-pcie-gen-4-mz-v9p1t0bw/.

[11] Stoica R, Pletka R, Ioannou N, et al. Understanding the design trade-offs of hybrid flash controllers[C]. 2019 IEEE 27th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS). New York: IEEE, 2019: 152-164.

中国知网 https://www.cnki.net

[12]Chang L P. Hybrid solid-state disks: Combining heterogeneous NAND flash in large SSDs[C]. 2008 Asia and South Pacific Design Automation Conference. New York: IEEE, 2008: 428-433.

[13]Chang L P. A hybrid approach to NAND-flash-based solid-state disks[J]. IEEE Transactions on Computers, 2010, 59(10): 1337-1349.

[14]Im S, Shin D. ComboFTL: Improving performance and lifespan of MLC flash memory using SLC flash buffer[J]. Journal of Systems Architecture, 2010, 56(12): 641-653.

[15]Matsui C, Sun C, Takeuchi K. Design of hybrid SSDs with storage class memory and NAND flash memory[J]. Proceedings of the IEEE, 2017, 105(9): 1812-1821.

[16]Shibata N, Kanda K, Shimizu T, et al. A 1.33-Tb 4-bit/cell 3-D flash memory on a 96-word-line-layer technology[J]. IEEE Journal of Solid-State Circuits, 2019, 55(1): 178-188.

[17]Takai Y, Fukuchi M, Matsui C, et al. Analysis on Hybrid SSD Configuration with Emerging Non-Volatile Memories Including Quadruple-Level Cell (QLC) NAND Flash Memory and Various Types of Storage Class Memories (SCMs)[J]. IEICE Transactions on Electronics, 2020, 103(4): 171-180.

[18]PRODUCT BRIEF. Intel SSD 665p PCIe, 3D NAND[EB/OL]. [2024-03-13]. https://www.intel.com/content/dam/www/public/us/en/documents/product-briefs/665p-ssd-series-brief.pdf.

[19]The memory & storage experts. CRUCIAL P1 NVMe M.2 SSD[EB/OL]. [2024-03-13]. https://content.crucial.com/content/dam/crucial/ssd-products/p1/flyer/crucial-p1-nvme-m2-ssd-productflyer_zh-tw.pdf.

[20]Luo L, Li S, Lv Y, et al. Performance and reliability optimization for high-density flash-based hybrid SSDs[J]. Journal of Systems Architecture, 2023, 136: 102830.

[21]Chang D W, Lin W C, Chen H H. FastRead: Improving read performance for multilevel-cell flash memory[J]. IEEE Transactions on Very Large Scale Integration (VLSI) Systems, 2016, 24(9): 2998-3002.

[22]Shi L, Lv Y, Luo L, et al. Read latency variation aware performance optimization on high-density NAND flash based storage systems[J]. CCF Transactions on High Performance Computing, 2022, 4(3): 265-280.

[23]Xu R, Jin X, Tao L, et al. An efficient resource-optimized learning prefetcher for solid state drives[C]. 2018 Design, Automation & Test in Europe Conference &

中国知网 https://www.cnki.net

Exhibition (DATE). New York: IEEE, 2018: 273-276.

[24]Chakraborttii C, Litz H. Learning i/o access patterns to improve prefetching in ssds[C]. Joint European Conference on Machine Learning and Knowledge Discovery in Databases. Cham: Springer International Publishing, 2020: 427-443.

[25]Yang S P, Kim M, Nam S, et al. Overcoming the Memory Wall with CXL-Enabled SSDs[C]. 2023 USENIX Annual Technical Conference (USENIX ATC 23). 2023: 601-617.

[26]Park H, Lee E, Kim J, et al. Lightweight data lifetime classification using migration counts to improve performance and lifetime of flash-based SSDs[C]. Proceedings of the 12th ACM SIGOPS Asia-Pacific Workshop on Systems. 2021: 25-33.

[27]Zhou Y, Wu Q, Wu F, et al. Remap-SSD: Safely and Efficiently Exploiting SSD Address Remapping to Eliminate Duplicate Writes[C]. 19th USENIX Conference on File and Storage Technologies (FAST 21). 2021: 187-202.

[28]Cho H, Kim T. Data Placement Using a Classifier for SLC/QLC Hybrid SSDs[J]. Applied Sciences, 2024, 14(4): 1648.

[29]Yao Y, Fan J, Zhou J, et al. Hdftl: An on-demand flash translation layer algorithm for hybrid solid state drives[J]. IEEE Transactions on Consumer Electronics, 2021, 67(1): 50-57.

[30]Shi L, Luo L, Lv Y, et al. Understanding and optimizing hybrid ssd with high-density and low-cost flash memory[C]. 2021 IEEE 39th International Conference on Computer Design (ICCD). New York: IEEE, 2021: 236-243.

[31]Gao C, Shi L, Liu K, et al. Boosting the performance of SSDs via fully exploiting the plane level parallelism[J]. IEEE Transactions on Parallel and Distributed Systems, 2020, 31(9): 2185-2200.

[32]Zhang, Wenhui, et al. SPA-SSD: Exploit heterogeneity and parallelism of 3D SLC-TLC hybrid SSD to improve write performance[C]. 2019 IEEE 37th International Conference on Computer Design (ICCD). New York: IEEE, 2019.

[33]Lv, Yina, et al. Access Characteristic Guided Partition for NAND Flash based High-Density SSDs[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2023.

[34]Gupta, Aayush, Youngjae Kim, and Bhuvan Urgaonkar. DFTL: a flash translation layer employing demand-based selective caching of page-level address mappings[J]. Acm Sigplan Notices 44.3 (2009): 229-240.

[35]Choudhuri, Siddharth, and Tony Givargis. Performance improvement of block

中国知网 https://www.cnki.net

based NAND flash translation layer[C]. Proceedings of the 5th IEEE/ACM international conference on Hardware/software codesign and system synthesis. 2007.

[36]Lee, Sang-Won, et al. A log buffer-based flash translation layer using fully-associative sector translation[J]. ACM Transactions on Embedded Computing Systems (TECS) 6.3 (2007): 18-es.

[37]Chung, Tae-Sun, et al. System software for flash memory: A survey[J]. Embedded and Ubiquitous Computing: International Conference, EUC 2006, Seoul, Korea, August 1-4, 2006. Proceedings. Springer Berlin Heidelberg, 2006.

[38]Gal, Eran, and Sivan Toledo. Algorithms and data structures for flash memories[J]. ACM Computing Surveys (CSUR) 37.2 (2005): 138-163.

[39]Liu R, Chen X, Tan Y, et al. SSDKeeper: Self-adapting channel allocation to improve the performance of SSD devices[C]. 2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS). New York: IEEE, 2020: 966-975.

[40]Nadig R, Sadrosadati M, Mao H, et al. Venice: Improving Solid-State Drive Parallelism at Low Cost via Conflict-Free Accesses[C]. Proceedings of the 50th Annual International Symposium on Computer Architecture. 2023: 1-16.

[41]Pan, Yangyang, Guiqiang Dong, and Tong Zhang. Exploiting memory device Wear-Out dynamics to improve NAND flash memory system performance[C]. 9th USENIX Conference on File and Storage Technologies (FAST 11). 2011.

[42]Woo, Yeong-Jae, and Jin-Soo Kim. Diversifying wear index for MLC NAND flash memory to extend the lifetime of SSDs[C]. 2013 Proceedings of the International Conference on Embedded Software (EMSOFT). New York: IEEE, 2013.

[43]Song Y, Lv Y, Shi L. Adaptive Differential Wearing for Read Performance Optimization on High-Density NAND Flash Memory[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2023.

[44]Zhu R, Lei T, Bai P, et al. A Latency-Aware Garbage Collection Strategy[C]. 2022 IEEE International Conference on Advances in Electrical Engineering and Computer Applications (AEECA). New York: IEEE, 2022: 172-179.

[45]Kim, Hyojun, and Seongjun Ahn. BPLRU: A Buffer Management Scheme for Improving Random Writes in Flash Storage[C]. FAST. Vol. 8. 2008.

[46]Rosenfeld, Paul, Elliott Cooper-Balis, and Bruce Jacob. DRAMSim2: A cycle accurate memory system simulator[J]. IEEE computer architecture letters 10.1 (2011): 16-19.

中国知网 https://www.cnki.net

[47]Megiddo, Nimrod, and Dharmendra S. Modha. ARC: A Self-Tuning, low overhead replacement cache[C]. 2nd USENIX Conference on File and Storage Technologies (FAST 03). 2003.

[48]Shi, Liang, et al. ExLRU: A unified write buffer cache management for flash memory[C]. Proceedings of the ninth ACM international conference on Embedded software. 2011.

[49]Wei, Qian, et al. Reinforcement Learning-Assisted Management for Convertible SSDs[C]. 2023 60th ACM/IEEE Design Automation Conference (DAC). New York: IEEE, 2023.

[50]Kwon K, Kang D H, Eom Y I. An advanced SLC-buffering for TLC NAND flash-based storage[J]. IEEE Transactions on Consumer Electronics, 2017, 63(4): 459-466.

[51]Alsalibi, Ahmed Izzat, et al. A survey of techniques for architecting SLC/MLC/TLC hybrid Flash memory–based SSDs[J]. Concurrency and Computation: Practice and Experience 30.13 (2018): e4420.

[52]Li J, Xu X, Cai Z, et al. Pattern-based prefetching with adaptive cache management inside of solid-state drives[J]. ACM Transactions on Storage (TOS), 2022, 18(1): 1-25.

[53]万方祺. 边缘计算应用粒度的块级存储工作负载研究[D]. 杭州: 杭州电子科技大学, 2021.

[54]Xu X, Cai Z, Liao J, et al. Frequent access pattern-based prefetching inside of solid-state drives[C]. 2020 Design, Automation & Test in Europe Conference & Exhibition (DATE). New York: IEEE, 2020: 720-725.

[55]Wang H, Luo L, Shi L, et al. Sfp: Smart file-aware prefetching for flash based storage systems[C]. Proceedings of the 2021 on Great Lakes Symposium on VLSI. 2021: 45-50.

[56]Laga A, Boukhobza J, Koskas M, et al. Lynx: A learning linux prefetching mechanism for ssd performance model[C]. 2016 5th Non-Volatile Memory Systems and Applications Symposium (NVMSA). New York: IEEE, 2016: 1-6.

[57]Jung S, Lee H, Jo H. CluMP: Clustered Markov Chain for Storage I/O Prefetch[J]. Electronics, 2023, 12(15): 3293.

[58]LeCun Y, Bengio Y, Hinton G. Deep learning[J]. nature, 2015, 521(7553): 436-444.

[59]Li Y. Deep reinforcement learning: An overview[J]. arXiv preprint arXiv:1701.07274, 2017.

中国知网 https://www.cnki.net

[60]Ganfure G O, Wu C F, Chang Y H, et al. Deepprefetcher: A deep learning framework for data prefetching in flash storage devices[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2020, 39(11): 3311-3322.

[61]Akgun I U, Aydin A S, Shaikh A, et al. A machine learning framework to improve storage system performance[C]. Proceedings of the 13th ACM Workshop on Hot Topics in Storage and File Systems. 2021: 94-102.

[62]Li M, Wu C, Gao C, et al. RLAlloc: A Deep Reinforcement Learning-Assisted Resource Allocation Framework for Enhanced Both I/O Throughput and QoS Performance of Multi-Streamed SSDs[C]. 2023 60th ACM/IEEE Design Automation Conference (DAC). New York: IEEE, 2023: 1-6.

[63]Yang C, Man X, Shao J. G&l: An attention-based model for improving prefetching in solid-state drives[C]. 2023 International Joint Conference on Neural Networks (IJCNN). New York: IEEE, 2023: 1-8.

[64]Cui, Jinhua, et al. Exploiting latency variation for access conflict reduction of NAND flash memory[C]. 2016 32nd Symposium on Mass Storage Systems and Technologies (MSST). New York: IEEE, 2016.

[65]Li, Jun, et al. Mitigating negative impacts of read disturb in SSDs[J]. ACM Transactions on Design Automation of Electronic Systems (TODAES) 26.1 (2020): 1-24.

[66]Deng F, Cao Q, Wang S, et al. SeRW: Adaptively separating read and write upon SSDs of hybrid storage server in clouds[C]. Proceedings of the 49th International Conference on Parallel Processing. 2020: 1-11.

[67]Zhu J, Wang L, Xiao L, et al. CFIO: A conflict-free I/O mechanism to fully exploit internal parallelism for Open-Channel SSDs[J]. Journal of Systems Architecture, 2023, 135: 102803.

[68]Tripathy S, Satpathy M. SSD internal cache management policies: A survey[J]. Journal of Systems Architecture, 2022, 122: 102334.

[69]Jun Y, Park J, Kang J U, et al. Analysis and Mitigation of Patterned Read Collisions in Flash SSDs[J]. IEEE Access, 2022, 10: 96997-97009.

[70]Yoo S, Shin D. Reinforcement Learning-Based SLC Cache Technique for Enhancing SSD Write Performance[C]. 12th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 20). 2020.

[71]Choe G, Lee Y, Ro W W. Analysis of SSD with Logical to Physical Address

中国知网 https://www.cnki.net

Mapping of Hot Data to Single Level Cell Area[C]. 2022 37th International Technical Conference on Circuits/Systems, Computers and Communications (ITC-CSCC). New York: IEEE, 2022: 1-4.

[72]Li Q, Shi L, Di Y, et al. Process variation aware read performance improvement for LDPC-based NAND flash memory[J]. IEEE Transactions on Reliability, 2019, 69(1): 310-321.

[73]Lin H, Li J, Sha Z, et al. Adaptive Management with Request Granularity for DRAM Cache inside NAND-based SSDs[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2022.

[74]Long L, Huang J, Gao C, et al. ADAR: Application-specific Data Allocation and Reprogramming Optimization for 3D TLC Flash Memory[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2022.

[75]Zhu W, Tong W. Turn Waste Into Wealth: Alleviating Read/Write Interference in ZNS SSDs[C]. 2023 IEEE 41st International Conference on Computer Design (ICCD). New York: IEEE, 2023: 316-319.

[76]Chen Y C, Wu C F, Chang Y H, et al. ZoneLife: How to Utilize Data Lifetime Semantics to Make SSDs Smarter[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2022.

[77]Li S, Luo L, Lv Y, et al. Latency Aware Page Migration for Read Performance Optimization on Hybrid SSDs[C]. 2022 IEEE 11th Non-Volatile Memory Systems and Applications Symposium (NVMSA). New York: IEEE, 2022: 33-38.

[78]Chen Q, Wang S, Zhou Y, et al. PACA: A Page Type Aware Read Cache Scheme in QLC Flash-based SSDs[C]. 2022 IEEE 40th International Conference on Computer Design (ICCD). New York: IEEE, 2022: 59-66.

[79]Narayanan, Dushyanth, et al. Migrating server storage to SSDs: analysis of tradeoffs[C]. Proceedings of the 4th ACM European conference on Computer systems. 2009.

[80]Fengguang W U, Hongsheng X I, Chenfeng X U. On the design of a new linux readahead framework[J]. ACM SIGOPS Operating Systems Review, 2008, 42(5): 75-84.

[81]Li H, Tong D, Huang K, et al. FEMU: A firmware-based emulation framework for SoC verification[C]. Proceedings of the eighth IEEE/ACM/IFIP international conference on Hardware/software codesign and system synthesis. 2010: 257-266.

[82]Li H, Hao M, Tong M H, et al. The CASE of FEMU: Cheap, accurate, scalable and

中国知网 https://www.cnki.net

extensible flash emulator[C]. 16th USENIX Conference on File and Storage Technologies (FAST 18). 2018: 83-90.

[83]nvme-cli, https://github.com/linux-nvme/nvme-cli

[84]Zhang X, Zhu F, Li S, et al. Optimizing performance for open-channel ssds in cloud storage system[C]. 2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS). New York: IEEE, 2021: 902-911.

中国知网 https://www.cnki.net